ROBOTS MADE SIMPLE

SimpleRobot Programming Manual

Blocks and Simple Robot Script — concepts, instruction reference, functions and error codes

LANGUAGE
Simple Robot Script v1

STEP CATALOG
v1 · 58 instructions

BUILT
2026-11-07



Contents

PART I — CONCEPTS

1 Introduction 6
How this manual is organised 6
The system at a glance 6
Blocks and text 7
A first program 9
Conventions used in this manual 9
Units at a glance 10
2 Program Structure 11
Program kinds 11
Steps 11
Sections and notes 12
The layout of a text program 12
Names and ids 13
Saving, revisions and files 13
Program settings 13
3 Data and Variables 14
Declaring variables 14
Types 14
Modifiers 14
Initial values 15
Lists 15
Computed variables 16
Text in templates 16
Built-in values 16
4 Expressions and Conditions 18
What an expression can contain 18
Precedence 18
Truth and equality 19
Conditions 19
IO names 19
When an expression fails 20
5 Points, Frames and Tools 21
Positions 21
Saved points 21
Frames 21
Tools 22
Targets 22
Offsets, overrides and frames on a move 23
Grids and stacks 23
Saving the current position 23
6 Motion 24
Linear and joint moves 24
Speed, acceleration and deceleration 25
Blending 25
Arcs 26
Jumps 27
Stop conditions — move until 27
Homing and manual homing 28
Errors from motion 28
7 I/O, Waits and Aux Axes 29
Where I/O lives 29

Setting outputs 29
Waiting 30
Handshakes 31
Load cells 31
Aux axes, grippers, conveyors and rotary tables 32

8 Flow Control 33
If, elif, else 33
Switch 34
Loops 34
Labels and Go To 35
Ending and pausing 35
Disabled steps 36

9 Routines and Parameters 37
Making a routine 37
Parameters 37
Calling 37
Rules 38

10 Error Handling 39
Catchable and fatal codes 39
Try, retry and catch 39
How an error travels 41
Raising your own errors 41
Waits that fail 42
Which steps fail with which code 42

11 Running and Debugging 43
Checking a program before it runs 43
Program states 44
The monitor 45
Breakpoints 45
Step mode 46
Run from step 46
Debugging tips 46

12 Background Programs 47
Making and starting one 47
What runs in the background 47
Sharing data 47
Example 48

13 Vision 49
Vision programs 49
Run Vision 49
Coordinates 50
Using the results 50
Timing and failures 51
Images 51

14 Operator Prompts and Communication 52
Status messages 52
Asking the operator 52
HTTP 53
Exporting data 54

15 Plugins 55
Plugin steps 55
Plugin functions and properties 56

PART II — REFERENCE

16 Instruction Reference 58

Motion 59
Motion Settings 73
Logic & Flow 79
Variables & Timing 92
I/O & Aux Axes 98
Vision 111
Background 114
Communication 117
Plugins 122

17 Functions and Properties Reference 124
Operators 124
Notes on functions 124
Function list 124
Properties 126
IO names 128

18 Error and Problem Codes 129
Runtime error codes 129
Validation problems 130
Parse problems 131

APPENDICES

A Simple Robot Script Grammar 134
Lexical rules 134
Grammar 134
Expression grammar 135

B Keyword List 136
Statement keywords 136
Other reserved words 138

C Quick Reference Cards 139
Card 1 — Program skeleton 139
Card 2 — Motion 139
Card 3 — Flow 140
Card 4 — Data and I/O 140
Card 5 — Expressions 140
Card 6 — Error codes 141

Part I — Concepts

1 Introduction
2 Program Structure
3 Data and Variables
4 Expressions and Conditions
5 Points, Frames and Tools
6 Motion
7 I/O, Waits and Aux Axes
8 Flow Control
9 Routines and Parameters
10 Error Handling
11 Running and Debugging
12 Background Programs
13 Vision
14 Operator Prompts and Communication
15 Plugins

CHAPTER 1

Introduction

This manual describes how to program a SimpleRobot: the concepts behind a program, every instruction (step) you can use, the functions and properties available in expressions, and the error codes a program can raise. It covers both ways of writing a program — the **Blocks** editor and **Simple Robot Script**, its text form — because they are two views of the same thing.

How this manual is organised

Part	What it contains
Part I — Concepts (chapters 1–15)	How programs are built and run: structure, data, expressions, points and frames, motion, I/O, flow control, routines, error handling, debugging, background programs, vision, operator prompts and plugins. Read these in order the first time.
Part II — Reference (chapters 16–18)	One entry per instruction, in the same layout every time; the functions and properties you can use in expressions; every error and problem code. Use these to look things up.
Appendices	The text grammar, the keyword list and quick-reference cards.

The instruction reference, the function list and the error-code tables are generated from the controller's own step catalog, so they always match the controller the manual was built for.

The system at a glance

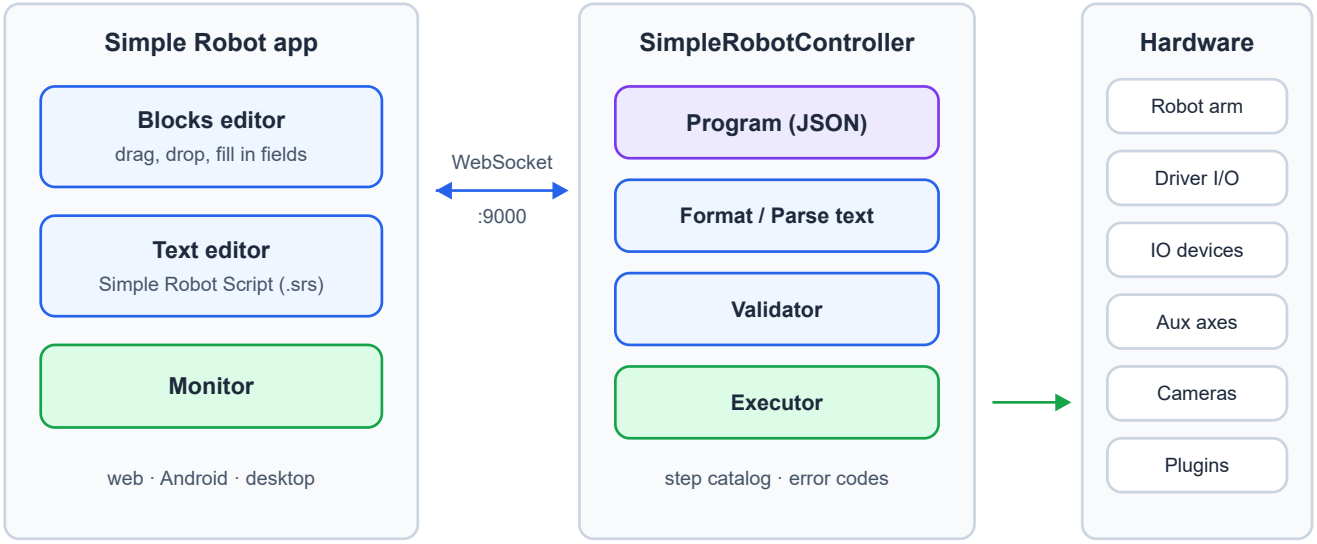


Figure 1. One program, two views: the app edits it, the controller stores, checks and runs it.

- **SimpleRobotController** runs on the computer attached to the robot. It stores programs, checks them, and runs them. It is the single source of truth: every app connected to it sees the same programs and the same live state.
- **The Simple Robot app** (web, Android and desktop) connects to the controller over the network. You write programs in it, run them, and watch them on the monitor.
- **The hardware** — the arm, the driver board's inputs and outputs, IO devices (such as Arduino Nano boards), auxiliary stepper axes (grippers, conveyors, rotary tables), cameras — and plugins are all reached through the controller.

Blocks and text

Every program can be edited as **Blocks** (a card per step, filled in with forms) or as **Text** (one line per step). Switch between them at any time with the **Blocks | Text** control at the top of the step list; nothing is lost in either direction.

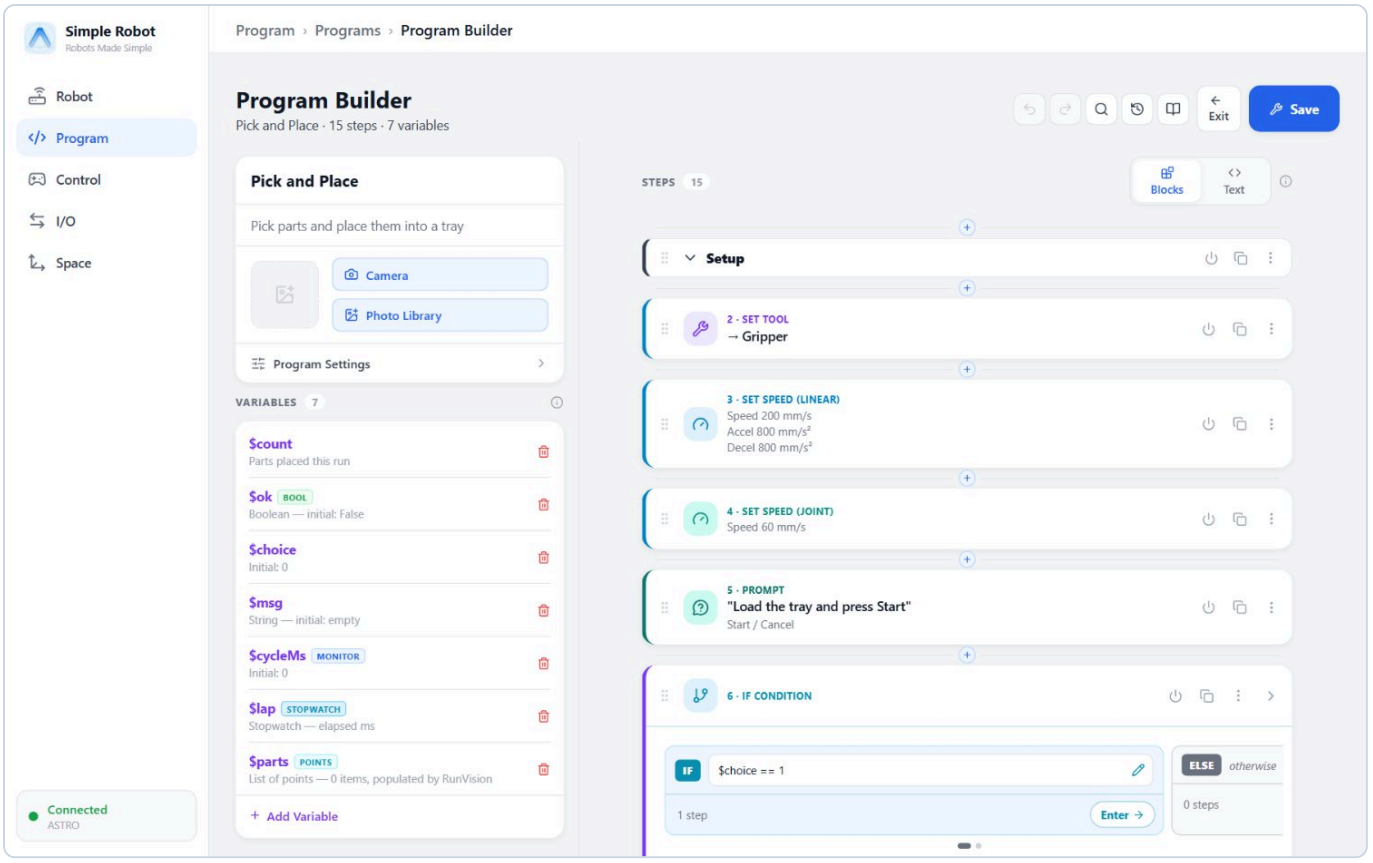


Figure 2. The Program Builder in Blocks view: program settings and variables on the left, the steps on the right.

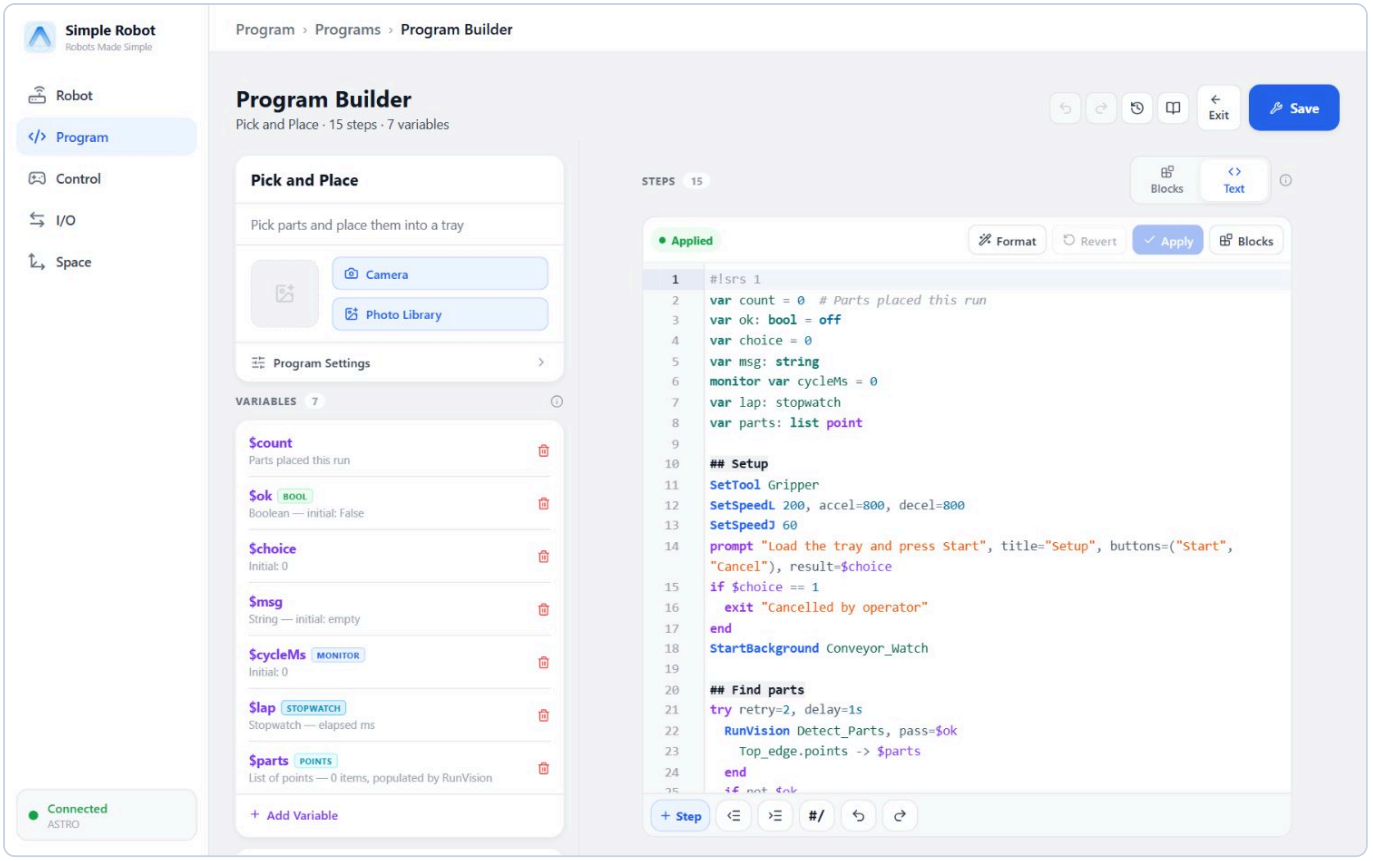


Figure 3. The same program in Text view (Simple Robot Script), with syntax colouring.

The text language uses the same names as the blocks (`MoveL`, `SetOutput`, `RunVision` …), so what you learn in one carries over to the other. Flow control and assignment use familiar lowercase words (`if`, `repeat`, `$x = …`).

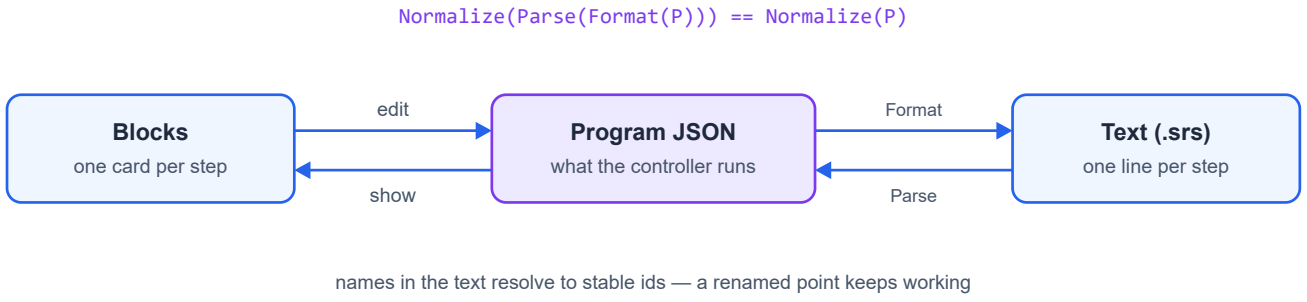


Figure 4. Blocks and text are two views of the same stored program. Switching views never loses a field or a step id.

A first program

```
#!srs 1
var count = 0              # Parts placed this run

## Setup
SetTool Gripper
SetSpeedL 200, accel=800, decel=800

## Pick and place
repeat 12 as $i
  JumpL stack Bins[$i], z=60
  GripperAction close
  JumpL grid Tray[$i], z=60
  GripperAction open
  $count = $count + 1
end
status "Placed {$count} parts"
```

Reading it from the top:

1. `var count = 0` declares a number variable. The comment after it becomes its description.
2. `## Setup` is a section header — it is not executed, it organises the program.
3. `SetTool Gripper` makes the gripper's tool frame active, so points are reached with the gripper tip. `SetSpeedL` sets the default linear speed for the moves that follow.
4. `repeat 12 as $i … end` runs its body twelve times with `$i` counting 0 to 11.
5. `JumpL` lifts, travels and lowers — here to a slot of the stack `Bins` and to a cell of the grid `Tray`. `GripperAction` closes and opens the gripper.
6. `$count = $count + 1` assigns a variable; `status "…"` shows a message on the monitor, with `{$count}` filled in.

Conventions used in this manual

Notation	Meaning
`MoveL <target>`	Angle brackets: a value you supply.
`[, speed=…]`	Square brackets: optional.
`a | b`	One of the alternatives.
`…`	More of the same, or a value you fill in.
`$name`	A variable. Variables are always written with `$`.
Move Linear (`MoveL`)	A step's name in the Blocks editor, then its keyword in Simple Robot Script.
mm, deg, mm/s, mm/s², ms	Units: positions in millimetres, angles in degrees, times in milliseconds.

> **NOTE**
> Positions are Cartesian: `x y z` in millimetres and `rx ry rz` in degrees. Times in step fields are in milliseconds; in text you can also write `500ms` or `1.5s`.

Units at a glance

Quantity	Unit
Position `x y z`	mm
Orientation `rx ry rz`	degrees
Linear speed / acceleration	mm/s, mm/s^2
Durations (Wait, timeouts, pulses)	ms (`500ms` , `2s` in text)
Load cells	grams in expressions; Read Load Cell can convert to kg, N, lb or oz
Angles in functions (`sin` , `atan2` …)	degrees, in and out

CHAPTER 2

Program Structure

A program is a **list of steps** plus the **variables** it declares. The controller runs the steps from top to bottom; flow-control steps (loops, branches, calls, jumps) change that order. Some steps have **bodies** — the steps inside a loop, the branches of an `if`, the body and catch of a `try` — so a program is really a tree of steps.

Program kinds

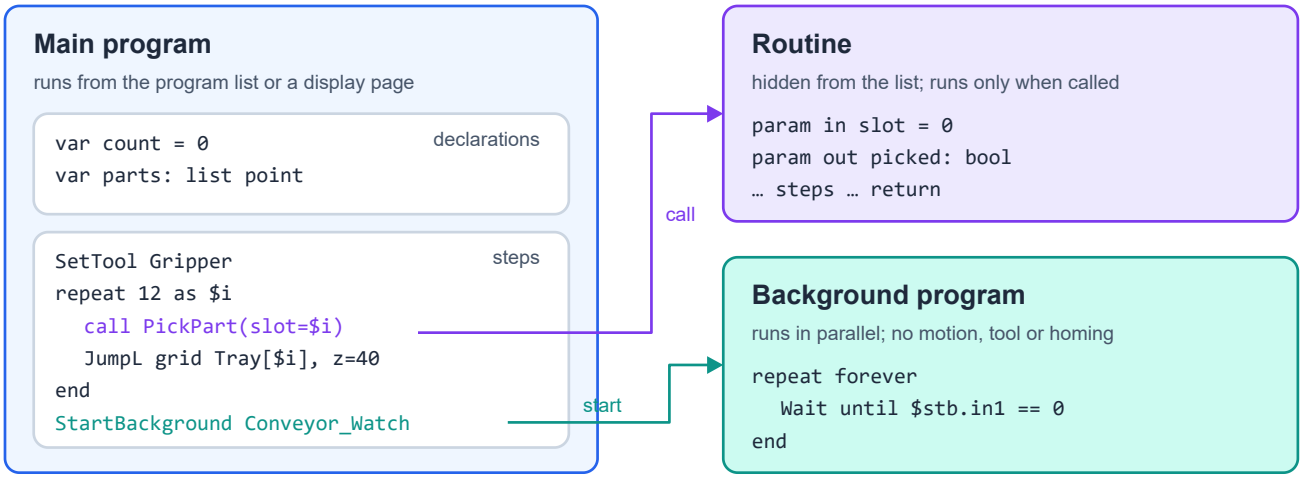


Figure 5. Program kinds: a main program calls routines and starts background programs.

Kind	Runs when	Notes
Program	started by the operator (program list, monitor, a display page or a station tag)	The normal case.
Routine	called by Call Routine (`call`) from another program	Hidden from the program list. Can take parameters — see Routines and Parameters.
Background program	started by Start Background (`StartBackground`)	Runs in parallel with the main program. Motion, tool and homing steps are skipped — see Background Programs.

The kind is set in **Program Settings**. In an exported `.srs` file it travels in the `@program {…}` header (`"isRoutine": true`, `"isBackground": true`).

Steps

Every step, whatever its type, carries the same few common properties:

Property	In Blocks	In text	Meaning
type	the card's kind	the keyword (`MoveL`, `if` …)	What the step does.
name	the card title (pencil icon)	`@name "…"` above the step	An optional label shown instead of the summary.
comment	the step's description	`# …` lines directly above the step, or at the end of its line	Free text, never executed.

Property	In Blocks	In text	Meaning
enabled	the power button on the card	`@off` above the step	A disabled step is skipped — and its whole body with it.
id	hidden	`@id "…"` (only when asked for)	A stable identifier. Breakpoints, the monitor and links between steps use it.

> **NOTE**
> A disabled step still counts in the step total and is logged as `[Skipped – disabled]` when the program passes it. The validator reports it once (`disabledStep` , a warning) and does not check anything inside it.

Sections and notes

A **Note** step is not executed and does not count as a step. It has two styles:

- **Section** — a header that groups the steps below it; the editor can collapse a section. In text: a line starting with `##` .
- **Note** — a paragraph of explanation. In text: a comment paragraph followed by a blank line, `end` , or the end of the file.

```
## Pick                    # a section header

# Check the tray first.    # a comment paragraph followed by a blank line:
# Then pick.               # a Note step

# Approach slowly          # comment lines directly above a statement: its comment
@name "Approach"           # the step's name
MoveL Pick1, speed=50
```

The layout of a text program

```
#!srs 1
@program {"name": "Pick and place", "isRoutine": false}   # only in exported files
var count = 0                                             # declarations first
var parts: list point
## Setup                                                  # then the steps
SetTool Gripper
```

Element	Rule
`#!srs 1`	The language version. The formatter writes it; the parser accepts text without it.
`@program {…}`	Name, kind and display page of an exported file. Read only when importing.
Declarations	`var` , `computed` and `param` lines. The formatter puts them first.
Statements	One per line. A line continues while a `(` , `[` or `{` is open, or after a trailing `,` .
Blocks	`if` , `repeat` , `while` , `for` , `switch` , `try` and the row bodies (`RunVision` , `HttpRequest` …) close with `end` .
Comments	`#` to the end of the line, outside strings.
Case	Keywords are case-insensitive; the formatter writes the canonical spelling and indents with two spaces.

The formatter is canonical: formatting a program, parsing it and formatting it again gives the same text. A step can carry fields the text has no words for (left over from an older editor, or a CNC toolpath); those travel in an `@with {…}` JSON line above the step. Leave `@with` lines as they are.

Names and ids

Programs refer to points, tools, local frames, grids, stacks, routines, devices, cameras and vision programs **by name in the editor, but by id in storage**. A renamed point keeps working, and the text shows its new name.

- A name that is a plain identifier is written bare: `MoveL Pick1`.
- Any other name can be written in quotes (`MoveL "Pick 1"`) or in its **underscore form**, where every run of characters an identifier cannot hold becomes one `_`: `"Pick 1"` → `Pick_1`, `"Arduino Nano"` → `Arduino_Nano`.
- Names match exactly first, then ignoring case, then by underscore form. If an underscore form fits two names (`"Pick 1"` and `"Pick-1"`), write the quoted name (`ambiguousName`).
- An unknown name is a **warning**, not an error: you can write a program before teaching its points. The validator keeps reporting it until the point exists.

Saving, revisions and files

- **Save** stores the program on the controller. Every save that changes the program first keeps the previous version as a **revision**; the newest 30 are kept. The history button in the editor lists them and can restore one (restoring is itself a new revision).
- A program can be exported and imported as a `.srs` text file. Importing creates fresh step ids.
- Parse errors block saving in Text view; validation errors do not block saving, they block running.

Program settings

Setting	Meaning
Name, description, image	Shown in the program list and on the monitor.
Kind	Program, routine or background program.
Display page	An optional operator page for the program (buttons, values, images). Variables marked `input` can be set from it at start.

CHAPTER 3

Data and Variables

Programs keep data in **variables**. A variable has a name, a type, an initial value and a few optional modifiers. In expressions and text it is always written with a `$`: `$count`.

Declaring variables

In Blocks, variables are listed and edited in the **Variables** panel. In text they are declared at the top of the program:

```
var count = 0                    # Parts placed      ← a trailing comment is the description
var ok: bool = off
var label: string = "Bin {$i}"
var startZ = $robot.z            # an expression → computed once at program start
var pts: list point = ((x=1, y=2, z=3), (x=4, rz=90))
var vals: list number = (1, 2.5, 3)
var flags: list bool = (on, off)
var cells: list record = ((row=0, col=1), (row=1, col=0))
var photo: image
var lap: stopwatch
computed area = $w * $h          # a formula, evaluated every time it is read
global persistent input monitor var runs = 0   # modifiers, in this order
param in height = 20             # routine parameters: in | out | inout
```

Types

Type	Text	Holds	Notes
Number	`number` (the default)	a floating-point number	
Boolean	`bool`	1 or 0	Shown as true / false. Any non-zero value counts as true.
Text	`string`	text	Assigned with a template in quotes: `$label = "Bin {$i}"`.
Image	`image`	a camera frame (JPEG)	Filled by Capture Image or a plugin; sent by HTTP Request or Export Data.
Stopwatch	`stopwatch`	elapsed milliseconds	Driven by the Stopwatch step.
List	`list number | bool | point | record`	a list of elements	See below.

NOTE

Expressions work with numbers. Booleans are stored as 1 / 0, so a comparison can be assigned straight to a boolean: `$ok = $grams > 100`. In an expression write `true / false`; the words `on / off` are for declarations and step options.

Modifiers

Word	Field	Meaning
`global`	`isGlobal`	Shared with every running program — see the figure below.

Word	Field	Meaning
`persistent`	`isPersistent`	Saved when the run finishes and restored at the next start.
`input`	`isInput`	A display page may set it when the program starts.
`monitor`	`displayOnMonitor`	Shown on the monitor while the program runs.
`computed`	`isComputed`	A formula instead of a stored value.
`param in / out / inout`	`paramMode`	A routine parameter.

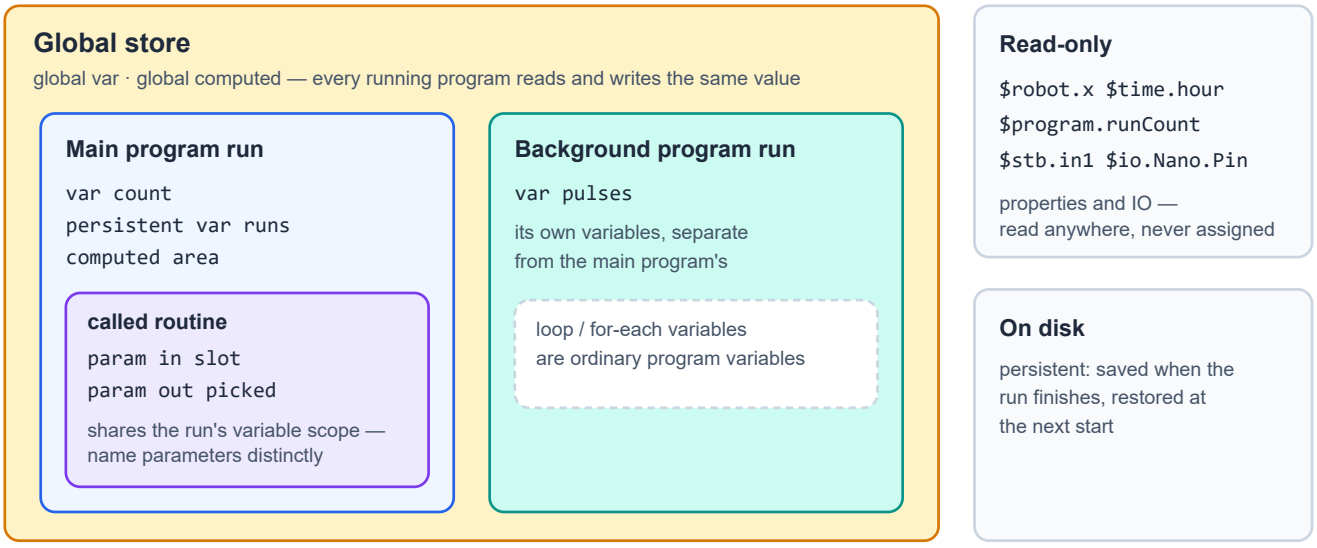


Figure 6. Where a variable lives: the global store is shared by every running program; a program's variables (and its routines' parameters) belong to its run.

Initial values

A number or boolean variable can start from an **expression** instead of a literal: `var startZ = $robot.z`. The expression is evaluated **once when the program starts** (and again each time a routine is entered), in declaration order — it can read variables declared *above* it, plus IO and properties.

Lists

There is one list type; the **element type** says what the elements are.

Element type	One element	Read as	Typical source
Number	a number	`$vals[$i]`	typed in, List step, plugin
Boolean	1 / 0	`$flags[$i]`	typed in
Point	`x y z rx ry rz`	`$pts[$i].x`, or `$pts[$i][2]` for z	Run Vision `points`, Save Position, stop conditions
Record	named numeric fields	`$recs[$i].coverage`	Run Vision `cells` / `items`, recordings

- `$list.length` and `$list.count` give the number of elements; `len($list)` does the same.
- In an expression an index out of range, or an unknown field, reads as **0** rather than failing.
- Only a **Point** list can be a move target (`MoveL $pts[$i]`).
- Lists are changed with the List step: `append`, `insert`, `removeAt`, `clear`, set an element or field, `sort`, `reverse`, `copyFrom`.

```
var pts: list point
$pts.append(x=$robot.x, y=$robot.y, z=50)   # add an element
$pts.append()                               # append the current position
$pts[0].z = 30                              # set one field
for _, $i in $pts                           # walk it
  MoveL $pts[$i]
end
```

> **NOTE**
> A List step evaluates everything before it changes the list, so a failing step leaves the list as it was. The declared list itself is never changed: the next run starts from it again.

Computed variables

A **computed** variable is a named formula — a property you define. Its expression is evaluated **every time** it is read, against the live variables, IO and properties.

```
computed area = $w * $h
computed ready: bool = $area > 100 and $stb.in1
```

- It has no stored value and cannot be assigned (`computedVariable` , fatal).
- Formulas may use variables declared below them, and other computed variables — but not in a cycle (`computedCycle`).
- A `global computed` variable is registered by the program that declares it and readable from every program; its formula may only use global variables, IO and properties.
- It cannot also be `persistent` , a string, an image, a stopwatch or a list.

Text in templates

Text fields — status messages, prompts, file paths, a string variable's value — are **templates**: variable references inside them are filled in when the step runs.

Form	Result
`$name` or `{$name}`	The value. A list gives a count: `"3 points"` .
`{$name[expr]}`	One element: a number, `True` / `False` , `(x=…, y=…)` for a point.
`{$name[expr].z}`	One field of an element.
`{expr}`	Any expression: `{$index + 1}` , `{round($mm, 1)}` , `{$robot.z}` .

Braces let a reference sit against other text (`{$prefix}{$index}` → `bin3`) and are needed for properties and full expressions. Anything that does not resolve is left in the text as written.

Built-in values

Name	Value
`$time_ms`	Unix time in milliseconds — handy in file names: `"shots/{$time_ms}.jpg"` .

Name	Value
`$robot.*`, `$program.*`, `$time.*`, `$aux.*`, `$camera.*`	Read-only properties — see Functions and Properties.
`$stb.in1`, `$relay.3`, `$io.<device>.<pin>`	Live IO values.

Properties and IO names can be read anywhere but never assigned (`readOnlyProperty`). A variable of the same name wins over a property.

CHAPTER 4

Expressions and Conditions

Almost every numeric field of a step can be an **expression** instead of a plain number — a speed, an offset, a count, an index. In Blocks, tap the **fx** toggle on a field; in text, write the expression after `name=` or in parentheses in a positional slot:

```
MoveL Pick1, speed=$base * 2, offset=(z=$layer * 5)
SetSpeedL ($base * 2)          # positional values need parentheses
Wait ($delay)
```

Expressions are evaluated when the step runs, so they always use the current values.

What an expression can contain

Element	Examples
Numbers	`3.14` `-5` `1e3`
Booleans	`true` `false` (1 and 0)
Variables	`$count`, `$pts[$i].x`, `$vals[0]`, `$list.length`
IO	`$stb.in1` `$stb.out2` `$relay.3` `$io.Arduino_Nano.Input_1`
Properties	`$robot.z` `$program.runCount` `$time.hour`
Arithmetic	`+ - * / % ^` — `%` is the remainder, `^` is power; `/` and `%` by zero give 0
Comparison	`== != < <= > >=` — yield 1 or 0
Logic	`and or not` (also `&& || !`)
Conditional	`cond ? a : b`, or `if(cond, a, b)`
Functions	`abs(x)`, `round(x, 2)`, `min(a, b, …)`, `dist(x1, y1, x2, y2)`, `len($list)` …
Text helpers	`$s.contains("x")`, `$s.startsWith("x")`, `$s.indexOf("x")`, `$s.toNumber()`, `$s.length`
Plugin calls	`scale.tare()`, `$scale.weight`
Grouping	`( … )`

The full list of functions and properties is in Functions and Properties.

> **CAUTION**
> Variables need the `$`. A bare word such as `count` is not a lookup — it evaluates to **0**, and the validator warns about it. This also applies to `on` and `off` inside an expression: write `true` and `false` instead.

Precedence

Tightest first:

Level	Operators	Associativity
1	`^`	right (`2 ^ 3 ^ 2` is `2 ^ 9`)
2	unary `-`	— (`-2 ^ 2` is `-4`)

Level	Operators	Associativity
3	`*` `/` `%`	left
4	`+` `-`	left
5	`==` `!=` `<` `<=` `>` `>=`	left (`1 < 2 < 3` is `(1 < 2) < 3`)
6	`not`	— (`not $a > 5` is `not ($a > 5)`)
7	`and`	left
8	`or`	left
9	`? :`	right

So `$count > 5 and $count < 10` needs no parentheses, and neither does `$n > 0 ? $sum / $n : 0`.

Truth and equality

- Any non-zero value is **true**; comparisons and logic produce exactly 1 or 0.
- `==` compares within `1e-9`, so `0.1 + 0.2 == 0.3` holds.
- Both sides of `and` / `or` and both branches of `?:` are always evaluated. Nothing has side effects, and a misspelt variable fails the run whichever way the condition goes.

Conditions

`if`, `elif`, `while`, Wait until, stop conditions (`until=`) and computed booleans all take a **condition**, which is simply an expression read as true or false.

```
if $count > 5 and $stb.in1
  …
end
Wait until $io.Arduino_Nano.Input_1 == 1, timeout=5s
MoveL Pick1, until=$io.Arduino_Nano.Load_Cell >= 2000
```

In Blocks a condition is shown as **rows** (`left · operator · right`) joined by **ALL** (and) or **ANY** (or). The text form's top-level `and` / `or` become those rows, so both views stay in step.

IO names

Name	Value
`$stb.in1` … `$stb.in4`	Driver board inputs, 1 / 0
`$stb.out1` … `$stb.out4`	Driver board outputs (as last set), 1 / 0
`$relay.1` … `$relay.4`	USB relay channels, 1 / 0
`$io.<device>.<pin>`	An IO device pin: a digital pin as 1 / 0, a load cell as its weight in **grams**

Device and pin names with spaces or other characters use their underscore form: device "Arduino Nano", pin "Load Cell" → `$io.Arduino_Nano.Load_Cell`.

When an expression fails

Problem	When found	Result at run time
A misspelt `$name`	validator: `unknownVariable`	the program stops (`unknownVariable`, fatal)
A syntax error: `1 2`, unbalanced `(`, unknown function, wrong number of arguments	validator: `expressionSyntax`, `unknownFunction`, `badArity`	the program stops (`expressionSyntax`, fatal)
A list index out of range, an unknown field	—	reads as 0
Division by zero	—	0

These failures are **fatal** on purpose: a typo in an offset must never move the robot somewhere unintended, and a Try cannot hide it.

> **TIP**
> The expression tester in the editor evaluates an expression against the live controller (`EvaluateExpression`) — use it to check a formula before running.

CHAPTER 5

Points, Frames and Tools

Motion steps send the robot to **targets**. This chapter explains where targets come from and which coordinate frame they are measured in.

Positions

A position (a *pose*) has six values: `x y z` in millimetres and `rx ry rz` in degrees. Positions are measured at the **tool centre point** (TCP) — the flange when no tool is active, the tool tip when one is.

Saved points

A **point** is a named, saved position. Points are taught by jogging the robot and pressing *Teach* (or with Save Position from a program), and edited under **Space** → **Points**. Programs refer to them by name; storage links them by id, so renaming a point does not break programs.

```
MoveL Pick1              # by name
MoveL "Pick 1"           # a name with a space, quoted …
MoveL Pick_1             # … or in its underscore form
```

Every change to a point is recorded in the points history.

Frames

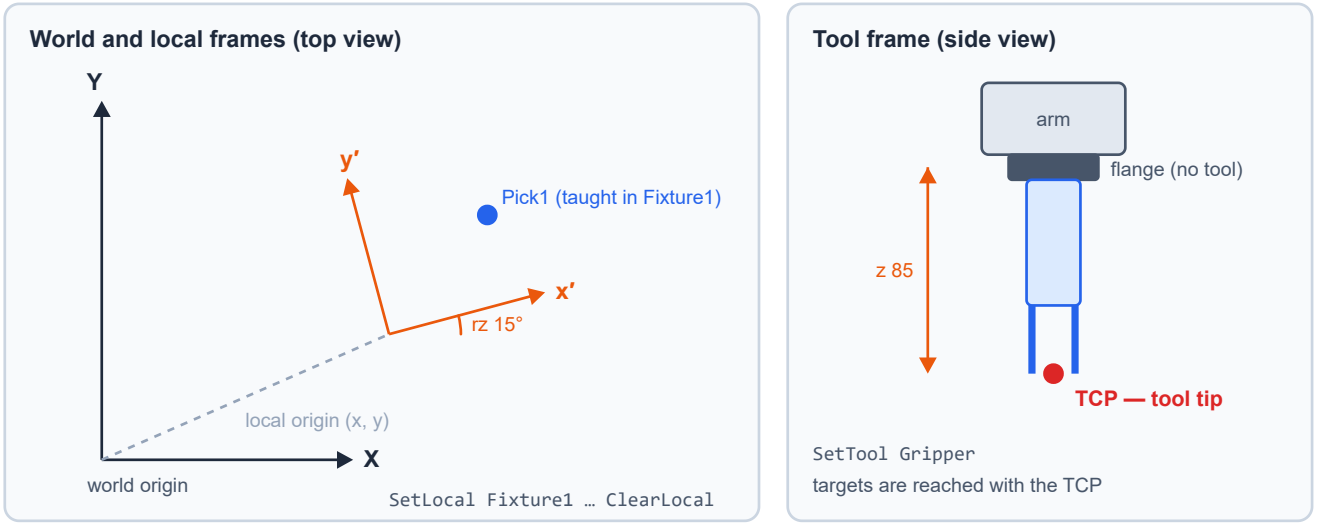


Figure 7. Coordinate frames. Left (top view): the world frame and a local frame (Fixture1) offset and turned 15°; a point taught in the local frame moves with it. Right (side view): the tool frame puts the TCP at the gripper tip.

Frame	What it is	Set with
World (base)	The robot's own coordinate system.	Always there; Clear Local returns to it.
Local	A named frame offset and turned from the world frame — a fixture, a tray, a machine.	Set Local for the moves that follow, or `local=` on one move.

Frame	What it is	Set with
Tool	The offset from the flange to the tool tip.	Set Tool.

Use a local frame whenever a group of points belongs to something that can move: teach the points in the local frame, and if the fixture moves, re-teach only the frame.

```
SetLocal Fixture1
MoveL here, offset=(x=10, y=10)   # 10 mm along the fixture's own x and y
ClearLocal
```

Tools

A **tool** describes the gripper, probe or other end effector: the offset `x y z rx ry rz` from the flange to its tip. With a tool active, targets are reached with the tip, and tool-frame offsets move along the tool's own axes. Tools are edited under **Space → Tools**.

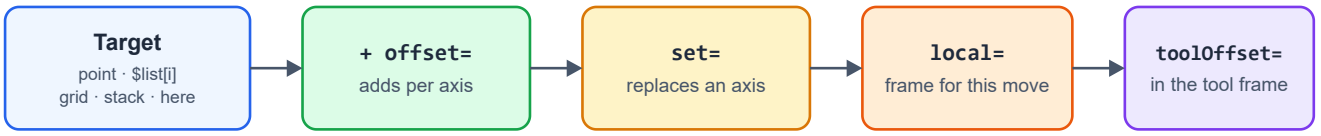
```
SetTool Gripper
MoveL Pick1          # the gripper tip goes to Pick1
SetTool Probe
MoveL Pick1          # now the probe tip goes there
```

Targets

A move's target is one of:

Target	Text	Notes
A saved point	`MoveL Pick1`	
An element of a Points list	`MoveL $pts[$i]`	The index is an expression.
A template naming a point	`MoveL "bin{$i}"`, `MoveL $target`	Filled in, then looked up by name; `pointNotFound` if nothing matches.
A grid cell	`MoveL grid Tray[$r, $c]` or `grid Tray[$n]`	See below.
A stack slot	`MoveL stack Bins[$i]`	See below.
Here	`MoveL here, offset=(z=20)`	No target: the move starts from the current position, so offsets are relative.

Offsets, overrides and frames on a move




```
MoveL Pick1, offset=(z=20), set=(rz=90), local=Fixture1, toolOffset=(z=-5)
```

Pick1 z=40 → offset gives z=60 → rz forced to 90 → resolved in Fixture1 → TCP backed off 5 mm along the tool's z

Figure 8. How a move target is built: the base target, then offset, then override, all in the active (or step) local frame; the tool offset applies in the tool frame at the end.

Modifier	Text	Effect
Offset	`offset=(x=…, z=…)`	Added to the target, per axis.
Override	`set=(z=30)`	Replaces an axis of the result outright.
Tool offset	`toolOffset=(z=-5)`	Applied in the tool frame at execution time — e.g. back off along the tool axis.
Local frame	`local=Fixture1`	Resolve this one move in another frame.

Each axis value may be an expression: `offset=(z=$layer * 5)`.

Grids and stacks

![A grid addresses cells by [row, col] or by index (row-major: index = row × columns + col). A stack addresses slots base + index × offset.](../figures/grid-stack.svg)

A **grid** is a base point plus a row offset and a column offset (and optionally a rotation about the base's Z). Cell `[row, col]` is *base* + *row* × *row offset* + *col* × *column offset*. With a column count set, a single index also works: `Tray[6]` is row 1, column 2 of a four-column grid.

A **stack** is a base point plus an offset per slot: slot *i* is *base* + *i* × *offset*. With a maximum count set, the index wraps round.

Both are edited under **Space**. The validator reports `untaughtGrid` (a warning) for a grid or stack whose offsets are all zero — every cell would be the same position.

Saving the current position

Save Position stores where the robot is — into a Points list or into a saved point:

```
var taught: list point
SavePosition $taught, mode=append      # build a path by teaching
SavePosition point Pick1               # overwrite (or create) a saved point
```

The position is expressed in the program's active local frame, so moving to it later under the same local lands on the same spot.

CHAPTER 6

Motion

Step	Keyword	Path
Move Linear	`MoveL`	Straight line to the target.
Move Joint	`MoveJ`	Joint-interpolated: all joints start and stop together.
Jump Linear / Joint	`JumpL` / `JumpJ`	Lift, traverse, lower — the pick-and-place move.
Move Arc	`MoveC`	Circular arc through a via point.
Thread Move	`ThreadMove`	Z travel synchronised with RZ rotation.
CNC Program	`CncProgram`	A toolpath from a DXF or SVG file.
Run Homing	`RunHoming`	The full homing sequence.

All motion steps are **skipped in background programs**.

Linear and joint moves

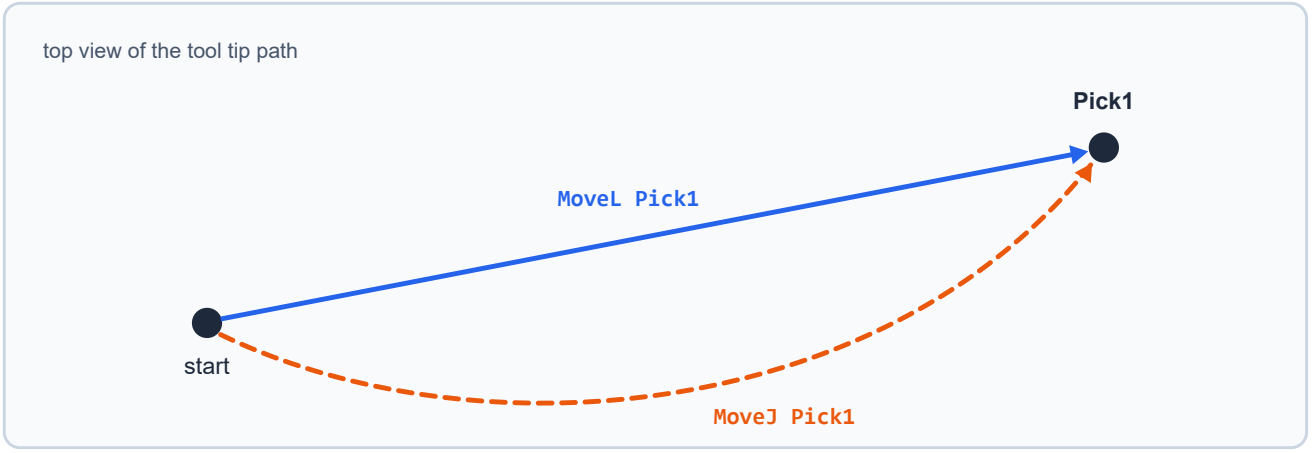
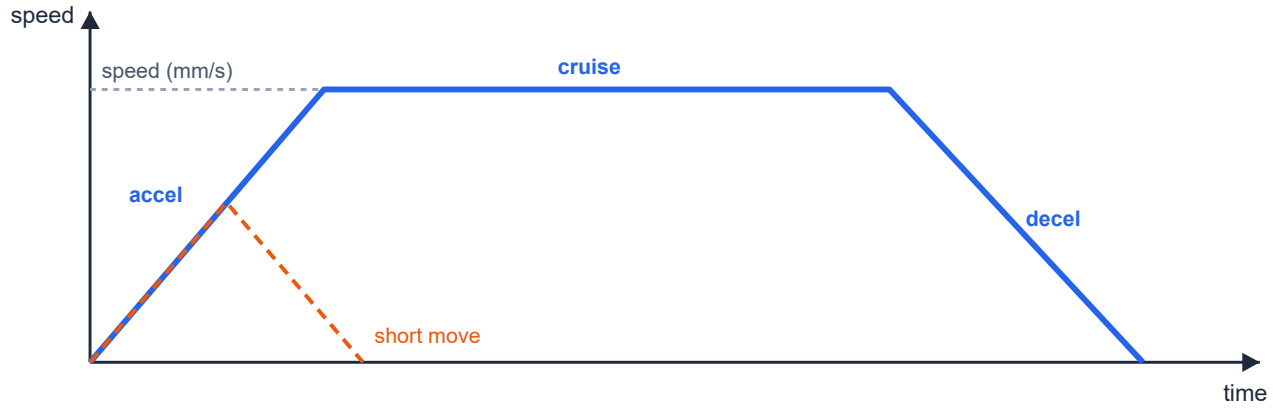


Figure 9. Move Linear keeps the tool on a straight line; Move Joint moves every joint proportionally, so the tool follows a curve — usually faster in open space.

- Use **MoveL** when the path matters: approaching a part, inserting, following an edge.
- Use **MoveJ** to travel through open space, where the exact path does not matter: it is usually faster. Check that the curved path is clear of obstacles.

```
MoveJ Home
MoveL Approach
MoveL Pick1, speed=50
```

Speed, acceleration and deceleration




```
SetSpeedL 200, accel=800, decel=800
```

Figure 10. A move ramps up at accel, cruises at speed, and ramps down at decel. A short move never reaches full speed (dashed). Program speeds are scaled by the speed override × Set Speed Factor.

Where	Sets
Set Speed (Linear) `SetSpeedL 200, accel=800, decel=800`	Defaults for later linear moves.
Set Speed (Joint) `SetSpeedJ 60`	Defaults for later joint moves.
`speed=` `accel=` `decel=` on one move	This move only.
Set Speed Factor `SetSpeedFactor 50`	Scales every later move of this program, 1–100 %.
The operator's **speed override** (monitor, 5–200 %)	Scales every program move.

The speed a program move actually runs at is *speed* × *operator override* × *speed factor / 100*. The speed factor can only slow a program down; manual jogging is never affected by either. Read the current values in expressions as `$robot.speedS`, `$robot.speedOverride` and `$program.speedFactor`.

Blending

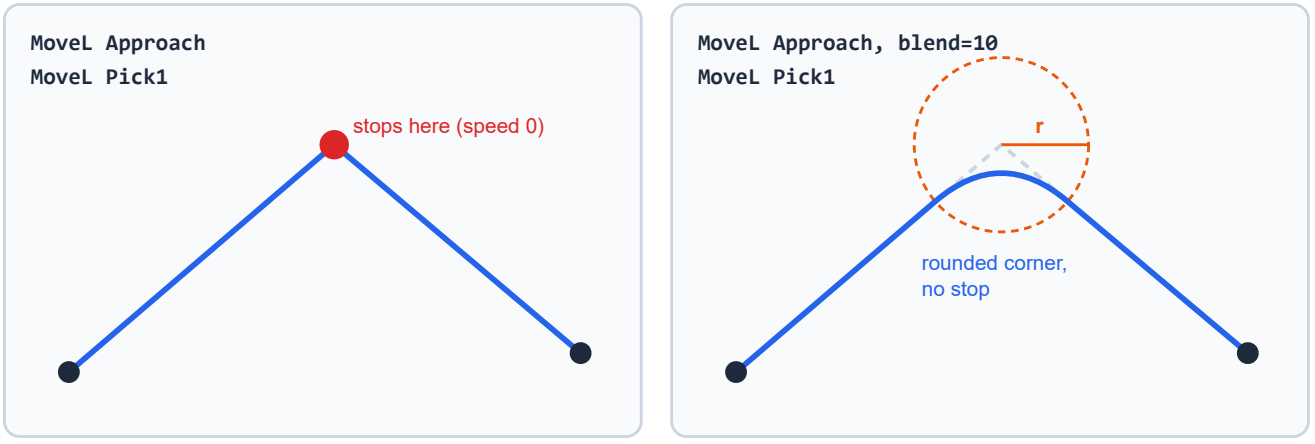


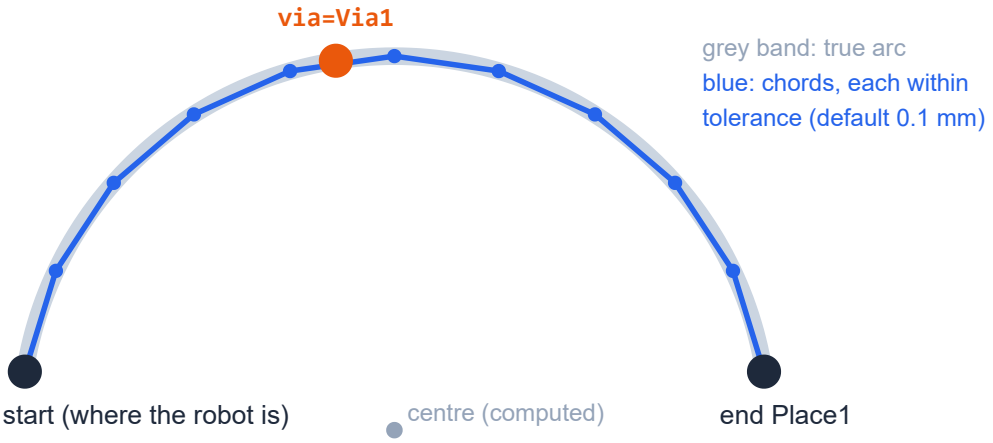
Figure 11. Without blending the robot stops exactly on each point. With a blend radius it rounds the corner inside the radius and never stops.

A Move Linear (or the traverse of a Jump) with `blend=` does not stop on its point: it rounds the corner into the next blend-enabled move, and a run of blended moves becomes one continuous path.

```
SetBlendRadius 5
MoveL Approach, blend=on        # uses the radius set above
MoveL Via1, blend=10            # its own 10 mm radius
MoveL Pick1                     # not blended: stops exactly here
```

- `blend=on` uses the radius from Set Blend Radius; `blend=<mm>` or `blend=$r` gives one. `blend=off` stops.
- The robot does not pass exactly through a blended point — it stays within the radius of it. Do not blend into a point you must reach exactly (a pick or place).
- A move with a stop condition never blends.

Arcs




```
MoveC Place1, via=Via1, speed=80, tolerance=0.1
```

Figure 12. Move Arc follows the circle through the start, the via point and the end. It is cut into chords within tolerance= of the true arc and run as one continuous path.

```
MoveL Pick1                              # the arc starts where the robot is
MoveC Place1, via=Via1, speed=80, tolerance=0.1
```

The orientation changes linearly from the start's to the end's; the via point's orientation is not used. Start, via and end on one line fail with `arcDegenerate`.

Jumps



Figure 13. Jump (side view): lift straight up to zStart, traverse to above the target at zEnd, lower onto the target. z= sets both heights; the heights are absolute Z values.

A Jump is three moves in one step — the classic pick-and-place motion:

```
JumpL Place1, z=60                 # lift to Z 60, travel, lower onto Place1
JumpJ grid Tray[$i], z=80, zEnd=50  # joint legs; lower from Z 50
```

`JumpL` uses straight legs; `JumpJ` uses joint-interpolated legs (faster, curved traverse).

Stop conditions — move until

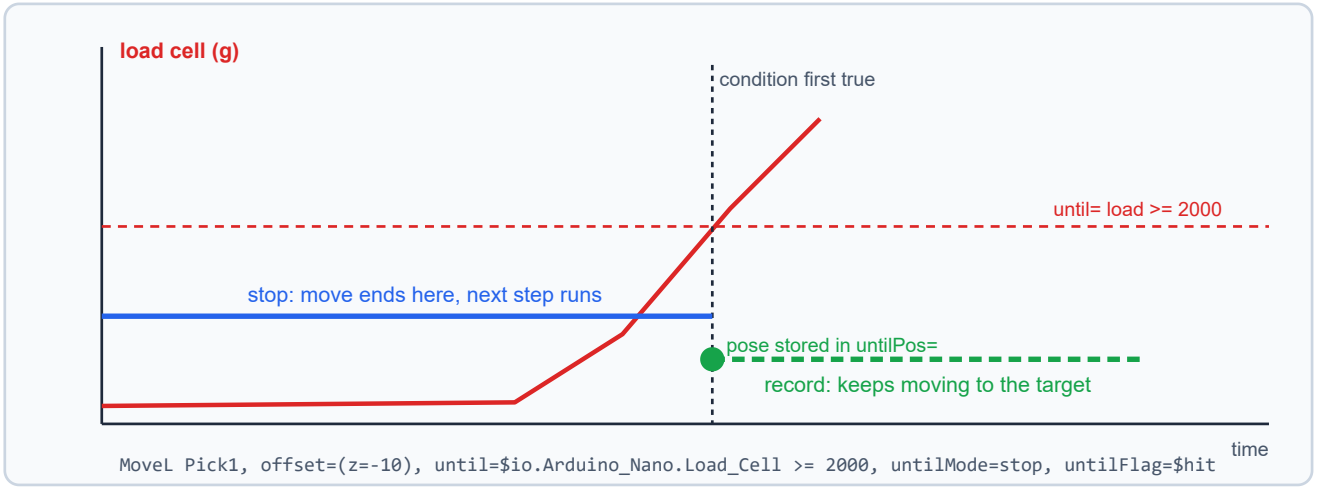


Figure 14. A move with a stop condition: the condition is checked every tick. stop ends the move where it is; record lets it finish and stores the pose where the condition first held.

Move Linear, Move Joint, the Jumps, Aux Move and Gripper can watch a **condition** while they move. The common case is "move with force": press down until a load cell reads a limit.

```
var hit = 0
var touch: list point
MoveL Pick1, offset=(z=-10), until=$io.Arduino_Nano.Load_Cell >= 2000, untilFlag=$hit
MoveL Pick1, offset=(z=-10), until=$stb.in2, untilMode=record, untilPos=$touch
```

Argument	Meaning
`until=`	The condition, checked every controller tick while the move runs.
`untilMode=stop` (default)	End the move where it is the first time the condition holds, and continue with the next step — no error, no pause.
`untilMode=record`	Let the move finish; store the position where the condition first held.
`untilPos=$v`	Receives that position: a one-element Points list (`$v[0].z`), or a number for an aux axis.
`untilFlag=$v`	1 if the condition fired during the move, else 0.

On a Jump the watch spans all three legs; in stop mode the remaining legs are skipped.

Homing and manual homing

`RunHoming` runs the full homing sequence and waits for it. `SetJointPosition J4, 0` declares a joint's current value without moving it (manual homing). `$robot.homed` tells whether the robot has been homed:

```
if not $robot.homed
  RunHoming
end
```

Errors from motion

Code	Catchable	When
`pointNotFound`	yes	A template or list target names no saved point.
`arcDegenerate`	yes	Move Arc points on one line.
`motionFault`	no	The motion system reports a fault.
`jointLimit`	no	A target is outside a joint's soft limits.
`driverDisconnected`	no	The motor driver is not connected.
`emergencyStop`	no	The emergency stop was pressed.

CHAPTER 7

I/O, Waits and Aux Axes

Where I/O lives

Card	In text	Outputs	Read in expressions
Driver board	`stb`	1–4	`$stb.in1` … `$stb.in4` , `$stb.out1` … `$stb.out4`
USB relay	`relay`	channels 1–4	`$relay.1` … `$relay.4`
IO device (e.g. Arduino Nano)	`io <device>`	its output pins	`$io.<device>.<pin>` (digital 1 / 0, load cell in grams)

Pins and relays can be given friendly names under **I/O**; names with spaces are written in their underscore form in expressions (`$io.Arduino_Nano.Load_Cell`).

Setting outputs

```
SetOutput stb 1 on                      # card number value
SetOutput relay 3 off
SetOutput io Arduino_Nano 7 on          # an IO device pin
SetOutput stb 2 toggle                  # flip the current state
SetOutput stb 2 ($count > 3)            # an expression: non-zero = on
SetOutput relay 2 on, pulse=250ms       # on for 250 ms, then off again
SetOutput relay 2 on, pulse=250ms, blocking=on
```

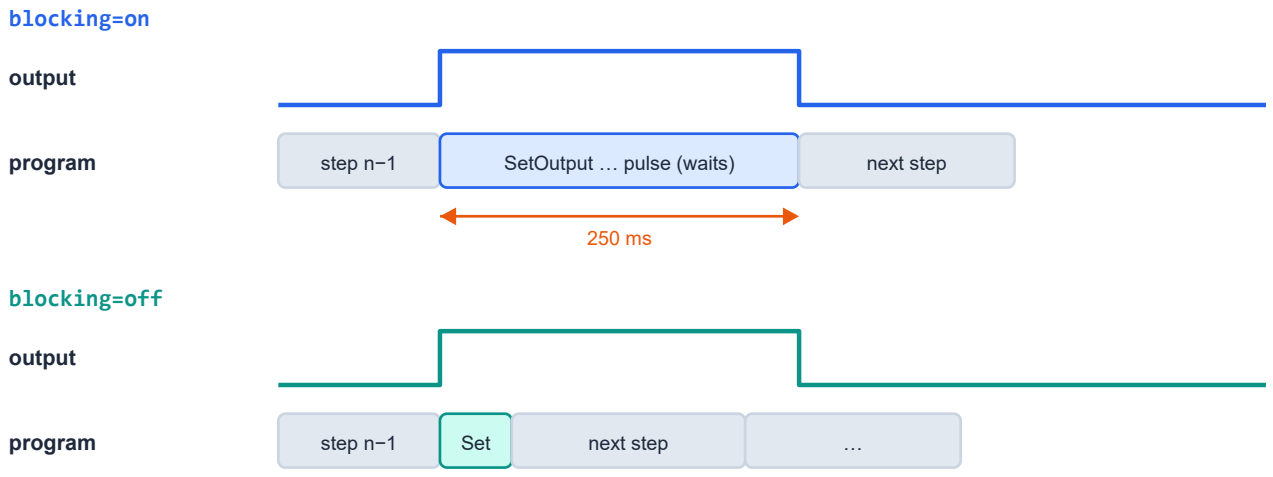


Figure 15. Set Output with pulse=250ms. blocking=on holds the program until the output has flipped back; without it the next step starts at once and the flip-back happens in the background.

A pulse always returns the output to the opposite of the state the step set, so a toggle pulse flips the output and flips it back.

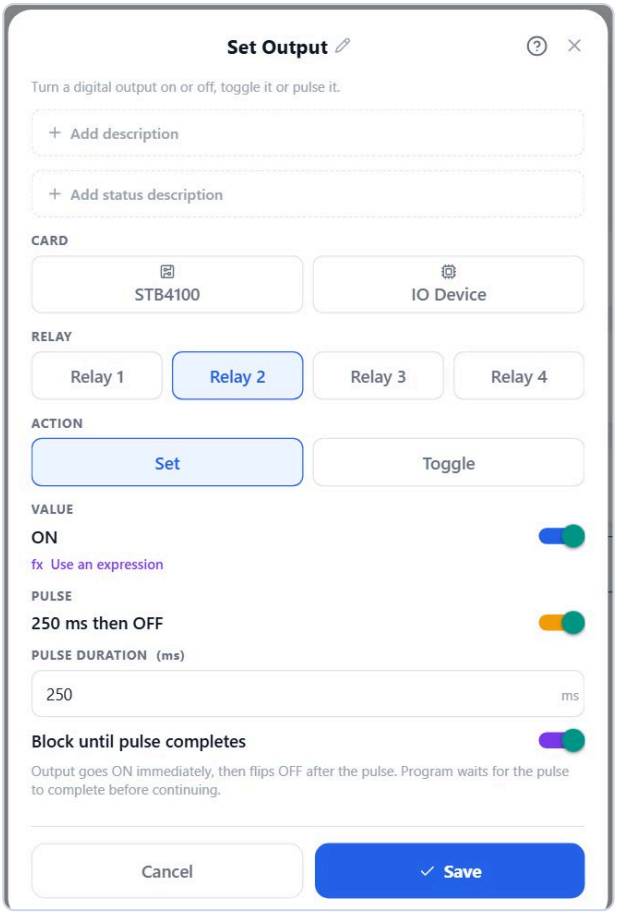


Figure 16. The Set Output step editor with a 250 ms blocking pulse.

Waiting

Wait pauses the program for a time, or until a condition holds.

```
Wait 500ms                                  # a fixed time (also: Wait 2s, Wait $delay)
Wait until $stb.in1 == 1                    # until an input comes on — forever if need be
Wait until $stb.in1, timeout=5s, timedOut=$late
Wait until $stb.in1, timeout=5s, onTimeout=error
```

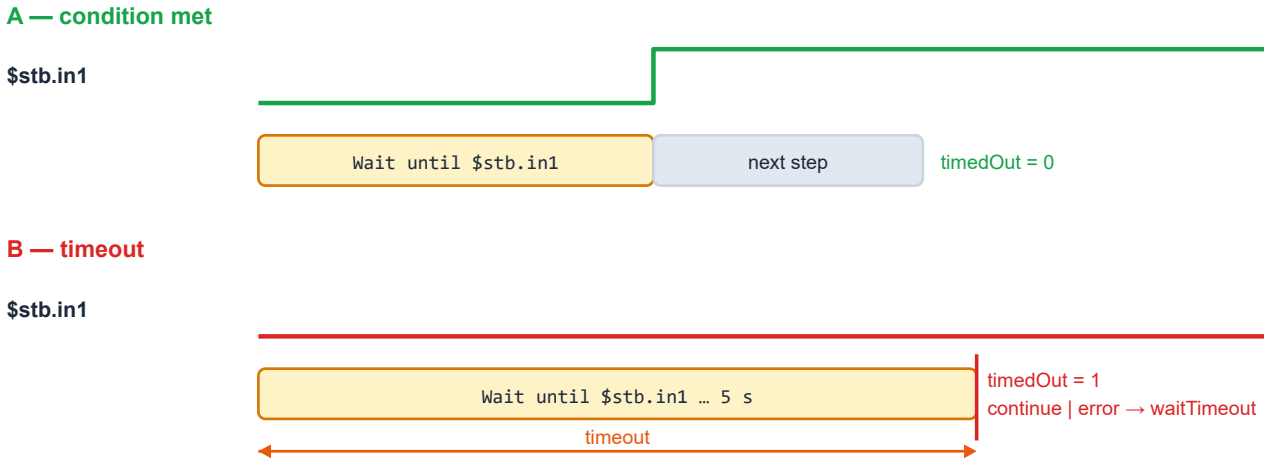


Figure 17. Wait until <condition>, timeout=5s. If the input comes on in time the program continues; otherwise timedOut= is set to 1 and onTimeout decides: continue (default) or fail with waitTimeout.

Argument	Meaning
`timeout=`	Give up after this long. Empty or 0 waits forever.
`timedOut=$v`	Set to 1 when the wait timed out, 0 when the condition held. Created if not declared.
`onTimeout=continue`	(Default) carry on with the next step; check `$v` yourself.
`onTimeout=error`	Fail with `waitTimeout` — catch it with a Try to retry or recover.

The condition is checked every controller tick; one that holds on the tick the timeout falls due counts as met.

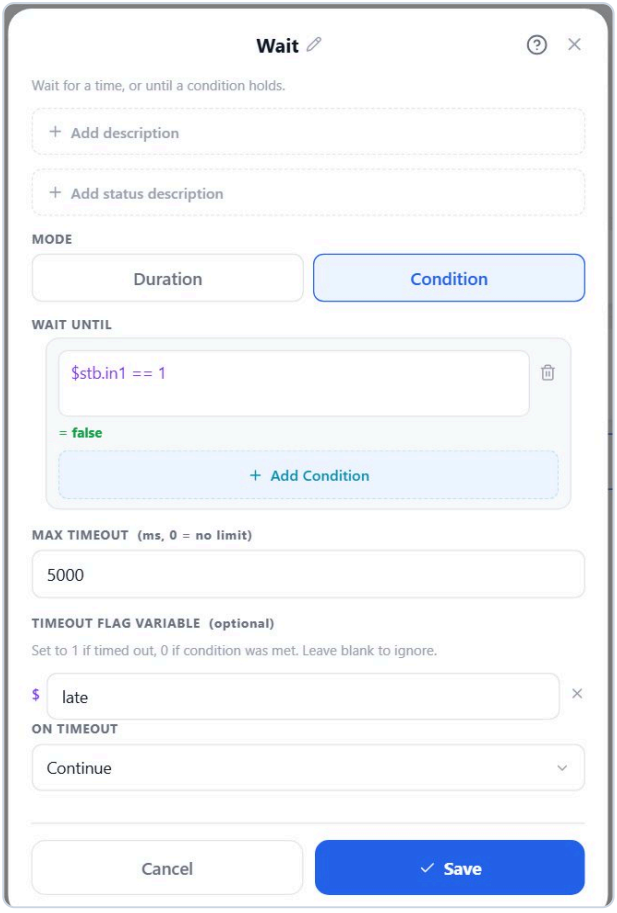


Figure 18. The Wait step editor in condition mode, with a timeout and a timed-out flag.

Handshakes

A typical handshake with another machine: raise a request, wait for its answer with a timeout, drop the request.

```
var late: bool = off
SetOutput stb 1 on                              # "part ready"
Wait until $stb.in2, timeout=10s, timedOut=$late # "taken"
SetOutput stb 1 off
if $late
  status warning "No answer from the machine"
end
```

Load cells

An IO device pin configured as a load cell reads a calibrated weight. In expressions it is always in **grams**; Read Load Cell can convert to kg, N, lb or oz.

```
var grams = 0
var rec: list record
Tare device=Arduino_Nano, pin=2                       # zero it
ReadLoadCell $grams, device=Arduino_Nano, pin=2       # into a variable
StartRecording $rec, device=Arduino_Nano, pin=2       # record while moving …
MoveL Pick1, until=$io.Arduino_Nano.Load_Cell >= 2000 # … press until 2 kg
StopRecording device=Arduino_Nano, pin=2              # … samples {t, v, r} land in $rec
ExportData csv "logs/press-{$time_ms}.csv"
  "series" = list $rec
end
```

Aux axes, grippers, conveyors and rotary tables

Auxiliary stepper axes are driven by an aux card. Each axis has a role set under **I/O**; the role-specific steps use its configured distances and speeds.

Step	Use
Aux Move	Move by a distance (`unit=mm` / `deg`) or raw `steps=` ; `absolute=on` to go to a position; `wait=off` to fire and forget.
Aux Continuous Run	Run at a velocity until Aux Stop.
Aux Motor Enable	Enable or disable the drivers.
Gripper	Open, close, or grip to a jaw gap. Takes a stop condition (close until a grip force).
Conveyor	Forward, reverse or stop.
Rotary Index	Turn a number of positions.

```
GripperAction open
GripperAction close, until=$io.Arduino_Nano.Input_1 >= 300
ConveyorAction forward, device=Aux_Stepper, axis=2
Wait until $stb.in1
ConveyorAction stop, device=Aux_Stepper, axis=2
RotaryIndex +1, count=3, device=Aux_Stepper, axis=3
```

Aux positions read as `$aux.<deviceId>.<axis>.position` (steps) and `$aux.<deviceId>.moving`. Aux steps run in background programs too.

CHAPTER 8

Flow Control

Step	Text	Use
If Condition	`if … elif … else … end`	Choose between branches.
Switch	`switch … case … default … end`	Choose by a number.
Loop	`repeat`, `while`, `for … in`	Repeat steps.
Break / Next	`break`, `continue`	Leave a loop / skip to its next pass.
Label / Go To Label	`label`, `goto`	Jump to a marked place.
Call Routine / Return	`call`, `return`	Run a routine — see Routines.
End Program	`exit`	Finish now, from anywhere.
Pause Program	`pause`	Pause until the operator continues.
Try / Raise Error	`try … catch … end`, `raise`	Handle errors — see Error Handling.

If, elif, else

```
if $count >= 12
…
elif $count > 8
…
else
…
end
```

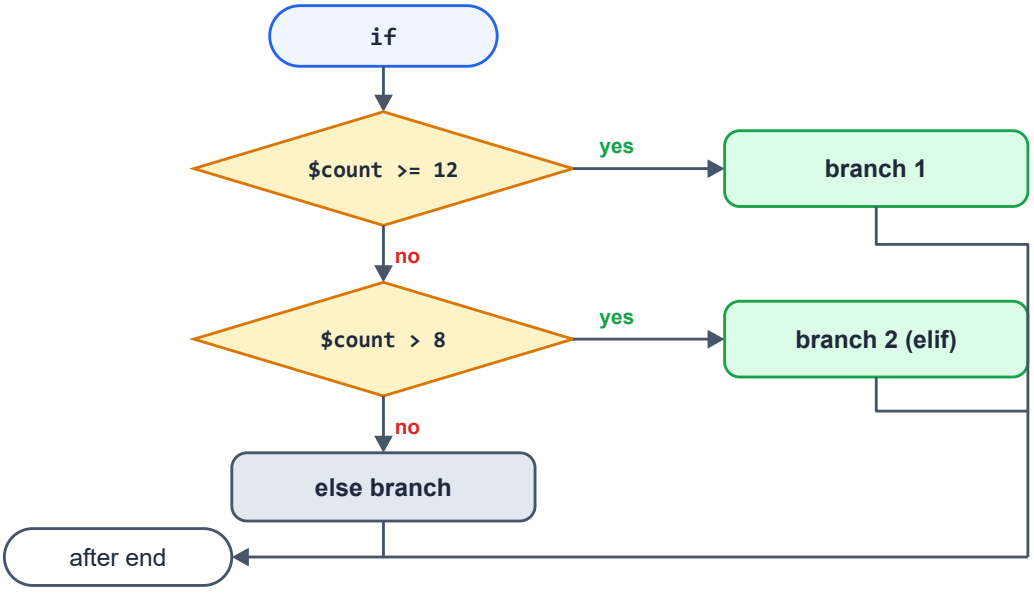


Figure 19. if / elif / else: conditions are tested top to bottom; the first that holds runs its branch, then execution continues after end.

```
if $count >= 12
  status "Tray full"
  exit
elif $count > 8                # "else if" works too
  status warning "Almost full"
else
  JumpL grid Tray[$count], z=40
end
```

In Blocks the branches of an If are shown side by side; tap **Enter** to edit a branch's steps and **+ ELSE IF** to add a branch.

Switch

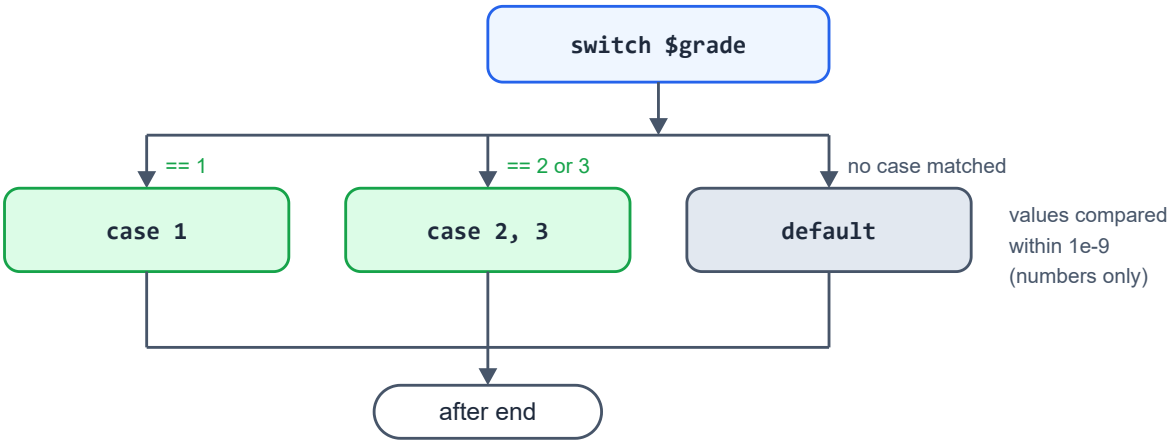


Figure 20. switch evaluates its value once and runs the first case with an equal value (no fall-through), else default.

```
switch $grade
case 1
  MoveL Place1
case 2, 3                    # several values for one case
  MoveL stack Bins[0]
default
  raise "Unknown grade {$grade}", code=badGrade
end
```

Switch compares numbers. To branch on text use `if $s.contains("…")`.

Loops

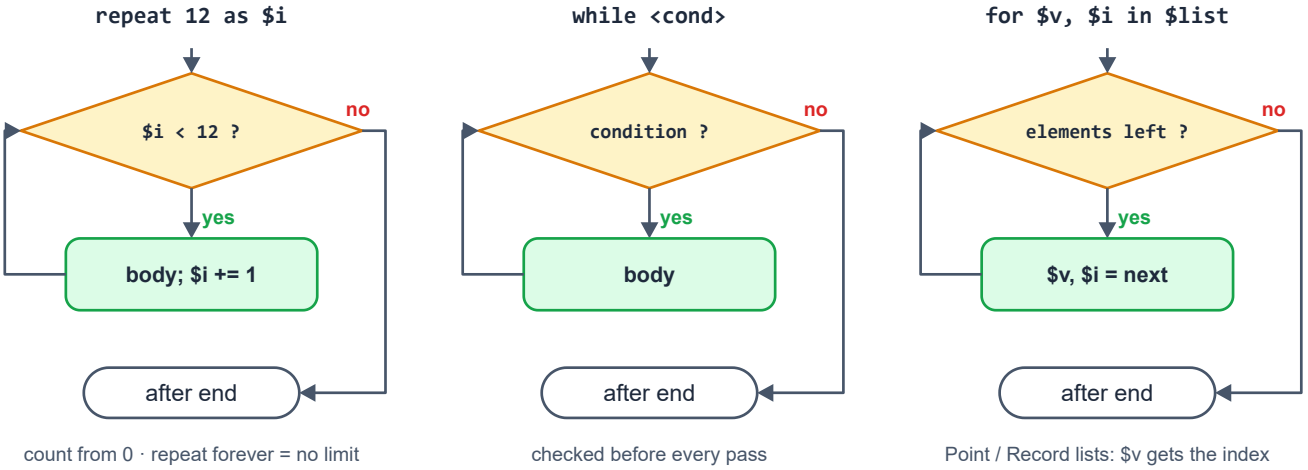


Figure 21. The three loop forms. break leaves the loop at once; continue skips to the next pass.

```
repeat 12 as $i                # $i = 0, 1, … 11
  JumpL grid Tray[$i], z=40
end

repeat forever                 # until stopped, or break
  Wait until $stb.in1
  …
end

while $stb.in1 == 0            # checked before every pass
  Wait 10ms
end

for $v, $i in $vals            # Number / Boolean list: $v is the element
  $sum = $sum + $v
end
for _, $i in $parts            # Point / Record list: use the index
  JumpL $parts[$i], z=40
end
```

The loop variables (`as $i` , `$v` , `$i`) are ordinary program variables, readable in any expression inside the body. A count of 0 (`repeat forever`) has no limit.

Break and next

```
repeat forever
  if $stb.in2                  # stop button
    break                      # → continues after end
  end
  if not $stb.in1
    continue                   # → next pass
  end
  …
end
```

`break` and `continue` act on the nearest enclosing loop, leaving any branches and Try bodies in between. They never cross a routine boundary: a `break` in a routine called from a loop is an error (`loopControlOutsideLoop`).

Labels and Go To

```
label again
Wait until $stb.in1, timeout=5s, timedOut=$late
if $late
  goto again
end
```

`goto` jumps to a label in the same program (or the same routine). Loops usually express the same thing more clearly; use labels for the occasional retry or skip.

Ending and pausing

- `exit "message"` finishes the program with **Complete** from any depth — inside loops, routines or a Try — with the message as its final status. In a background program it ends only that program.
- `return` leaves a routine; in a main program it finishes the program.

- `pause` pauses the run with its place kept; the operator continues or stops from the monitor.
- `raise "message"` fails the program with an error — unless a Try catches it.

Disabled steps

A disabled step (`@off`, or the power button on the card) is skipped together with its whole body — a quick way to switch off part of a program while testing.

CHAPTER 9

Routines and Parameters

A **routine** is a program that other programs call. Use routines for anything you do in more than one place — picking a part, changing a tool, a homing check — so it is written and fixed once.

Making a routine

Set the program's kind to **Routine** in Program Settings (in a file: `@program {"isRoutine": true}`). A routine is hidden from the program list and runs only when called.

Parameters

A routine's variables can be **parameters**:

Mode	Text	Direction
in	`param in slot = 0`	The caller passes a value in.
out	`param out picked: bool = off`	The routine passes a value back.
in-out	`param inout tally = 0`	Both.

```
#!srs 1
@program {"name": "PickPart", "isRoutine": true}
param in slot = 0
param out picked: bool = off

MoveJ Approach
JumpL stack Bins[$slot], z=40, speed=150
GripperAction close
Wait 200ms
MoveL here, offset=(z=30), speed=100
$picked = true
```

Calling

```
call PickPart                                  # no arguments
call PickPart(slot=$i)                         # in arguments
call PickPart(slot=$i + 1), out picked=$ok     # and an output
```

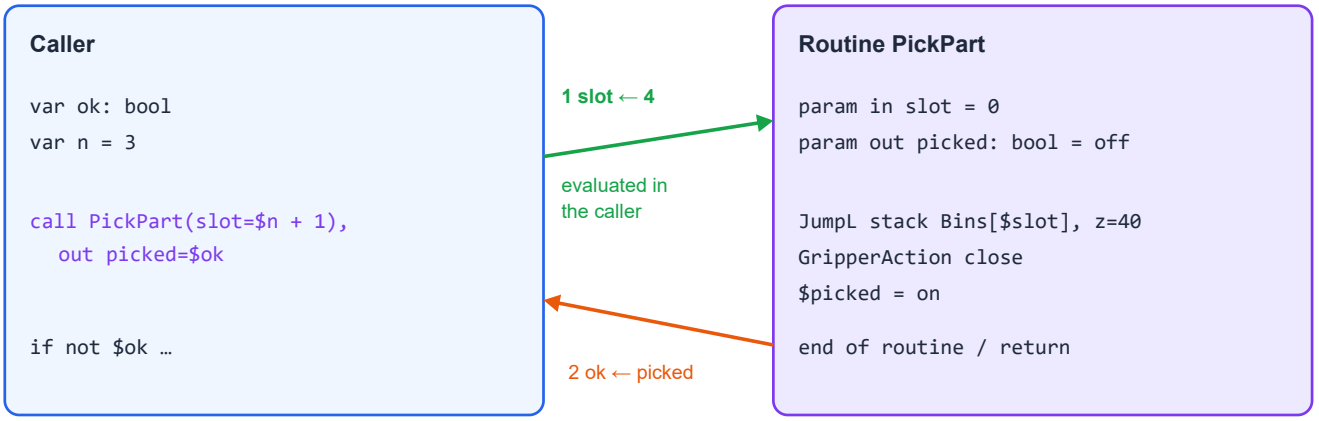


Figure 22. Calling a routine: in / inout arguments are evaluated in the caller, then copied into the parameters; when the routine ends (or returns), out / inout parameters are copied back.

1. Each `in` / `inout` argument is evaluated **in the caller's scope** before the routine's variables are set up — so `slot=$slot + 1` reads the caller's `$slot`. A text parameter takes a template; a list parameter takes a list variable (copied).
2. The routine runs.
3. When it ends — its last step or a `return` — each mapped `out` / `inout` parameter is copied into the caller's variable (created if not declared). An `in` parameter is never copied back.

A parameter with no argument keeps its declared value (validator warning `paramMissing`).

Rules

- Routine variables live in the **same variable scope** as the caller's, so give parameters names that do not clash with the caller's variables.
- A routine is a boundary: `goto` and `break` / `continue` inside it never reach the caller's labels or loops.
- A routine that does not finish (it fails, or a Try abandons it) copies no outputs.
- A routine must not call itself, directly or through others (`routineRecursion`).
- An error inside a routine travels out to the caller's Try, like any other error — see Error Handling.

CHAPTER 10

Error Handling

Problems reach you at two moments:

When	What	Effect
Before the run	Parse problems (text) and validation problems (both views) — unknown names, misspelt variables, missing fields.	Errors block **running** (and parse errors block saving in Text view); warnings do not. See Running and Debugging.
During the run	A step **fails** with an error code — a wait timed out, a vision result did not come, an HTTP server did not answer.	The program ends with **Error** — unless a Try catches the code.

Catchable and fatal codes

Every runtime failure has a code. Codes are either **catchable** — runtime conditions a program can reasonably recover from — or **fatal** — hardware, safety or authoring problems a Try must never hide.

Catchable	Fatal (never caught)
`waitTimeout` `visionTimeout` `visionFailed` `httpFailed` `httpTimeout` `receiveTimeout` `pluginFailed` `pluginTimeout` `pluginNotRunning` `promptTimeout` `pointNotFound` `arcDegenerate` `indexOutOfRange` `userError` — and any code you raise yourself	`motionFault` `jointLimit` `driverDisconnected` `emergencyStop` `userStop` `unknownVariable` `expressionSyntax` `computedVariable` `badListOperation`

The full table, with what raises each code, is in Error and Problem Codes.

Try, retry and catch

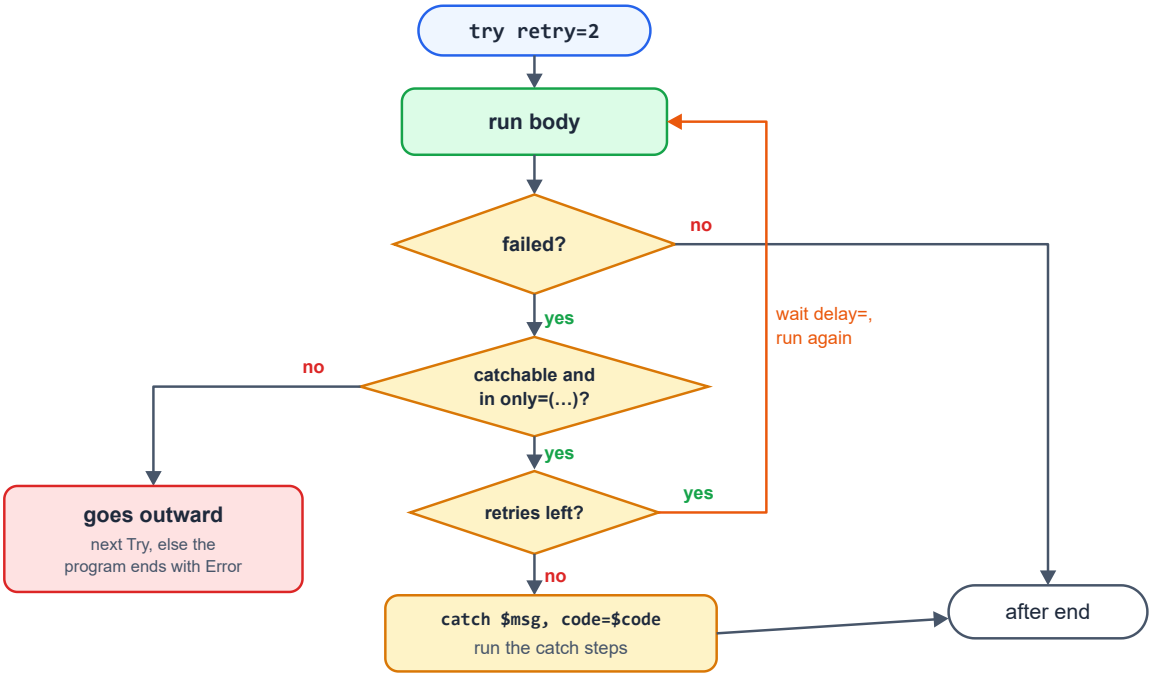


Figure 23. try retry=2: a catchable failure abandons the body and runs it again until the retries are used up; then the catch steps run with the message and code. Fatal errors and codes not in only=(...) go outward.

```
var msg: string
var code: string
try retry=2, delay=500ms
  RunVision Detect_Parts, pass=$ok
  if not $ok
    raise "No parts found", code=noParts
  end
catch $msg, code=$code, only=(visionTimeout, noParts)
  status error "Vision failed: {$msg} ({$code})"
  exit
end
```

When a step inside the body fails with a catchable code — at any depth, inside loops, branches and called routines:

1. everything inside the Try is abandoned (a routine that did not finish copies no outputs);
2. while retries are left, the body runs again from the top after `delay=`;
3. then the message and code go into the catch variables and the **catch** steps run. An empty catch simply ignores the error. Either way the program carries on after `end`.

Argument	Meaning
`retry=`	How many times to run the body again (default 0).
`delay=`	Wait between attempts (`500ms`, `2s`).
`catch $msg`	Text variable for the error message (created if not declared).
`code=$code`	Text variable for the error code.
`only=(a, b, …)`	Catch only these codes; others go to the next enclosing Try. Empty = every catchable code.

Each catch is logged, e.g. `[Caught waitTimeout] … — retry 1 of 2`. A failure inside the catch steps goes outward too.

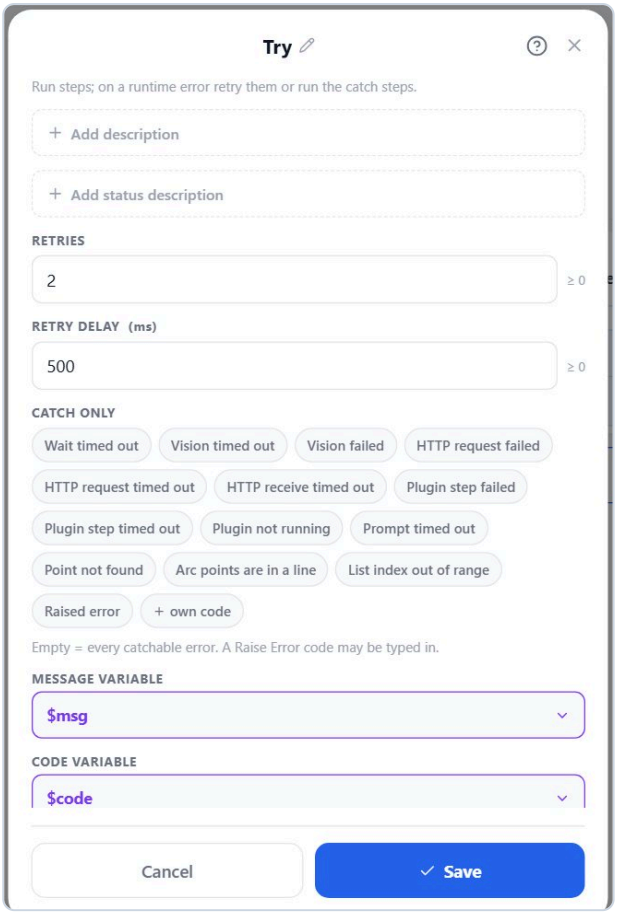


Figure 24. The Try step editor: retries, delay, which codes to catch and where the message goes.

How an error travels

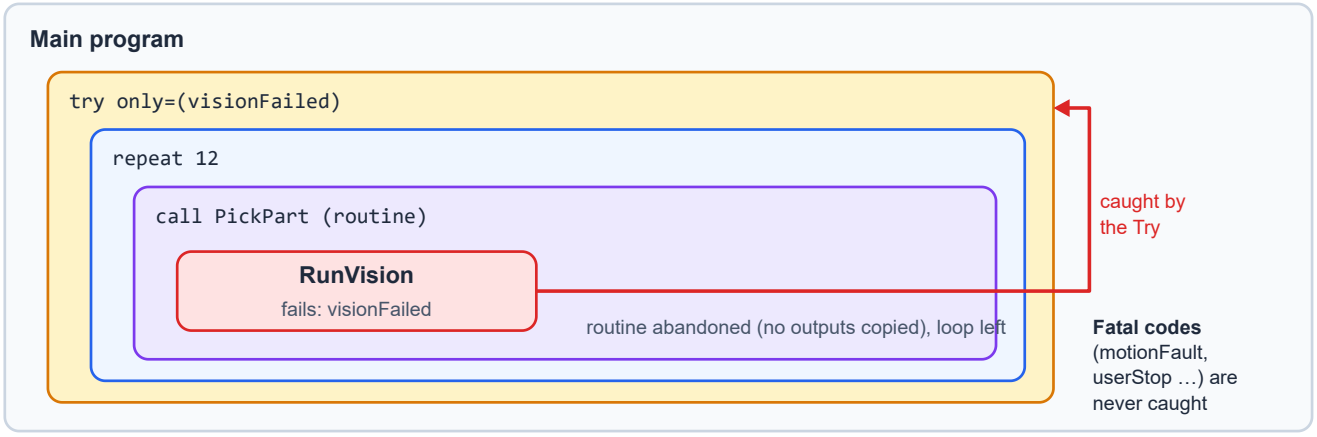


Figure 25. A catchable error travels outward from the failing step — through loops, branches and called routines — to the nearest Try that catches its code. With none, the program ends with Error.

When the program ends with Error, its status carries the message and the `errorCode`, and the monitor shows the step that failed.

Raising your own errors

Raise Error fails on purpose with your message and an identifier of your choice:

```
if $grams < 5
  raise "Slot {$i} is empty", code=emptySlot
end
```

A code of your own is always catchable, so an enclosing `try … only=(emptySlot)` can handle exactly this case. The default code is `userError`. Raising a fatal code (e.g. `motionFault`) is a validator warning — no Try could catch it.

Waits that fail

A condition Wait only fails when you ask it to: with `onTimeout=error` it raises `waitTimeout`; otherwise it continues and sets its timed-out flag. Choose per case:

```
Wait until $stb.in1, timeout=5s, timedOut=$late   # handle it inline
try retry=3
  Wait until $stb.in1, timeout=5s, onTimeout=error  # or let a Try retry it
catch
end
```

Which steps fail with which code

Step	Catchable failure
Wait (condition, `onTimeout=error`)	`waitTimeout`
Run Vision	`visionTimeout` (no result in time), `visionFailed` (processor lost, robot frame not available)
Capture Image / Save Image	`visionFailed` (no camera frame)
HTTP Request (waiting for the reply)	`httpFailed` (network error, non-2xx status), `httpTimeout`
HTTP Receive	`receiveTimeout`
Plugin step, plugin functions	`pluginFailed`, `pluginTimeout`, `pluginNotRunning`
Moves with a template or list target	`pointNotFound`
Move Arc	`arcDegenerate`, `pointNotFound`
List, Save Position	`indexOutOfRange`
Prompt (timeout without a timeout button)	`promptTimeout`
Raise Error	`userError` or your own code

CHAPTER 11

Running and Debugging

Checking a program before it runs

The controller checks a program every time it changes. The problem count appears in the editor header; tap it to see the list. Each problem names its step (and field), and tapping it jumps there.

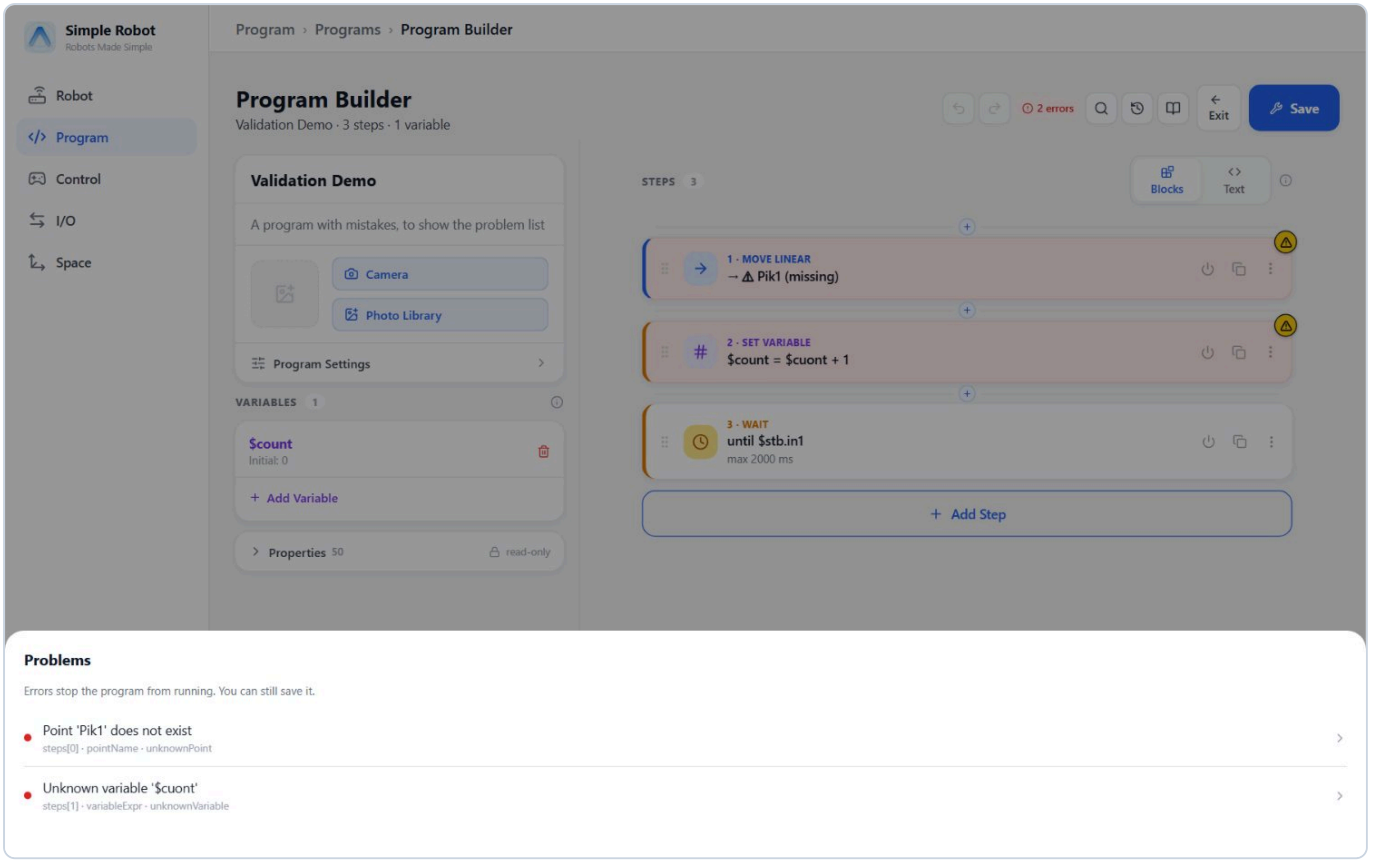


Figure 26. Problems found before a run: an unknown point (warning) and a misspelt variable (error). Tap a problem to jump to its step.

- **Errors** block running: an unknown variable, an expression that does not parse, a missing required field, a reference to something that does not exist, a loop or branch with no steps.
- **Warnings** do not: an unknown point you have not taught yet, an unused variable, a disabled step, a step after an endless loop.

In Text view the same problems are underlined on their lines, together with parse problems such as a missing `end`.

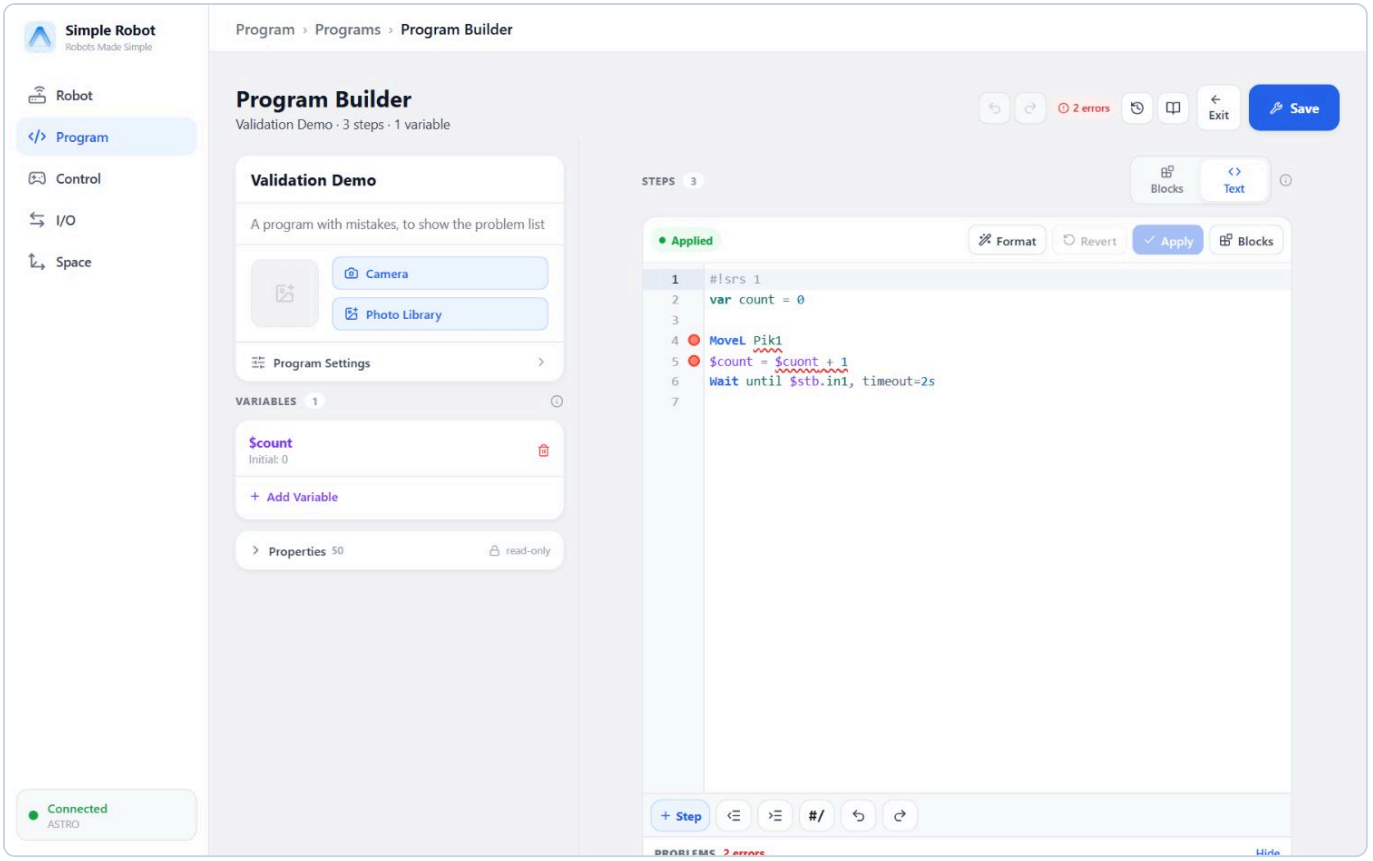


Figure 27. In Text view the same problems are underlined on their lines.

The codes are listed in Error and Problem Codes.

Program states

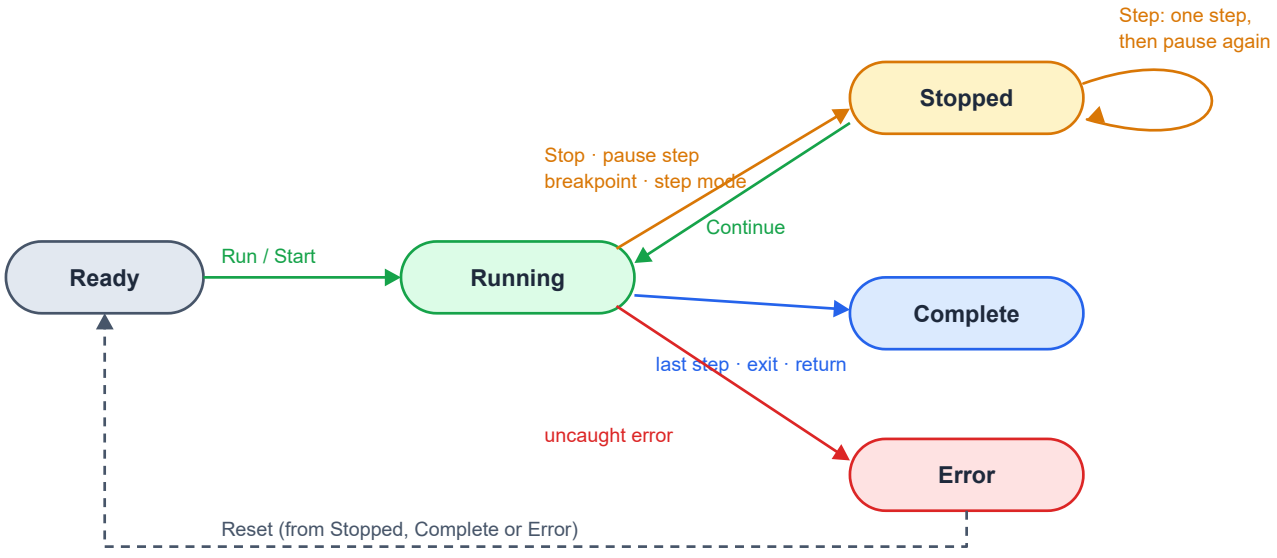


Figure 28. Program states. Stop, a Pause Program step, a breakpoint or step mode all pause the run (Stopped) with its place kept; Continue resumes, Step runs one step and pauses again.

State	Meaning
Ready	Not running; the next start begins at the top.
Running	Executing steps.

State	Meaning
Stopped	Paused with its place kept — by the operator's Stop, a Pause Program step, a breakpoint or step mode. A move that was interrupted resumes on Continue.
Complete	Finished normally: after the last step, an `exit`, or a `return` in the main program.
Error	Ended by an uncaught error; the status carries the message and `errorCode`, and the failed step is marked.

The monitor

The monitor shows the running program: its status text (from Status Update), the current step, the variables marked **monitor**, display images, the run controls (start, stop, and while paused Continue and Step) and the operator speed override.

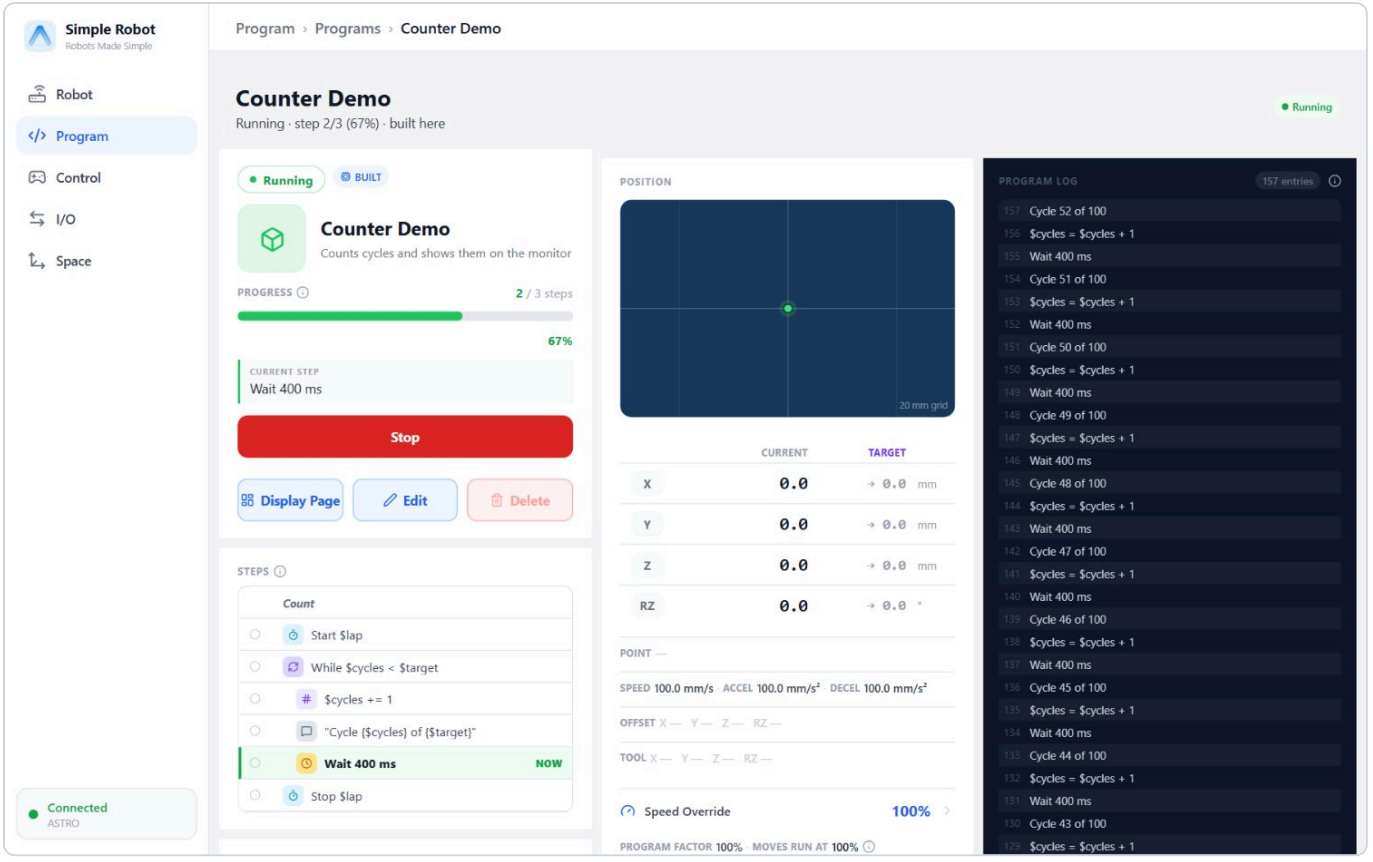


Figure 29. The monitor while a program runs: status, current step, monitor variables and the run controls.

Program moves are scaled by the speed override; manual jogging is not. The run log (per program) records each step, skipped steps, caught errors and status messages.

Breakpoints

Set a breakpoint on a step in the editor (in Text view: in the gutter). The program pauses **before** that step, with pause reason *breakpoint*; Continue runs on. A breakpoint inside a loop pauses on every pass. Breakpoints are kept per program in the app and can be changed while the program runs.

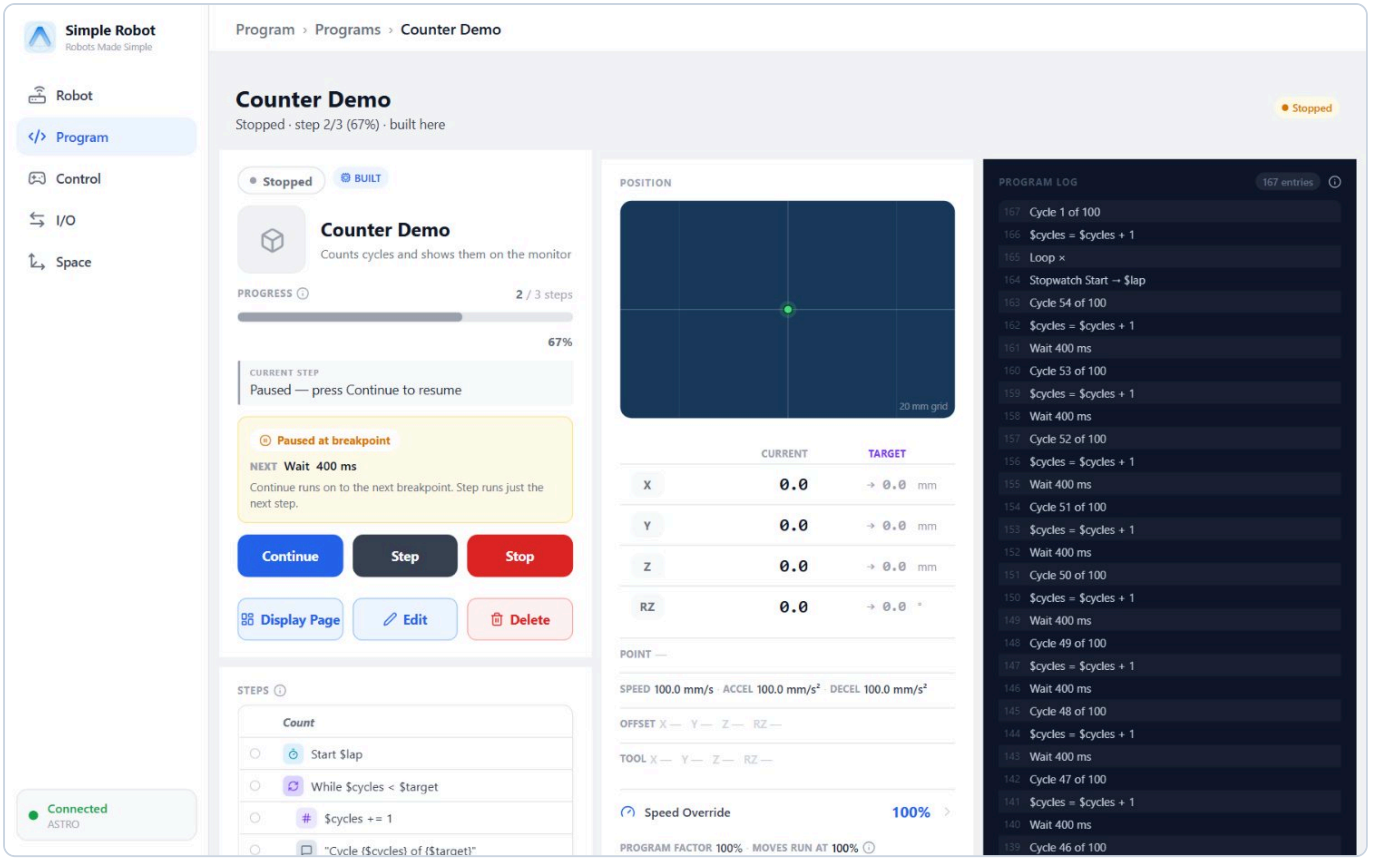


Figure 30. Paused at a breakpoint: the pause reason is shown, with Continue, Step and Stop.

Step mode

In step mode the program pauses before **every** step (pause reason *step*). **Step** runs the paused step and pauses before the next; **Continue** leaves step mode and runs on. Notes and disabled steps never pause, and a step that takes time (a Wait, the legs of a Jump, a vision run) is never paused half-way.

Run from step

Run from step starts the program at a chosen step:

- enclosing loops start at their first pass (count 0, for-each element 0; a while condition is not checked), and enclosing branches, Try bodies or catches are entered without evaluating anything;
- the steps before it do not run, so variables start at their declared values;
- a step inside a routine, a disabled step or a step inside a CNC block cannot be a starting point.

Background programs ignore breakpoints, step mode and run from step; a plain start clears them.

Debugging tips

- Put `status "…{$var}…"` steps at interesting places — the message appears on the monitor and in the log.
- Mark variables **monitor** to watch them live.
- Use `SetSpeedFactor 25` at the top while proving out a new program, and remove it after.
- Disable (`@off`) a section instead of deleting it while you test the rest.
- Test a formula with the expression tester before running it.

CHAPTER 12

Background Programs

A **background program** runs in parallel with the main program — watching a sensor, pulsing a conveyor, logging values, talking to another machine — while the main program moves the robot.

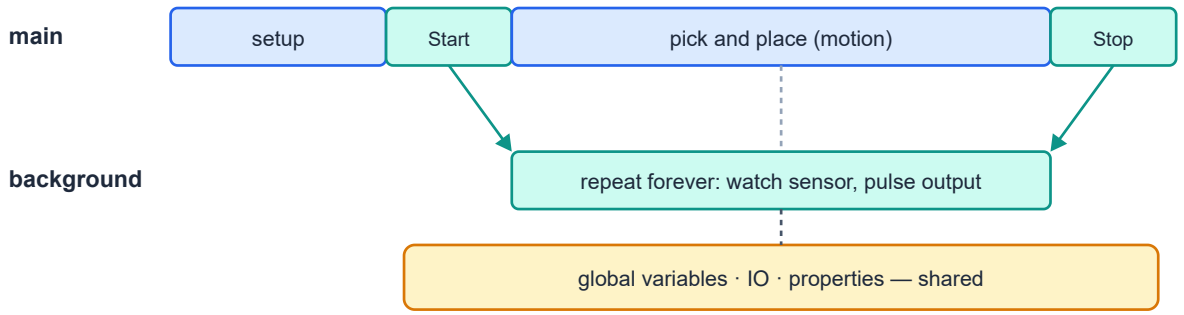


Figure 31. A background program runs beside the main program. It shares global variables and IO, skips motion, tool and homing steps, and can be stopped or waited for.

Making and starting one

Set the program's kind to **Background** in Program Settings (in a file: `@program {"isBackground": true}`). Start, stop and wait for it from another program:

```
StartBackground Conveyor_Watch       # starts it and continues at once
…
StopBackground Conveyor_Watch        # stops it
WaitForBackground Conveyor_Watch     # or: wait until it finishes by itself
```

A background program that ends itself (`exit`, `return`, its last step) ends only itself.

What runs in the background

Background programs **skip** the steps that would fight the main program for the robot. In the Instruction Reference each entry says whether the step runs in background programs.

Skipped in background	Runs in background
Move Linear, Move Joint, Jumps, Move Arc, Thread Move, Run Homing	Waits, variables, lists, flow control, Try
Set Speed (Linear / Joint), Set Speed Factor	Outputs, load cells, recordings, export
Set Tool, Set Local, Clear Local	Aux axes, gripper, conveyor, rotary
	Vision, HTTP, status, prompts, plugin steps

Breakpoints, step mode and run from step do not apply to background programs.

Sharing data

Each program run has its own variables. To share values between the main program and a background program, declare them **global** in both:

```
global var conveyorRuns = 0
```

IO and properties are shared by nature. A background program can also show a Prompt (one at a time per program).

Example

```
#!srs 1
@program {"name": "Conveyor Watch", "isBackground": true}
global var conveyorRuns = 0

repeat forever
  Wait until $stb.in1 == 0, timeout=10s
  SetOutput stb 2 on, pulse=500ms, blocking=on
  $conveyorRuns = $conveyorRuns + 1
  Wait 1s
end
```

CAUTION

A background program that writes a saved point (Save Position) changes it for every program; the validator warns (`pointWriteInBackground`).

CHAPTER 13

Vision

A **vision program** is a list of small tools that run on a camera image: locate something, inspect it, and output numbers. The Run Vision step runs one and copies its results into program variables.

Vision programs

Vision programs are built under **Program → Vision**. They follow the pattern used by commercial vision software — **locate → inspect → output**:

Tool family	Tools	Outputs
Locate	find line, find corner, find circle, blob, contour match, pattern match, ArUco marker, make fixture	a **pose** (`x` , `y` , `angle`), points, lines, items
Measure	caliper, distance, angle, corners, lines	distances, angles, points
Presence / count	pixel count, intensity, edge pixels, polygon — each can be split into a **grid** of cells	a main value per cell (`percent` , `mean` , `density` , `count`)
Read	barcode / QR code	text `value` , `format`

Any locating tool can be a **fixture** for later tools: their regions then move with the found part. Each tool has **limits** on its values; a tool *passes* when it ran, found what it looks for and meets its limits. The program passes when every tool that counts toward the pass passes.

Run Vision

```
var ok: bool = off
var edgeAngle = 0
var cells: list record
var parts: list point
RunVision Detect_Parts, frame=robot, pass=$ok
  Top_edge.angle -> $edgeAngle        # a value of a tool
  Slots.cells -> $cells               # one record per grid cell
  Top_edge.points -> $parts           # a Points list
end
```

Each row is `tool.value -> $variable` — the tool by name (underscore form for names with spaces). Variables named in rows are created if not declared.

Value	Variable	Written
any key of the tool's values: `angle` , `count` , `percent` , `distance` , `radius` , `x` , `y` …	number (text for a barcode's `value`)	the value; `x` / `y` converted by `frame=`
`passed`	boolean	1 when the tool's status is ok and every limit is met
`found`	boolean	the tool's `found` value
`status`	text	`ok` , `fail` , `notRun` or `error`
`cells`	Record list	per grid cell, row-major: `row` , `col` , `index` , `value` , `passed`

Value	Variable	Written
`items`	Record list	the tool's records (blobs, matches, markers, codes)
`points`	Point list	the tool's points
`lines`	Record list	`{x1, y1, x2, y2}` per line
`pose`	Record list	one record `{x, y, z, angle}` when found, else empty — read `$pose[0].x`

Coordinates

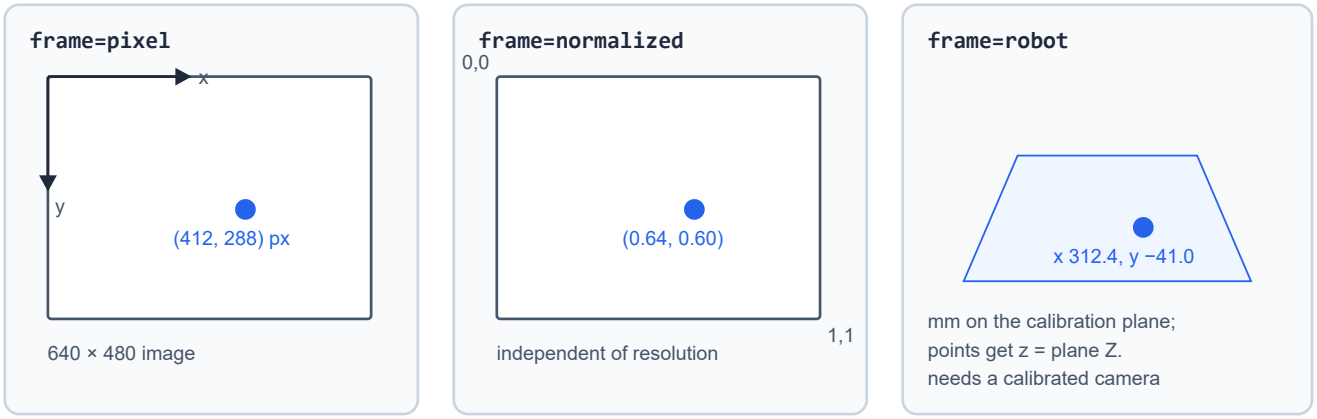


Figure 32. RunVision frame=: positions in pixels (origin top-left, y down), normalized 0–1, or robot millimetres on the camera's calibration plane.

`frame=`	Positions are
`pixel` (default)	pixels of the image, origin top-left, y down
`normalized`	0–1 of the image width and height
`robot`	millimetres on the camera's calibration plane; points get `z` = the plane's Z. Needs a calibrated camera (`cameraNotCalibrated` otherwise); `$camera.<id>.calibrated` tells.

Widths, radii and distances always stay in pixels. Angles are degrees, positive clockwise on screen.

Using the results

```
if not $ok
  raise "Inspection failed", code=inspectFail
end
for _, $i in $cells
  if $cells[$i].passed == 0
    status warning "Cell {$cells[$i].row},{$cells[$i].col} is empty"
  end
end
for _, $i in $parts
  JumpL $parts[$i], z=40          # with frame=robot, points are robot positions
end
```

A tool that did not run (its fixture was not found) writes empty lists and 0 for `passed`, `found` and `count`.

Timing and failures

Run Vision waits for a fresh result — up to 30 s by default. It fails with `visionTimeout` when no result comes in time and `visionFailed` when the vision processor is lost or the robot frame is not available. Both are catchable:

```
try retry=2, delay=500ms
  RunVision Detect_Parts, pass=$ok
catch $msg
  status error "Camera problem: {$msg}"
end
```

Images

```
var photo: image
CaptureImage $photo, camera=Top_Camera                # into an image variable
SaveImage "shots/part-{$time_ms}.jpg", camera=Top_Camera   # straight to a file
```

An image variable can be sent with HTTP Request (`send "photo" = image $photo`), handed to a plugin, exported, or shown on a display page. Both steps fail with `visionFailed` when the camera has no frame.

CHAPTER 14

Operator Prompts and Communication

Status messages

Status Update publishes a line to the monitor and the run log. The text is a template.

```
status "Placed {$count} parts"
status warning "Tray almost full"
status error "Vision failed: {$msg}"      # shown as an error — the program carries on
```

Asking the operator

Prompt shows a dialog with a message, 1–4 buttons and optionally a number or text input, and waits for the answer. The dialog appears in the app on whatever screen the operator is on.

```
var choice = 0
var qty = 0
prompt "Load the tray, then press Continue.", title="Setup", buttons=("Continue", "Skip"), result=$choice
if $choice == 1
  exit "Skipped by operator"
end
prompt "How many parts?", input=number, default=10, min=1, max=50, value=$qty
```

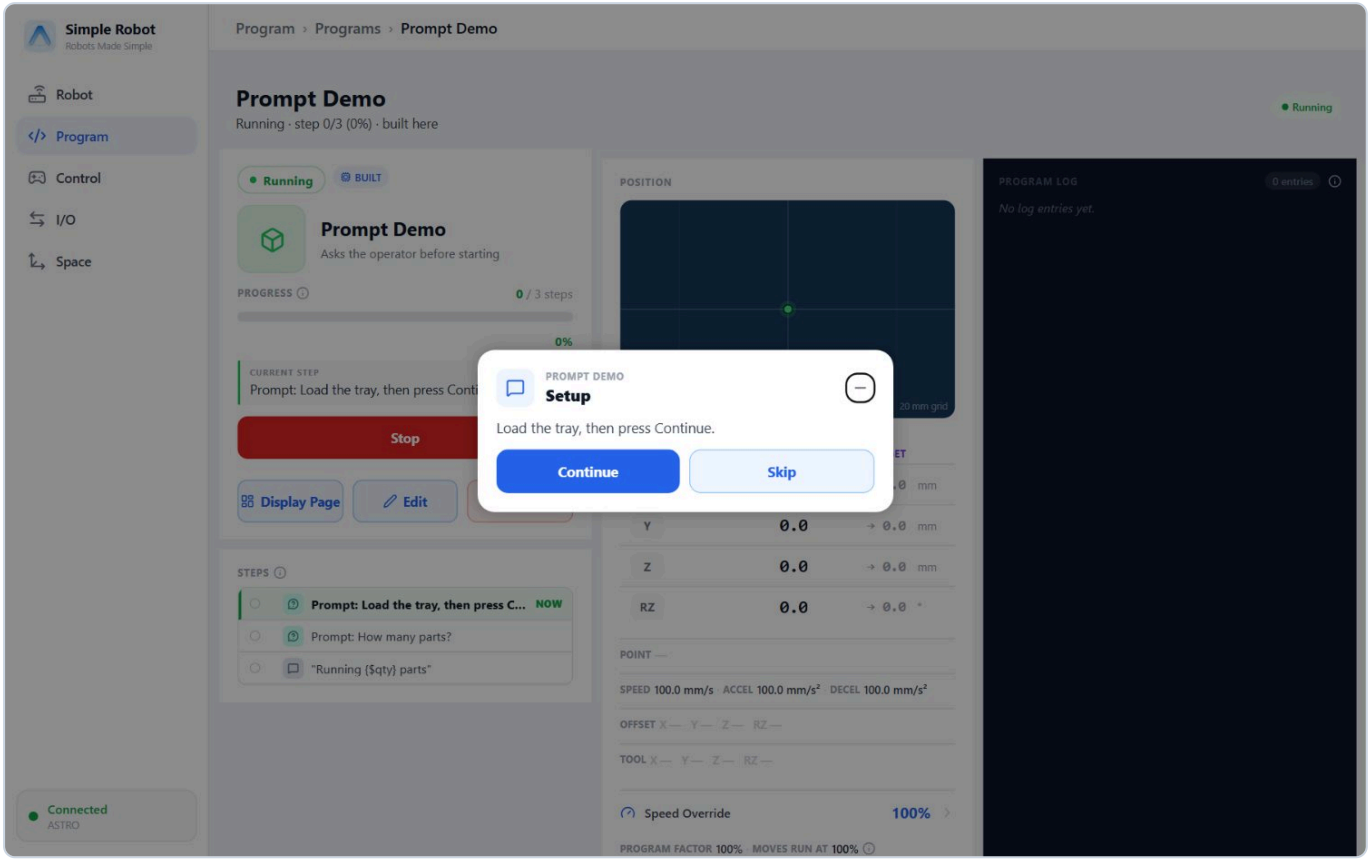


Figure 33. A Prompt step shows its dialog on whatever screen the operator is on.

Argument	Meaning
message (position 1)	The text, a template.
`title=`	Optional title.
`buttons=(…)`	1–4 labels (default `OK`).
`result=$v`	A number variable gets the button index (from 0); a text variable gets its label.
`input=number | text`	Ask for a value; `default=`, and for numbers `min=` / `max=`.
`value=$v`	Receives the typed value.
`timeout=`	Give up after this long (0 = wait forever).
`onTimeout=<n>`	The button pressed on timeout. Without it a timeout fails with `promptTimeout`.

The first answer wins (from any connected app). A **Stop** keeps the dialog on screen but refuses answers; **Continue** shows it again with its timeout restarted.

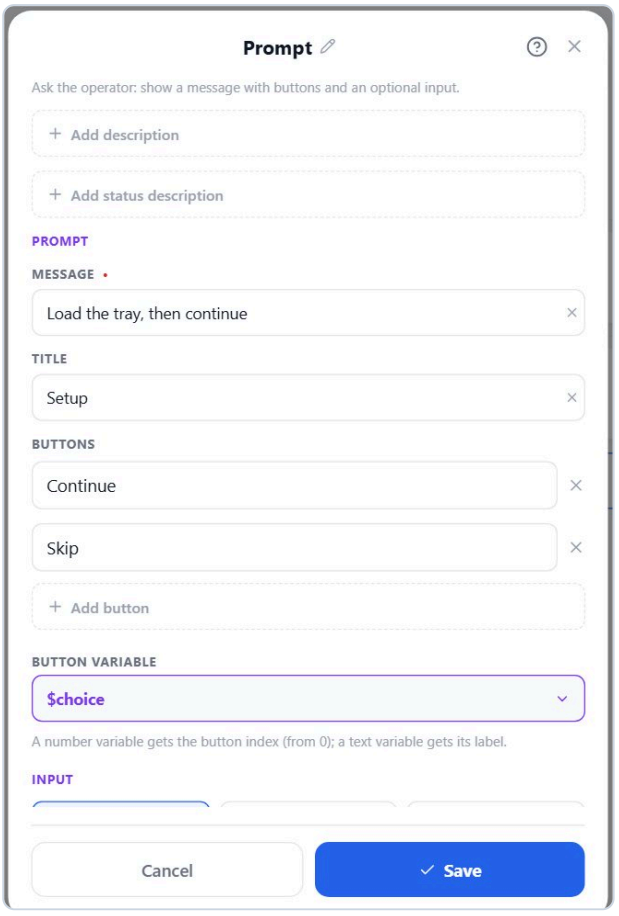


Figure 34. The Prompt step editor: message, title, buttons and the result variable.

For a simple "check this, then continue", Pause Program after a status message also works.

HTTP

HTTP Request POSTs JSON to another machine and can read values from the reply.

```
var count = 0
var next = 0
var photo: image
HttpRequest "http://192.168.4.10:8080/report", wait=on, timeout=5s
  send "count" = $count              # a number (any expression)
  send "photo" = image $photo        # an image, base64 JPEG
  receive "next" -> $next            # a key of the JSON reply
end
```

- Rows: `send "key" = expression`, `send "key" = list $var`, `send "key" = image $var`, and `receive "key" -> $var`.
- With `wait=on` (default) it fails `httpFailed` (network error, non-2xx status) or `httpTimeout` — both catchable. An empty or non-JSON reply is not a failure. With `wait=off` it never fails.

HTTP Receive waits for another machine to POST JSON to the controller at `/webhook/<name>`:

```
var x = 0
HttpReceive "orders", timeout=60s
  receive "x" -> $x
end
```

It fails `receiveTimeout` when nothing arrives in time.

Exporting data

Export Data writes values to a CSV or JSON file on the controller — one column (or key) per row:

```
ExportData csv "logs/run-{$time_ms}.csv"
  "parts" = $count
  "weights" = list $weights
  "photo" = image $photo
end
```

The path is a template; a relative path is relative to the controller's folder, and missing folders are created.

CHAPTER 15

Plugins

A **plugin** extends the controller with code of its own — a bench scale, a barcode reader, a PLC gateway, a database logger. It runs as a separate process (Python, C# or any executable) that the controller starts and supervises, and it can contribute three things to programs:

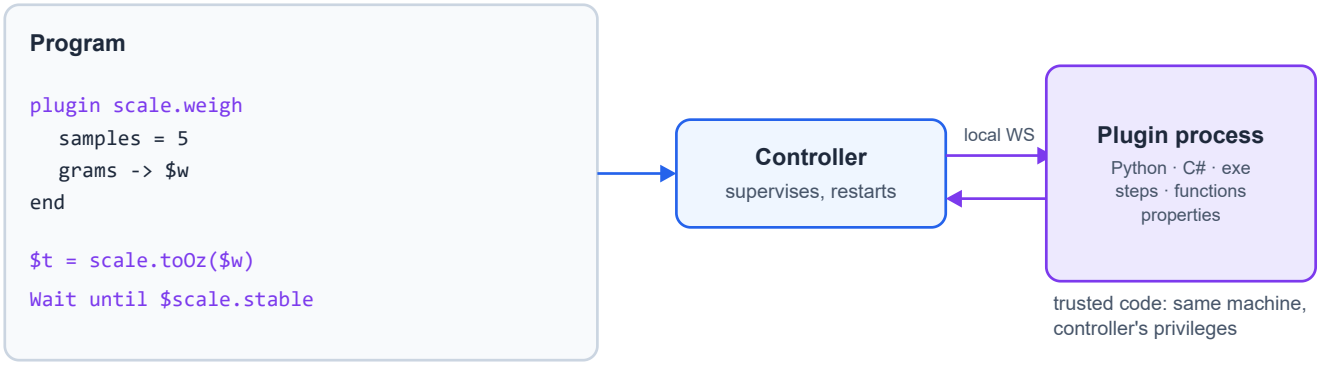


Figure 35. A plugin is a separate process the controller starts. It connects back over a local WebSocket and contributes steps, expression functions and live properties.

Contribution	Used as	Example
Steps	blocks in the Plugins category; `plugin <id>.<step>` in text	`plugin scale.weigh`
Functions	in any expression	`scale.tare()`, `scale.toOz($g)`
Properties	live read-only values	`$scale.weight`, `$scale.stable`

Plugins are installed and configured under **Program** → **Plugins**. They are trusted code: they run on the same machine with the controller's privileges.

Plugin steps

```
var w = 0
var history: list number
plugin scale.weigh, timeout=5s
  samples = $n * 2          # unquoted: an expression
  unit = "g"                # quoted: the exact text
  weights = history         # a list parameter: the list variable's name
  grams -> $w               # an output into a variable
end
```

- Parameter rows are typed by the plugin's manifest: numbers and booleans take expressions, text takes a template, an option is taken literally, a point takes a point name (or `$pts[$i]`, a grid or stack cell), a list or image takes a variable name.
- A missing parameter uses the manifest's default; a required one without a default stops the program.
- Output rows `key -> $var` write the plugin's results; an output the plugin does not return is skipped.
- While the step runs the monitor shows the plugin's progress. A Stop cancels the step; Continue sends it again.
- Plugin steps run in background programs too.

Failure	Code (catchable)
The plugin is not running	`pluginNotRunning`
The plugin answered with an error	`pluginFailed`
No answer within the timeout	`pluginTimeout`

Plugin functions and properties

```
$t = scale.toOz($w)              # a function call
Wait until $scale.stable          # a live property
```

A function call blocks the program until the plugin answers (default limit 250 ms) — prefer a **property** for anything read every tick, such as a wait or a stop condition. A property the plugin has not set yet is unknown and stops the program like a misspelt variable.

The validator knows the installed plugins: an unknown plugin or step, a missing required parameter or an output into the wrong kind of variable is reported before the run, and a plugin that is not running is a warning.

Part II — Reference

16 Instruction Reference
17 Functions and Properties Reference
18 Error and Problem Codes

CHAPTER 16

Instruction Reference

This chapter has one entry per step type, grouped by the categories of the Blocks step picker. Every entry has the same layout:

Part	Contents
Heading	The step's name in Blocks, and its keyword in Simple Robot Script.
Summary line	What it does, its category, and whether it runs in background programs.
Syntax	The text form. `<…>` is a value you supply, `[…]` is optional, `a | b` is a choice.
Parameters	Each field: where it goes in the text (*position n* = the n-th value after the keyword, `name=` = a named value), its type, unit and default. **fx** marks fields that also take an expression.
Description and notes	Behaviour and restrictions.
Errors	The runtime error codes the step can fail with; *catchable* ones can be handled with a Try. Expression errors (`unknownVariable`, `expressionSyntax`) apply to every field that takes an expression.
Example	The same example in Blocks (a screenshot) and in text.
See also	Related steps.

The parameter tables, syntax lines, units and defaults are generated from the controller's step catalog (`GetStepCatalog`).

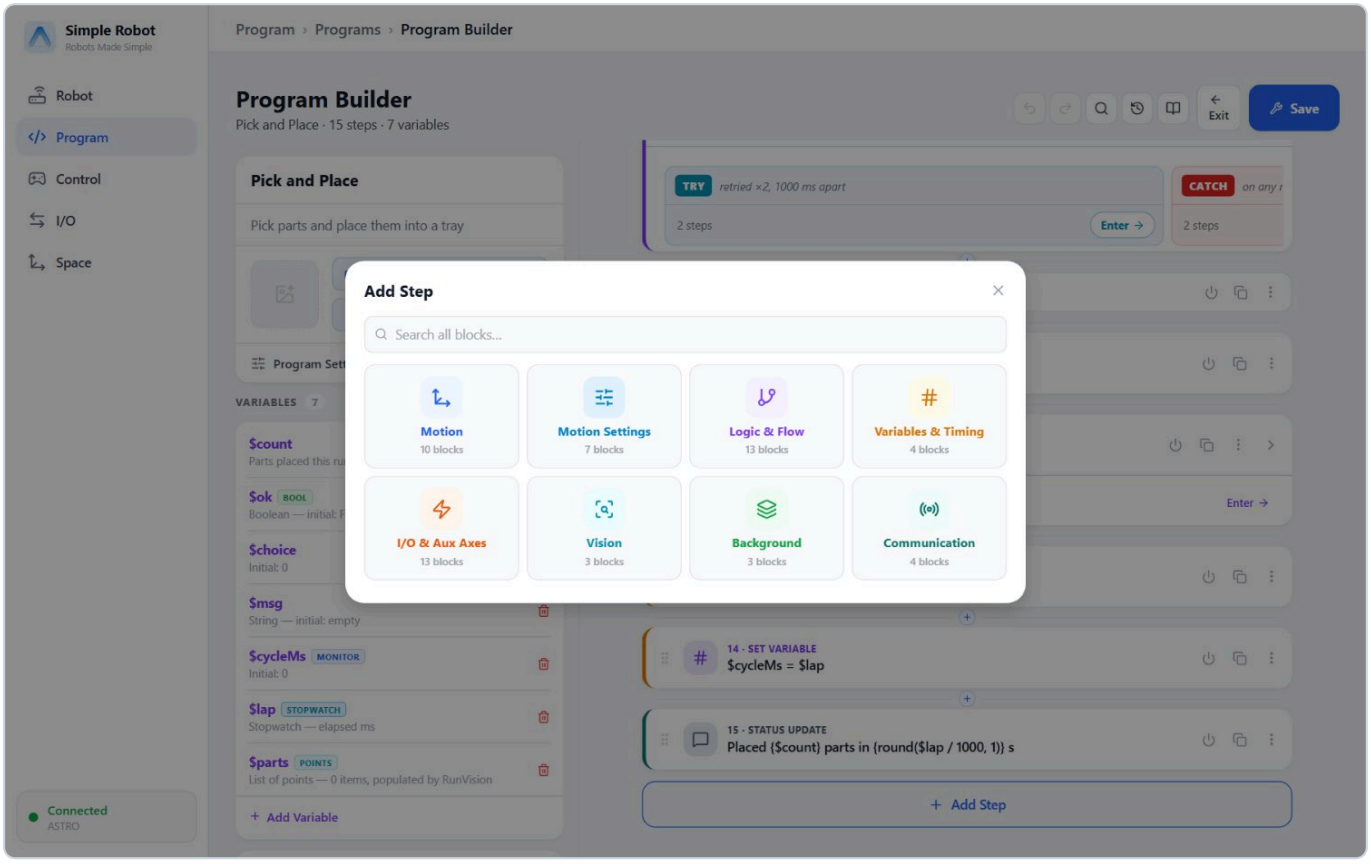


Figure 36. Add Step opens the step picker, grouped by category.

Motion

Send the robot somewhere — points, jumps, arcs, threads and homing.

Step	Keyword	Description	Page
Move Linear	`MoveL`	Move to a target in a straight line.	59
Move Joint	`MoveJ`	Move to a target by joint interpolation.	61
Jump Linear	`JumpL`	Lift, move linearly over the target, then lower.	63
Jump Joint	`JumpJ`	Lift, move by joint interpolation over the target, then lower.	65
Move Arc	`MoveC`	Move along a circular arc through a via point to the end target.	67
Thread Move	`ThreadMove`	Move the tool in Z while rotating RZ to follow a thread.	68
Run Homing	`RunHoming`	Run the full homing sequence and wait for it to finish.	69
Set Joint Position	`SetJointPosition`	Declare one joint's value with no motion (manual homing).	70
CNC Program	`CncProgram`	Follow a toolpath generated from a DXF or SVG file.	71
Save Position	`SavePosition`	Store the current position in a Points list or a saved point.	71

Move Linear

MoveL

Motion Background programs: No

Move to a target in a straight line.

SYNTAX

```
MoveL <target>[, speed=…][, blend=…][, offset=(…)][, until=…]
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Target fx A saved point, a $points[i] element, a template naming a point, a grid or stack cell — or nothing for the current position (use with offsets).	`position 1`	target		
Speed fx Overrides the program's speed for this move only.	`speed=`	number or (expr)	mm/s	
Acceleration fx	`accel=`	number or (expr)	mm/s²	
Deceleration fx	`decel=`	number or (expr)	mm/s²	
Blend into next move Round the corner into the next move instead of stopping.	`blend=`	on / off		`off`
Blend radius fx Empty = the radius set by Set Blend Radius. when blend = true	`blendRadius=`	number or (expr)	mm	
Offset fx Added to the target (relative move when there is no target).	`offset=`	(x=..., y=..., z=..., rx=..., ry=..., rz=...)	mm / deg	
Override fx Replaces an axis of the target outright.	`set=`	(x=..., y=..., z=..., rx=..., ry=..., rz=...)	mm / deg	

Parameter	In text	Type / values	Unit	Default
Tool offset fx Applied in the tool frame at execution time.	`toolOffset=`	(x=..., y=..., z=..., rx=..., ry=..., rz=...)	mm / deg	
Local frame Run this move in a local frame instead of the active one.	`local=`	local name		
Stop condition fx Watched every tick while the move runs, e.g. $io.Nano.Load >= 2000.	`until=`	condition		
When true	`untilMode=`	stop | record `stop` Stop the move · `record` Record the position, keep moving		`stop`
Position variable A Points list set to the one pose where the condition fired.	`untilPos=`	$variable		
Fired flag Set to 1 if the condition fired during the move, else 0.	`untilFlag=`	$variable		

DESCRIPTION AND NOTES

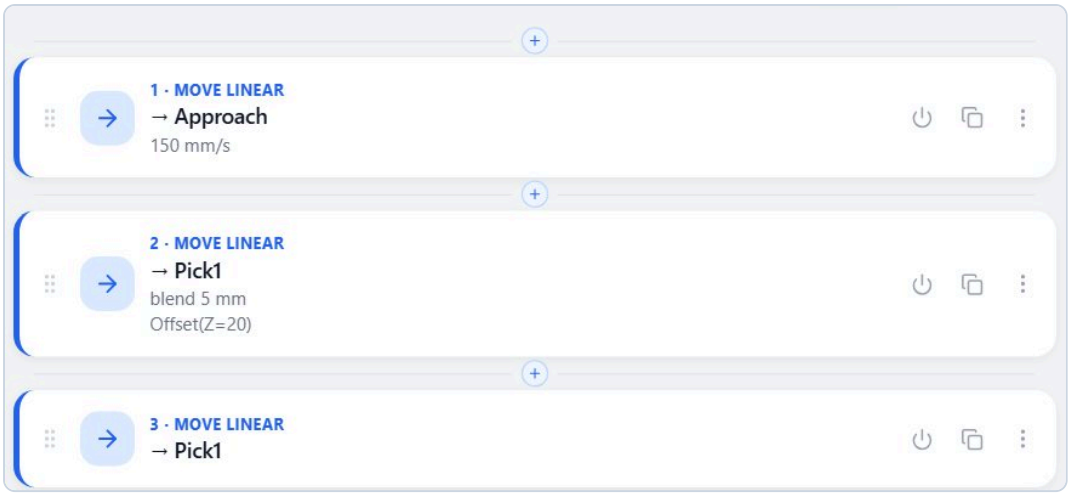
- The tool centre point travels a straight line in Cartesian space.
- `blend=on` (or `blend=<radius>`) rounds the corner into the next blend-enabled move instead of stopping; consecutive blended moves run as one continuous path. Without a radius the value from Set Blend Radius is used.
- A Move Linear with a stop condition never blends.
- The target is resolved from the first of: a grid cell, a stack slot, a `$list[i]` element or template (`pointNameExpr`), a saved point. With no target the move starts from the current position, so `offset=` makes it relative.
- `offset=` is added to the target, `set=` replaces an axis outright, `toolOffset=` is applied in the tool frame; each axis may be an expression.
- Program moves are scaled by the operator's speed override and by Set Speed Factor.
- Not run in background programs (motion is skipped there).
- A stop condition (`until=`) is checked every controller tick while the move runs. `untilMode=stop` ends the move where it is and continues with the next step (no error); `record` lets the move finish and stores the pose where the condition first held in `untilPos=` (a one-element Points list). `untilFlag=` gets 1 if it fired, else 0.

ERRORS

Code	Kind	When
`pointNotFound`	catchable	A template or `$list[i]` target names no saved point, or the Points list is empty.
`motionFault`	fatal	The motion system reports a fault (fatal).
`jointLimit`	fatal	The target is outside a joint's soft limits (fatal).
`driverDisconnected`	fatal	The motor driver is not connected (fatal).
`unknownVariable`	fatal	An expression names a variable that does not exist (fatal).
`expressionSyntax`	fatal	An expression does not parse (fatal).

EXAMPLE

BLOCKS



TEXT

```
MoveL Approach, speed=150
MoveL Pick1, offset=(z=20), blend=5
MoveL Pick1
```

See also: Move Joint (MoveJ, p. 61) · Jump Linear (JumpL, p. 63) · Move Arc (MoveC, p. 67) · Set Speed (Linear) (SetSpeedL, p. 73) · Set Blend Radius (SetBlendRadius, p. 75)

Move Joint

MoveJ

Motion Background programs: No

Move to a target by joint interpolation.

SYNTAX

```
MoveJ <target>[, speed=…][, offset=(…)][, until=…]
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Target fx A saved point, a $points[i] element, a template naming a point, a grid or stack cell — or nothing for the current position (use with offsets).	`position 1`	target		
Speed fx Overrides the program's speed for this move only.	`speed=`	number or (expr)	mm/s	
Acceleration fx	`accel=`	number or (expr)	mm/s²	
Deceleration fx	`decel=`	number or (expr)	mm/s²	
Offset fx Added to the target (relative move when there is no target).	`offset=`	(x=..., y=..., z=..., rx=..., ry=..., rz=...)	mm / deg	
Override fx Replaces an axis of the target outright.	`set=`	(x=..., y=..., z=..., rx=..., ry=..., rz=...)	mm / deg	
Tool offset fx Applied in the tool frame at execution time.	`toolOffset=`	(x=..., y=..., z=..., rx=..., ry=..., rz=...)	mm / deg	

Parameter	In text	Type / values	Unit	Default
Local frame Run this move in a local frame instead of the active one.	`local=`	local name		
Stop condition fx Watched every tick while the move runs, e.g. $io.Nano.Load >= 2000.	`until=`	condition		
When true	`untilMode=`	stop | record `stop` Stop the move · `record` Record the position, keep moving		`stop`
Position variable A Points list set to the one pose where the condition fired.	`untilPos=`	$variable		
Fired flag Set to 1 if the condition fired during the move, else 0.	`untilFlag=`	$variable		

DESCRIPTION AND NOTES

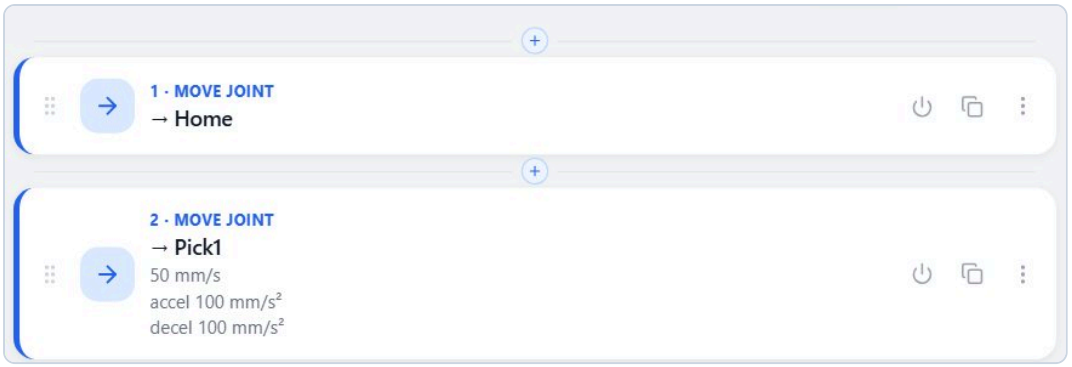
- All joints start and stop together; the tool path is generally *not* a straight line. Usually the fastest way between two poses in free space.
- Move Joint does not blend.
- The target is resolved from the first of: a grid cell, a stack slot, a `$list[i]` element or template (`pointNameExpr`), a saved point. With no target the move starts from the current position, so `offset=` makes it relative.
- `offset=` is added to the target, `set=` replaces an axis outright, `toolOffset=` is applied in the tool frame; each axis may be an expression.
- Program moves are scaled by the operator's speed override and by Set Speed Factor.
- Not run in background programs (motion is skipped there).
- A stop condition (`until=`) is checked every controller tick while the move runs. `untilMode=stop` ends the move where it is and continues with the next step (no error); `record` lets the move finish and stores the pose where the condition first held in `untilPos=` (a one-element Points list). `untilFlag=` gets 1 if it fired, else 0.

ERRORS

Code	Kind	When
`pointNotFound`	catchable	A template or `$list[i]` target names no saved point.
`motionFault`	fatal	The motion system reports a fault (fatal).
`jointLimit`	fatal	The target is outside a joint's soft limits (fatal).
`driverDisconnected`	fatal	The motor driver is not connected (fatal).
`unknownVariable`	fatal	An expression names a variable that does not exist (fatal).
`expressionSyntax`	fatal	An expression does not parse (fatal).

EXAMPLE

BLOCKS



TEXT

```
MoveJ Home
MoveJ Pick1, speed=50, accel=100, decel=100
```

See also: Move Linear (MoveL, p. 59) · Jump Joint (JumpJ, p. 65) · Set Speed (Joint) (SetSpeedJ, p. 74)

Jump Linear

`JumpL`

Motion Background programs: No

Lift, move linearly over the target, then lower.

SYNTAX

```
JumpL <target>, z=…[, zStart=…][, zEnd=…][, speed=…]
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Target fx A saved point, a $points[i] element, a template naming a point, a grid or stack cell — or nothing for the current position (use with offsets).	`position 1`	target		
Lift height fx Z used for the lift and lower legs.	`z=`	number or (expr)	mm	
Lift height (start) fx Overrides the lift leg only.	`zStart=`	number or (expr)	mm	
Lift height (end) fx Overrides the lower leg only.	`zEnd=`	number or (expr)	mm	
Speed fx Overrides the program's speed for this move only.	`speed=`	number or (expr)	mm/s	
Acceleration fx	`accel=`	number or (expr)	mm/s²	
Deceleration fx	`decel=`	number or (expr)	mm/s²	
Blend into next move Round the corner into the next move instead of stopping.	`blend=`	on / off		`off`
Blend radius fx Empty = the radius set by Set Blend Radius. when blend = true	`blendRadius=`	number or (expr)	mm	

Parameter	In text	Type / values	Unit	Default
Offset fx Added to the target (relative move when there is no target).	`offset=`	(x=..., y=..., z=..., rx=..., ry=..., rz=...)	mm / deg	
Override fx Replaces an axis of the target outright.	`set=`	(x=..., y=..., z=..., rx=..., ry=..., rz=...)	mm / deg	
Tool offset fx Applied in the tool frame at execution time.	`toolOffset=`	(x=..., y=..., z=..., rx=..., ry=..., rz=...)	mm / deg	
Local frame Run this move in a local frame instead of the active one.	`local=`	local name		
Stop condition fx Watched every tick while the move runs, e.g. $io.Nano.Load >= 2000.	`until=`	condition		
When true	`untilMode=`	stop | record `stop` Stop the move · `record` Record the position, keep moving		`stop`
Position variable A Points list set to the one pose where the condition fired.	`untilPos=`	$variable		
Fired flag Set to 1 if the condition fired during the move, else 0.	`untilFlag=`	$variable		

DESCRIPTION AND NOTES

- Three legs: lift straight up to the height `z=`, travel over the target, lower onto the target. The heights are absolute Z values, not offsets. `zStart=` / `zEnd=` set the lift and lower heights separately (the traverse runs from one to the other); `z=` is required unless both are given.
- Blending applies to the traverse leg.
- With a stop condition the jump takes the stepped (non-blended) path and the watch spans all three legs; in stop mode the remaining legs are skipped.
- The target is resolved from the first of: a grid cell, a stack slot, a `$list[i]` element or template (`pointNameExpr`), a saved point. With no target the move starts from the current position, so `offset=` makes it relative.
- `offset=` is added to the target, `set=` replaces an axis outright, `toolOffset=` is applied in the tool frame; each axis may be an expression.
- Program moves are scaled by the operator's speed override and by Set Speed Factor.
- Not run in background programs (motion is skipped there).

ERRORS

Code	Kind	When
`pointNotFound`	catchable	A template or `$list[i]` target names no saved point.
`motionFault`	fatal	The motion system reports a fault (fatal).
`jointLimit`	fatal	The target is outside a joint's soft limits (fatal).
`driverDisconnected`	fatal	The motor driver is not connected (fatal).
`unknownVariable`	fatal	An expression names a variable that does not exist (fatal).
`expressionSyntax`	fatal	An expression does not parse (fatal).

EXAMPLE

BLOCKS



TEXT

```
JumpL Place1, z=60
JumpL grid Tray[2, 1], z=40, zEnd=50
```

See also: Jump Joint (JumpJ, p. 65) · Move Linear (MoveL, p. 59) · Gripper (GripperAction, p. 107)

Jump Joint

JumpJ

Motion Background programs: No

Lift, move by joint interpolation over the target, then lower.

SYNTAX

```
JumpJ <target>, z=…[, zStart=…][, zEnd=…][, speed=…]
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Target fx A saved point, a $points[i] element, a template naming a point, a grid or stack cell — or nothing for the current position (use with offsets).	`position 1`	target		
Lift height fx	`z=`	number or (expr)	mm	
Lift height (start) fx	`zStart=`	number or (expr)	mm	
Lift height (end) fx	`zEnd=`	number or (expr)	mm	
Speed fx Overrides the program's speed for this move only.	`speed=`	number or (expr)	mm/s	
Acceleration fx	`accel=`	number or (expr)	mm/s²	
Deceleration fx	`decel=`	number or (expr)	mm/s²	
Offset fx Added to the target (relative move when there is no target).	`offset=`	(x=..., y=..., z=..., rx=..., ry=..., rz=...)	mm / deg	
Override fx Replaces an axis of the target outright.	`set=`	(x=..., y=..., z=..., rx=..., ry=..., rz=...)	mm / deg	
Tool offset fx Applied in the tool frame at execution time.	`toolOffset=`	(x=..., y=..., z=..., rx=..., ry=..., rz=...)	mm / deg	
Local frame Run this move in a local frame instead of the active one.	`local=`	local name		

Parameter	In text	Type / values	Unit	Default
Stop condition fx Watched every tick while the move runs, e.g. $io.Nano.Load >= 2000.	`until=`	condition		
When true	`untilMode=`	stop | record `stop` Stop the move · `record` Record the position, keep moving		`stop`
Position variable A Points list set to the one pose where the condition fired.	`untilPos=`	$variable		
Fired flag Set to 1 if the condition fired during the move, else 0.	`untilFlag=`	$variable		

DESCRIPTION AND NOTES

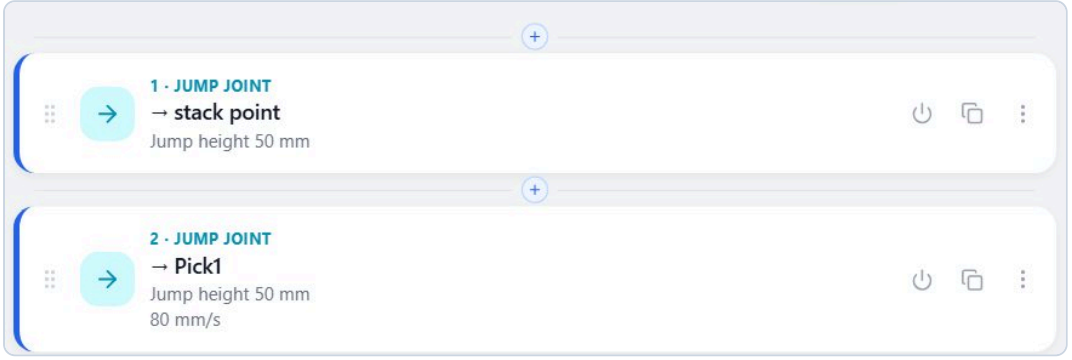
- Like Jump Linear, but each leg is a joint-interpolated move — faster, with a curved path between the lift and lower points.
- The target is resolved from the first of: a grid cell, a stack slot, a `$list[i]` element or template (`pointNameExpr`), a saved point. With no target the move starts from the current position, so `offset=` makes it relative.
- `offset=` is added to the target, `set=` replaces an axis outright, `toolOffset=` is applied in the tool frame; each axis may be an expression.
- Program moves are scaled by the operator's speed override and by Set Speed Factor.
- Not run in background programs (motion is skipped there).

ERRORS

Code	Kind	When
`pointNotFound`	catchable	A template or `$list[i]` target names no saved point.
`motionFault`	fatal	The motion system reports a fault (fatal).
`jointLimit`	fatal	The target is outside a joint's soft limits (fatal).
`driverDisconnected`	fatal	The motor driver is not connected (fatal).
`unknownVariable`	fatal	An expression names a variable that does not exist (fatal).
`expressionSyntax`	fatal	An expression does not parse (fatal).

EXAMPLE

BLOCKS



TEXT

```
JumpJ stack Bins[0], z=50
JumpJ Pick1, z=50, speed=80
```

See also: Jump Linear (JumpL, p. 63) · Move Joint (MoveJ, p. 61)

Move Arc

MoveC

Motion Background programs: No

Move along a circular arc through a via point to the end target.

SYNTAX

```
MoveC <end>, via=<point>[, speed=…][, tolerance=…]
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Target fx A saved point, a $points[i] element, a template naming a point, a grid or stack cell — or nothing for the current position (use with offsets).	`position 1`	target		
Via point * A point the arc passes through (saved point, $points[i] element or template).	`via=`	target		
Speed fx Overrides the program's speed for this move only.	`speed=`	number or (expr)	mm/s	
Acceleration fx	`accel=`	number or (expr)	mm/s²	
Deceleration fx	`decel=`	number or (expr)	mm/s²	
Arc tolerance Largest allowed gap between the true arc and its straight segments.	`tolerance=`	number at least 0.001	mm	`0.1`
Offset fx Added to the target (relative move when there is no target).	`offset=`	(x=…, y=…, z=…, rx=…, ry=…, rz=…)	mm / deg	
Override fx Replaces an axis of the target outright.	`set=`	(x=…, y=…, z=…, rx=…, ry=…, rz=…)	mm / deg	
Local frame Run this move in a local frame instead of the active one.	`local=`	local name		

DESCRIPTION AND NOTES

- Follows the circle through the start (where the robot is), the via point and the end, in any plane.
- The arc is cut into equal chords that stay within `tolerance=` of the true circle (and span at most 2°), sent as one continuous path; the robot stops exactly on the end.
- RX / RY / RZ change linearly from the start's to the end's; the via point's orientation is not used.
- `offset=`, `set=` and `local=` apply to both the via and the end. Stop conditions are not supported; tool offset and blending are ignored.
- Not run in background programs.

ERRORS

Code	Kind	When
`arcDegenerate`	catchable	Start, via and end are on one line, or two of them are the same.
`pointNotFound`	catchable	The via or end names no saved point.
`motionFault`	fatal	The motion system reports a fault (fatal).
`jointLimit`	fatal	The target is outside a joint's soft limits (fatal).
`driverDisconnected`	fatal	The motor driver is not connected (fatal).

EXAMPLE

BLOCKS



TEXT

```
MoveL Pick1
MoveC Place1, via=Via1, speed=80, tolerance=0.1
```

See also: Move Linear (MoveL, p. 59) · Set Speed (Linear) (SetSpeedL, p. 73)

Thread Move

`ThreadMove`

Motion Background programs: No

Move the tool in Z while rotating RZ to follow a thread.

SYNTAX

```
ThreadMove distance=…, pitch=…[, peck=on, peckDepth=…, peckRetract=…][, exitHeight=…][, reverseOut=on]
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Distance * fx	`distance=`	number or (expr)	mm	
Pitch * fx	`pitch=`	number or (expr)	mm	
Peck	`peck=`	on / off		`off`
Peck depth fx when threadPeck = true	`peckDepth=`	number or (expr)	mm	
Peck retract fx Empty = retract fully to the start each peck. when threadPeck = true	`peckRetract=`	number or (expr)	mm	
Exit height fx	`exitHeight=`	number or (expr)	mm	
Reverse out	`reverseOut=`	on / off		`off`
Speed fx Overrides the program's speed for this move only.	`speed=`	number or (expr)	mm/s	
Acceleration fx	`accel=`	number or (expr)	mm/s²	
Deceleration fx	`decel=`	number or (expr)	mm/s²	

DESCRIPTION AND NOTES

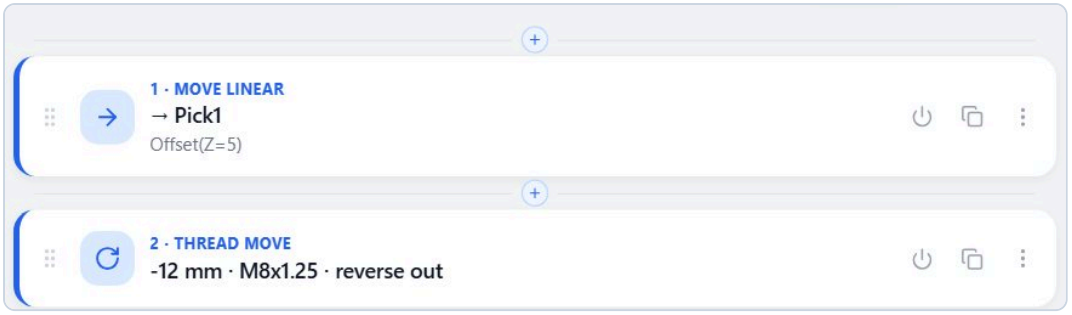
- Moves the tool `distance=` along Z while turning RZ by one revolution per `pitch=` mm — screwing in or tapping.
- `peck=on` advances in `peckDepth=` steps, retracting by `peckRetract=` between pecks (empty = back to the start each peck). `exitHeight=` and `reverseOut=on` control how the tool leaves.
- Not run in background programs.

ERRORS

Code	Kind	When
`motionFault`	fatal	The motion system reports a fault (fatal).
`jointLimit`	fatal	The target is outside a joint's soft limits (fatal).
`driverDisconnected`	fatal	The motor driver is not connected (fatal).

EXAMPLE

BLOCKS



TEXT

```
MoveL Pick1, offset=(z=5)
ThreadMove distance=-12, pitch=1.25, reverseOut=on
```

See also: Move Linear (MoveL, p. 59) · Set Tool (SetTool, p. 76)

Run Homing

`RunHoming`

Motion Background programs: No

Run the full homing sequence and wait for it to finish.

SYNTAX

```
RunHoming
```

PARAMETERS

No parameters.

DESCRIPTION AND NOTES

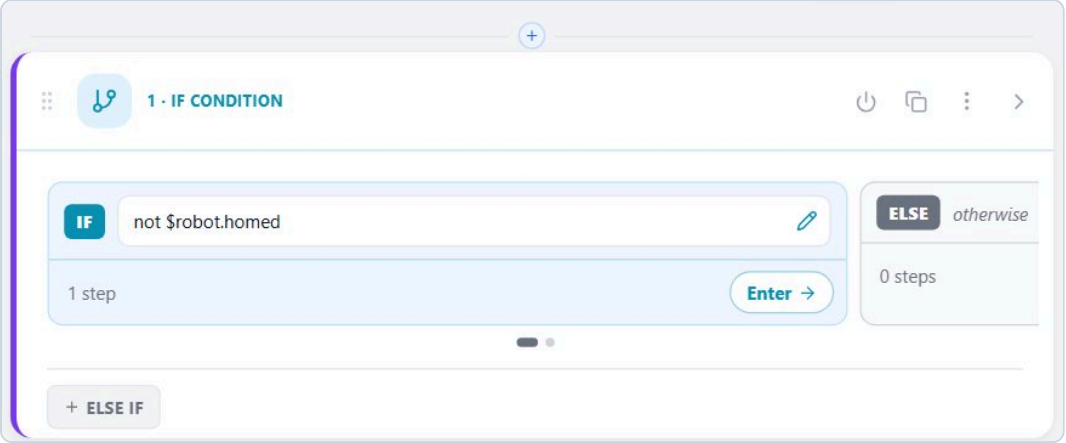
- Runs the full homing sequence and waits until it finishes.
- Not run in background programs.

ERRORS

Code	Kind	When
`motionFault`	fatal	The motion system reports a fault (fatal).
`jointLimit`	fatal	The target is outside a joint's soft limits (fatal).
`driverDisconnected`	fatal	The motor driver is not connected (fatal).

EXAMPLE

BLOCKS



TEXT

```
if not $robot.homed
  RunHoming
end
```

See also: Set Joint Position (SetJointPosition, p. 70)

Set Joint Position

`SetJointPosition`

Motion Background programs: Yes

Declare one joint's value with no motion (manual homing).

SYNTAX

```
SetJointPosition J<n>, <value>
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Joint	`position 1`	number `0` J1 · `1` J2 · `2` J3 · `3` J4		`0`
Value* fx	`position 2`	number or (expr)	deg / mm	

DESCRIPTION AND NOTES

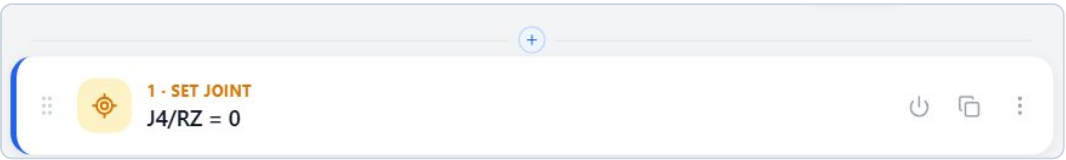
- Declares the current value of one joint without moving it (manual homing). `J1` … `J4`.

ERRORS

This step raises no runtime error codes of its own.

EXAMPLE

BLOCKS



TEXT

```
SetJointPosition J4, 0
```

See also: Run Homing (RunHoming, p. 69)

CNC Program

`CncProgram`

Motion Background programs: Yes

Follow a toolpath generated from a DXF or SVG file.

SYNTAX

```
CncProgram "<file>"[, safeZ=…]
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
File	`position 1`	text		
Safe Z	`safeZ=`	number	mm	

DESCRIPTION AND NOTES

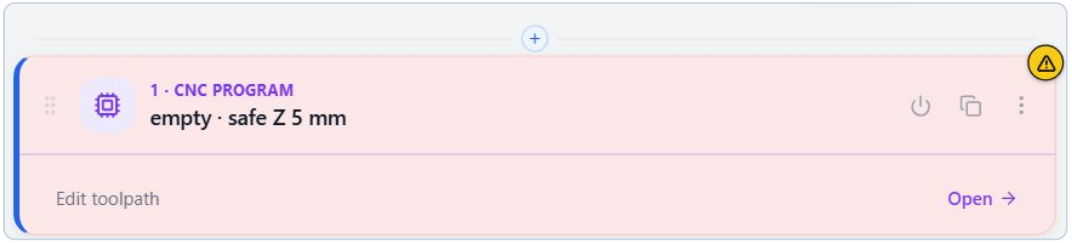
- Follows a toolpath generated from a DXF or SVG file. The toolpath itself is edited in the CNC builder and travels in the text as an `@with {…}` line.
- At run time the toolpath expands into linear moves (and thread moves). Run from step cannot start inside it.

ERRORS

Code	Kind	When
`motionFault`	fatal	The motion system reports a fault (fatal).
`jointLimit`	fatal	The target is outside a joint's soft limits (fatal).
`driverDisconnected`	fatal	The motor driver is not connected (fatal).

EXAMPLE

BLOCKS



TEXT

```
CncProgram "part.dxf", safeZ=5
```

See also: Move Linear (MoveL, p. 59) · Thread Move (ThreadMove, p. 68)

Save Position

`SavePosition`

Motion Background programs: Yes

Store the current position in a Points list or a saved point.

SYNTAX

```
SavePosition $<list>[, mode=replace|append|insert|set][, index=…][, source=actual|target] | SavePosition
point <name>
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Save to	`to=`	variable | point `variable` Points variable · `point` Saved point		`variable`
Points variable when savePositionTo = variable	`position 1`	$list		
Saved point Overwritten (or created) with the current position; recorded in the points history. when savePositionTo = point	`point=`	point name		
Mode when savePositionTo = variable	`mode=`	replace | append | insert | set `replace` Replace the list · `append` Append · `insert` Insert at index · `set` Set at index		`replace`
Index fx when savePositionMode = insert / set	`index=`	number or (expr)		
Position	`source=`	actual | target `actual` Actual position · `target` Last commanded target		`actual`

DESCRIPTION AND NOTES

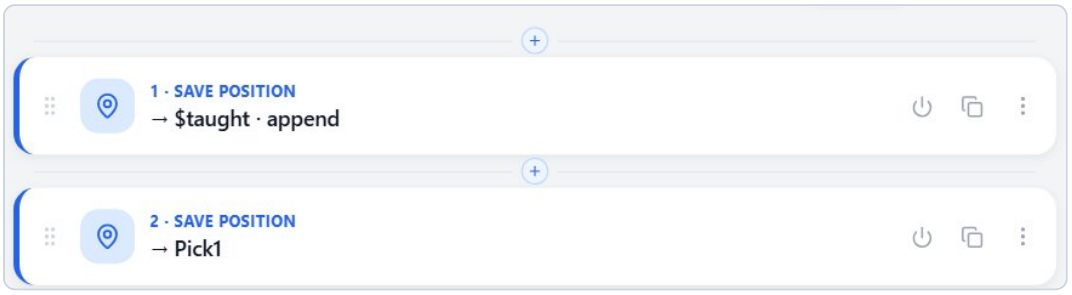
- Stores where the robot is ("Here"), expressed in the program's active local frame, so a later move under the same local lands on the same spot.
- Into a Points list: `mode=replace` (the list becomes exactly this pose), `append`, `insert` or `set` (at `index=`). A list that does not exist is created.
- `SavePosition point <name>` overwrites a saved point (or creates it); the change is recorded in the points history.
- `source=target` stores the last commanded target instead of the measured position.

ERRORS

Code	Kind	When
`indexOutOfRange`	catchable	`index=` is outside the list (catchable).
`badListOperation`	fatal	The variable is not a Points list (fatal).
`pointNotFound`	catchable	A point linked by id no longer exists.

EXAMPLE

BLOCKS



TEXT

```
var taught: list point
SavePosition $taught, mode=append
SavePosition point Pick1
```

See also: List ($, p. 93) · Move Linear (MoveL, p. 59)

Motion Settings

Change how later moves behave: speed, blending, tool and frame.

Step	Keyword	Description	Page
Set Speed (Linear)	`SetSpeedL`	Set the linear move speed, accel and decel.	73
Set Speed (Joint)	`SetSpeedJ`	Set the joint move speed, accel and decel.	74
Set Blend Radius	`SetBlendRadius`	Set the default corner radius for later blended moves.	75
Set Speed Factor	`SetSpeedFactor`	Scale this program's move speeds by a percentage.	75
Set Tool	`SetTool`	Change the active tool used by later moves.	76
Set Local	`SetLocal`	Activate a local coordinate frame for later moves.	77
Clear Local	`ClearLocal`	Return later moves to the world frame.	78

Set Speed (Linear)

SetSpeedL

Motion Settings Background programs: No

Set the linear move speed, accel and decel.

SYNTAX

```
SetSpeedL <speed>[, accel=…][, decel=…]
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Speed * fx	`position 1`	number or (expr)	mm/s	
Acceleration fx	`accel=`	number or (expr)	mm/s²	
Deceleration fx	`decel=`	number or (expr)	mm/s²	

DESCRIPTION AND NOTES

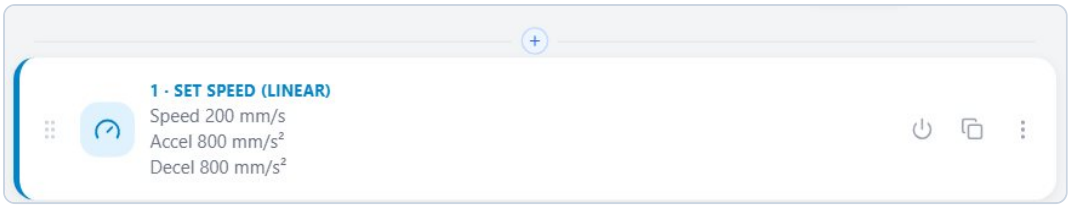
- Sets the default speed, acceleration and deceleration for later linear moves (Move Linear, Jump Linear legs, arcs). A move's own `speed=` overrides it for that move only.
- Not run in background programs.

ERRORS

Code	Kind	When
`unknownVariable`	fatal	An expression names a variable that does not exist (fatal).
`expressionSyntax`	fatal	An expression does not parse (fatal).

EXAMPLE

BLOCKS



TEXT

```
SetSpeedL 200, accel=800, decel=800
```

See also: Set Speed (Joint) (SetSpeedJ, p. 74) · Set Speed Factor (SetSpeedFactor, p. 75) · Move Linear (MoveL, p. 59)

Set Speed (Joint)

SetSpeedJ

Motion Settings Background programs: No

Set the joint move speed, accel and decel.

SYNTAX

```
SetSpeedJ <speed>[, accel=…][, decel=…]
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Speed * fx	`position 1`	number or (expr)	mm/s	
Acceleration fx	`accel=`	number or (expr)	mm/s²	
Deceleration fx	`decel=`	number or (expr)	mm/s²	

DESCRIPTION AND NOTES

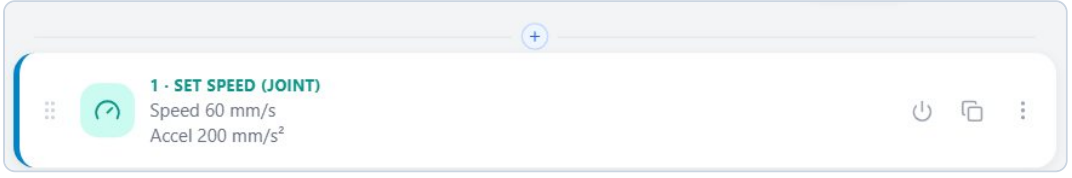
- Sets the default speed, acceleration and deceleration for later joint moves.
- Not run in background programs.

ERRORS

Code	Kind	When
`unknownVariable`	fatal	An expression names a variable that does not exist (fatal).
`expressionSyntax`	fatal	An expression does not parse (fatal).

EXAMPLE

BLOCKS



TEXT

```
SetSpeedJ 60, accel=200
```

See also: Set Speed (Linear) (SetSpeedL, p. 73) · Move Joint (MoveJ, p. 61)

Set Blend Radius

SetBlendRadius

Motion Settings Background programs: Yes

Set the default corner radius for later blended moves.

SYNTAX

```
SetBlendRadius <mm>
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Blend radius* fx	`position 1`	number or (expr)	mm	

DESCRIPTION AND NOTES

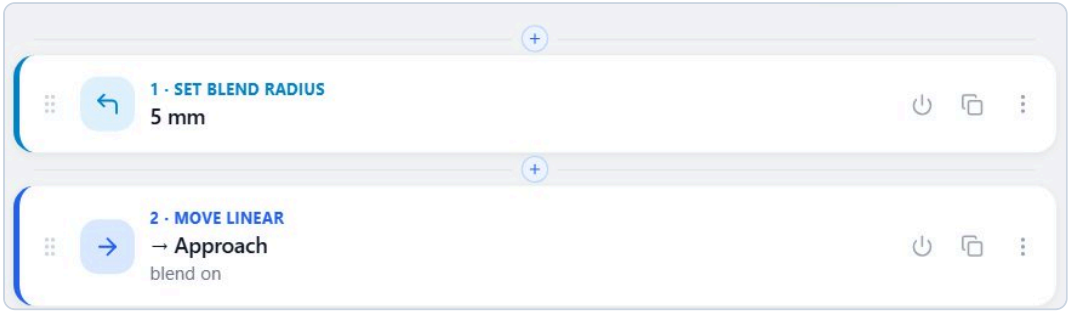
- Sets the default corner radius used by later moves that blend without giving their own radius.

ERRORS

Code	Kind	When
`unknownVariable`	fatal	An expression names a variable that does not exist (fatal).
`expressionSyntax`	fatal	An expression does not parse (fatal).

EXAMPLE

BLOCKS



TEXT

```
SetBlendRadius 5
MoveL Approach, blend=on
```

See also: Move Linear (MoveL, p. 59)

Set Speed Factor

SetSpeedFactor

Motion Settings Background programs: No

Scale this program's move speeds by a percentage.

SYNTAX

```
SetSpeedFactor <percent>
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Speed factor* fx Never exceeds the operator's speed override; resets to 100 when the program ends.	`position 1`	number or (expr) range 1 – 100	%	`100`

DESCRIPTION AND NOTES

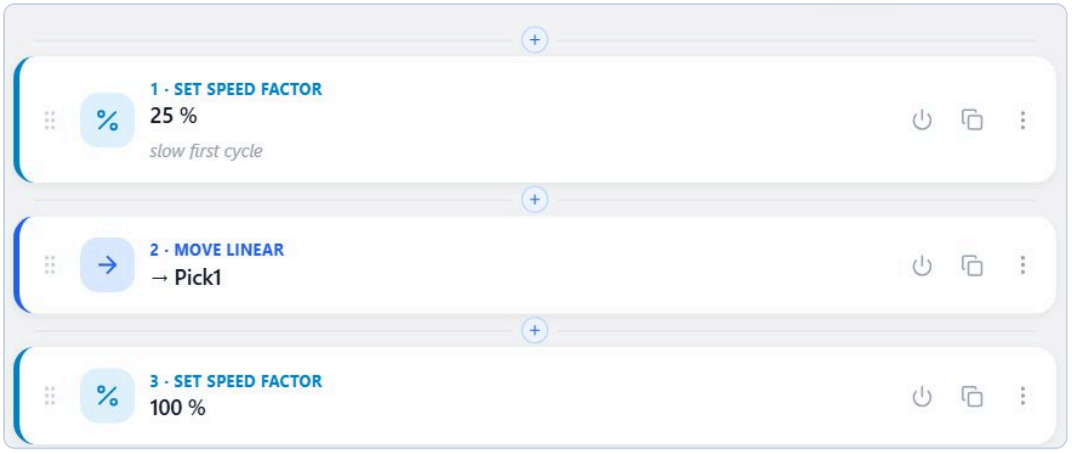
- Every later move of this program runs at *operator override* × *factor* / *100* — single moves, blended runs, jump legs, arcs and a move resumed after a stop.
- It can only slow a program down: values are clamped to 1–100 %. Manual moves and jogging are never affected.
- Readable as `$program.speedFactor`; back to 100 when the run ends.
- Not run in background programs.

ERRORS

Code	Kind	When
`unknownVariable`	fatal	An expression names a variable that does not exist (fatal).
`expressionSyntax`	fatal	An expression does not parse (fatal).

EXAMPLE

BLOCKS



TEXT

```
SetSpeedFactor 25     # slow first cycle
MoveL Pick1
SetSpeedFactor 100
```

See also: Set Speed (Linear) (SetSpeedL, p. 73) · Set Speed (Joint) (SetSpeedJ, p. 74)

Set Tool

`SetTool`

Motion Settings Background programs: No

Change the active tool used by later moves.

SYNTAX

```
SetTool <tool>
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Tool None = no tool offset.	`position 1`	tool name		

DESCRIPTION AND NOTES

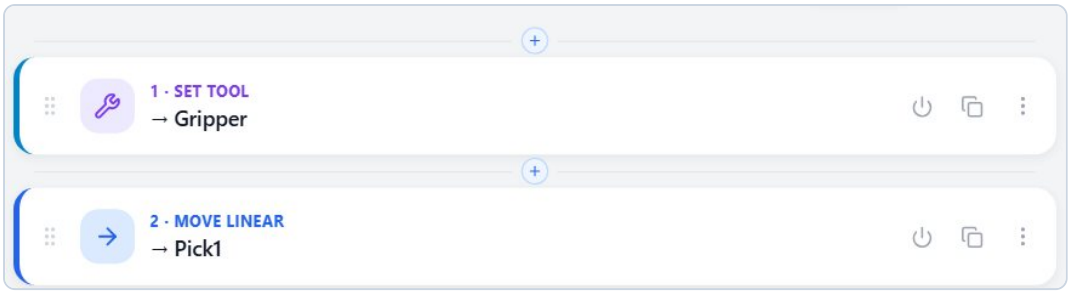
- Makes a tool (TCP offset) active for later moves; points are reached with the tool's tip. None = no tool offset.
- Not run in background programs.

ERRORS

This step raises no runtime error codes of its own.

EXAMPLE

BLOCKS



TEXT

```
SetTool Gripper
MoveL Pick1
```

See also: Set Local (SetLocal, p. 77) · Move Linear (MoveL, p. 59)

Set Local

`SetLocal`

Motion Settings Background programs: No

Activate a local coordinate frame for later moves.

SYNTAX

```
SetLocal <local>
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Local frame*	`position 1`	local name		

DESCRIPTION AND NOTES

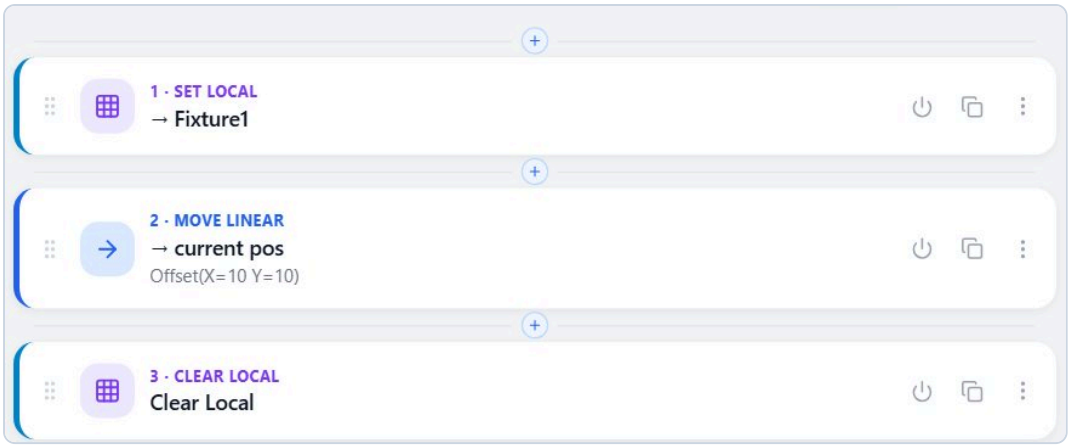
- Activates a local coordinate frame: later targets and offsets are interpreted in it.
- A single move can use another frame with `local=`.
- Not run in background programs.

ERRORS

This step raises no runtime error codes of its own.

EXAMPLE

BLOCKS



TEXT

```
SetLocal Fixture1
MoveL here, offset=(x=10, y=10)
ClearLocal
```

See also: Clear Local (ClearLocal, p. 78) · Set Tool (SetTool, p. 76)

Clear Local

`ClearLocal`

Motion Settings Background programs: No

Return later moves to the world frame.

SYNTAX

```
ClearLocal
```

PARAMETERS

No parameters.

DESCRIPTION AND NOTES

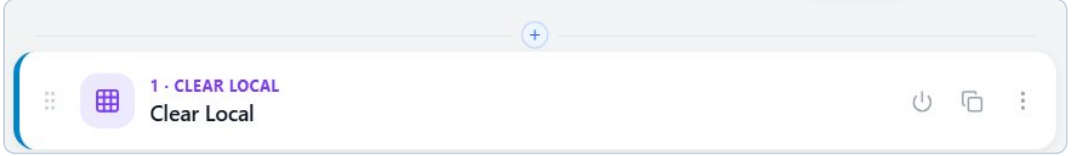
- Returns later moves to the world (base) frame.
- Not run in background programs.

ERRORS

This step raises no runtime error codes of its own.

EXAMPLE

BLOCKS



TEXT

```
ClearLocal
```

See also: Set Local (SetLocal, p. 77)

Logic & Flow

Decide what runs next — branches, loops, jumps, errors and pauses.

Step	Keyword	Description	Page
If Condition	`if`	Run steps only when a condition holds, with else-if and else branches.	79
Switch	`switch`	Run the case whose value equals an expression.	80
Loop	`repeat`	Repeat steps a number of times, for each list element, or while a condition holds.	81
Break / Next	`break`	Leave the enclosing loop, or skip to its next iteration.	83
Try	`try`	Run steps; on a runtime error retry them or run the catch steps.	83
Raise Error	`raise`	Fail with a message — caught by an enclosing Try, otherwise ends the program.	85
Call Routine	`call`	Run a routine, then continue.	86
Return	`return`	Leave the routine (or end the program).	87
End Program	`exit`	Finish the program now, from anywhere.	87
Label	`label`	Mark a place Go To Label can jump to.	88
Go To Label	`goto`	Jump to a Label.	89
Pause Program	`pause`	Pause — the operator continues or stops from the monitor.	90
Note	`#`	A note or section header — not executed.	91

If Condition

`if`

Logic & Flow Background programs: Yes

Run steps only when a condition holds, with else-if and else branches.

SYNTAX

```
if <condition> … [elif <condition> …] [else …] end
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Condition * fx	`position 1`	condition		

Contains steps: Then, Else if, Else.

DESCRIPTION AND NOTES

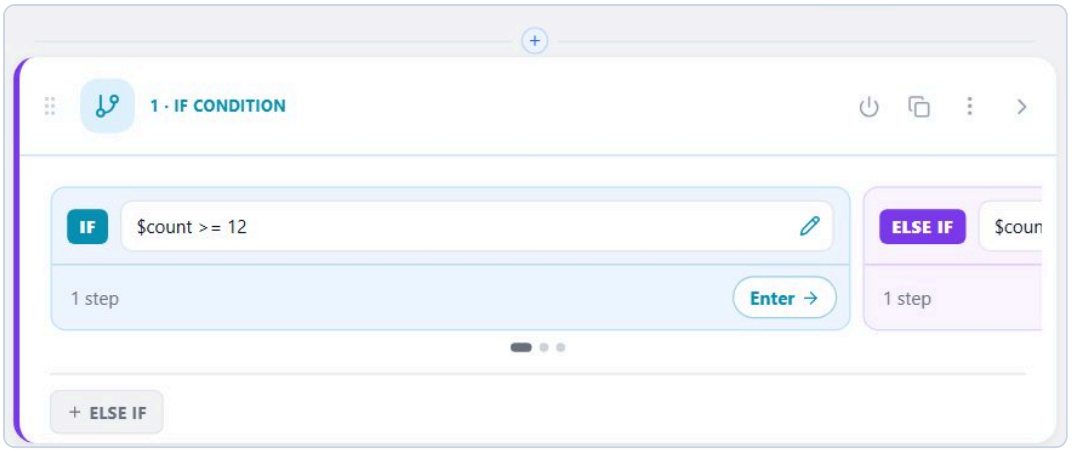
- Branches are tested top to bottom; the first whose condition holds runs, else the `else` branch.
- A condition is an ordinary expression; non-zero is true. Its top-level `and` / `or` become the separate condition rows the Blocks editor shows.

ERRORS

Code	Kind	When
`unknownVariable`	fatal	An expression names a variable that does not exist (fatal).
`expressionSyntax`	fatal	An expression does not parse (fatal).

EXAMPLE

BLOCKS



TEXT

```
var count = 0
if $count >= 12
  status "Tray full"
elif $count > 8
  status warning "Almost full"
else
  $count = $count + 1
end
```

See also: Switch (switch, p. 80) · Loop (repeat, p. 81)

Switch

`switch`

Logic & Flow Background programs: Yes

Run the case whose value equals an expression.

SYNTAX

```
switch <expr> case <v>[, <v>…] … [default …] end
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Value* fx Evaluated once; compared with each case value.	`position 1`	expression		

Contains steps: Cases, Default.

DESCRIPTION AND NOTES

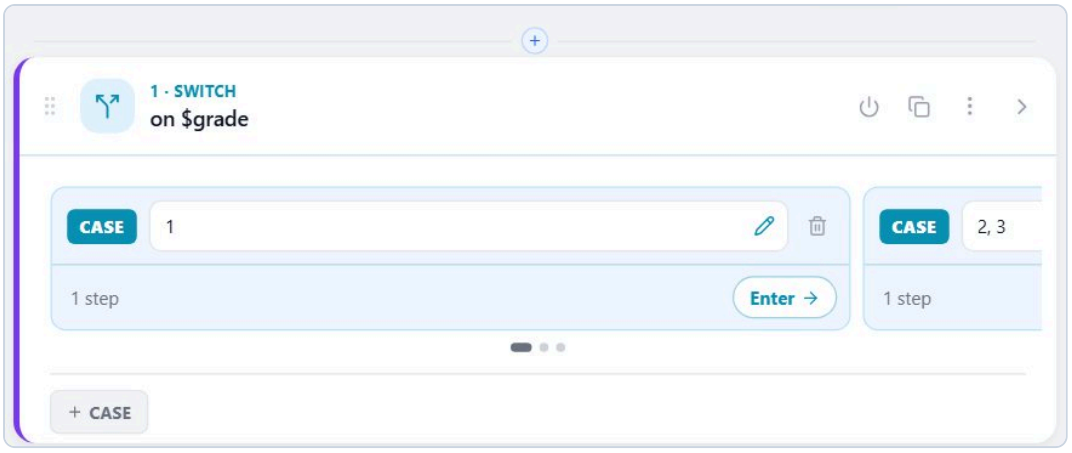
- The value is evaluated once; the first case with an equal value (within 1e-9) runs, otherwise `default`. There is no fall-through.
- Numbers only — test text with `if $s.contains(…)`.

ERRORS

Code	Kind	When
`unknownVariable`	fatal	An expression names a variable that does not exist (fatal).
`expressionSyntax`	fatal	An expression does not parse (fatal).

EXAMPLE

BLOCKS



TEXT

```
var grade = 1
switch $grade
case 1
  MoveL Place1
case 2, 3
  MoveL stack Bins[0]
default
  raise "Unknown grade {$grade}"
end
```

See also: If Condition (if, p. 79)

Loop

repeat

Logic & Flow Background programs: Yes Also: for, while

Repeat steps a number of times, for each list element, or while a condition holds.

SYNTAX

```
repeat <n>|forever [as $i] … end | for $v[, $i] in $list … end | while <condition> … end
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Repeat	`mode=`	count | forEach | while `count` A number of times · `forEach` For each element · `while` While a condition holds		`count`
Count fx 0 = forever. when loopMode = count	`position` `1`	number or (expr)		`1`
List when loopMode = forEach	`in=`	$list		
Value variable Number/Boolean lists: the element. Other lists: the index. when loopMode = forEach	`value=`	$variable		
Index variable when loopMode = count / forEach	`as=`	$variable		

Parameter	In text	Type / values	Unit	Default
While fx when loopMode = while	`while=`	condition		

Contains steps: Body.

DESCRIPTION AND NOTES

- `repeat n` runs the body n times (`as $i` counts from 0); `repeat forever` (count 0) runs until stopped or left with `break` .
- `for $v, $i in $list` walks a list. Number and Boolean lists give the element in `$v` ; Point and Record lists give the index, so read the element as `$list[$i]` . Write `for _, $i in $list` when you only want the index.
- `while <condition>` re-checks the condition before every pass.

ERRORS

Code	Kind	When
`unknownVariable`	fatal	An expression names a variable that does not exist (fatal).
`expressionSyntax`	fatal	An expression does not parse (fatal).

EXAMPLE

BLOCKS



TEXT

```
var vals: list number = (1, 2, 3)
var sum = 0
repeat 3 as $i
  MoveL grid Tray[$i], offset=(z=10)
end
for $v, $i in $vals
  $sum = $sum + $v
end
while $stb.in1 == 0
  Wait 10ms
end
```

See also: Break / Next (break, p. 83) · If Condition (if, p. 79)

Break / Next

break

Logic & Flow Background programs: Yes Also: continue

Leave the enclosing loop, or skip to its next iteration.

SYNTAX

```
break | continue
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Action	position 1	break | continue break Break loop · continue Next iteration		break

DESCRIPTION AND NOTES

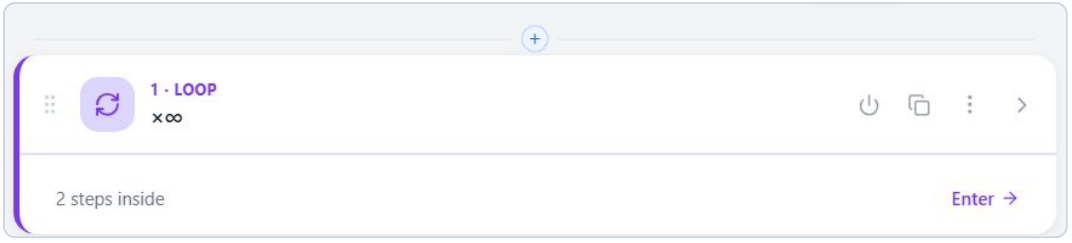
- break leaves the nearest enclosing loop; continue ends this pass (a count loop counts it, a for-each moves to the next element, a while re-checks its condition).
- Branches and Try bodies between the step and the loop are simply left. It never crosses a routine boundary.

ERRORS

This step raises no runtime error codes of its own.

EXAMPLE

BLOCKS



TEXT

```
repeat forever
  if $stb.in2
    break
  end
  Wait 50ms
end
```

See also: Loop (repeat, p. 81)

Try

try

Logic & Flow Background programs: Yes

Run steps; on a runtime error retry them or run the catch steps.

SYNTAX

```
try [retry=…][, delay=…] … [catch [$msg][, code=$code][, only=(…)] …] end
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Retries	retry=	number		0

Parameter	In text	Type / values	Unit	Default
Retry delay	`delay=`	duration (500ms, 2s)	ms	`0`
Catch only Empty = every catchable error. A Raise Error code may be typed in.	`only=`	(a, b, ...) `waitTimeout` Wait timed out · `visionTimeout` Vision timed out · `visionFailed` Vision failed · `httpFailed` HTTP request failed · `httpTimeout` HTTP request timed out · `receiveTimeout` HTTP receive timed out · `pluginFailed` Plugin step failed · `pluginTimeout` Plugin step timed out · `pluginNotRunning` Plugin not running · `promptTimeout` Prompt timed out · `pointNotFound` Point not found · `arcDegenerate` Arc points are in a line · `indexOutOfRange` List index out of range · `userError` Raised error		
Message variable	`message=`	$variable		
Code variable	`code=`	$variable		

Contains steps: Try, Catch.

DESCRIPTION AND NOTES

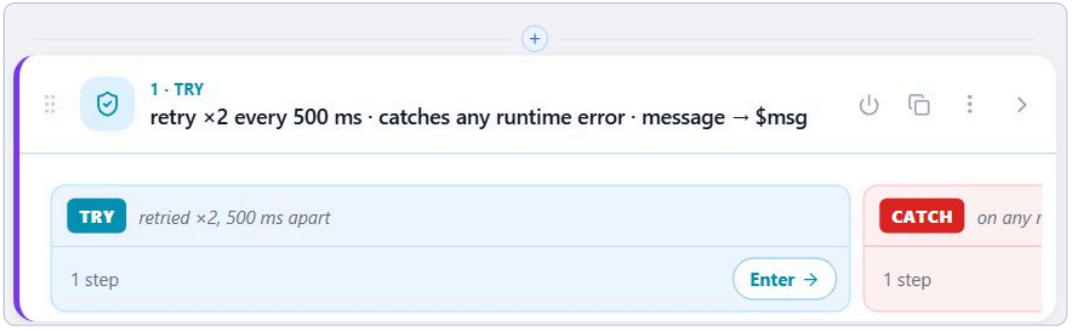
- When a step inside the body fails with a catchable code — at any depth, including called routines — the body is abandoned; while retries are left it runs again from the top after `delay=`; then the message and code go into the catch variables and the catch steps run.
- An empty catch ignores the error. The program continues after the Try either way.
- `only=(…)` limits which codes are caught; others go to the next enclosing Try. A failure inside the catch also goes outward.
- Fatal codes (motion faults, e-stop, user stop, unknown variables, syntax errors) are never caught.

ERRORS

This step raises no runtime error codes of its own.

EXAMPLE

BLOCKS



TEXT

```
var msg: string
var code: string
try retry=2, delay=500ms
  Wait until $stb.in1, timeout=2s, onTimeout=error
catch $msg, code=$code
  status warning "Gave up: {$msg}"
end
```

See also: Raise Error (raise, p. 85) · Wait (Wait, p. 96)

Raise Error

raise

Logic & Flow Background programs: Yes

Fail with a message — caught by an enclosing Try, otherwise ends the program.

SYNTAX

```
raise "<message>"[, code=<identifier>]
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Message*	`position 1`	"text {$var}"		
Code An identifier a Try can filter on.	`code=`	text		`userError`

DESCRIPTION AND NOTES

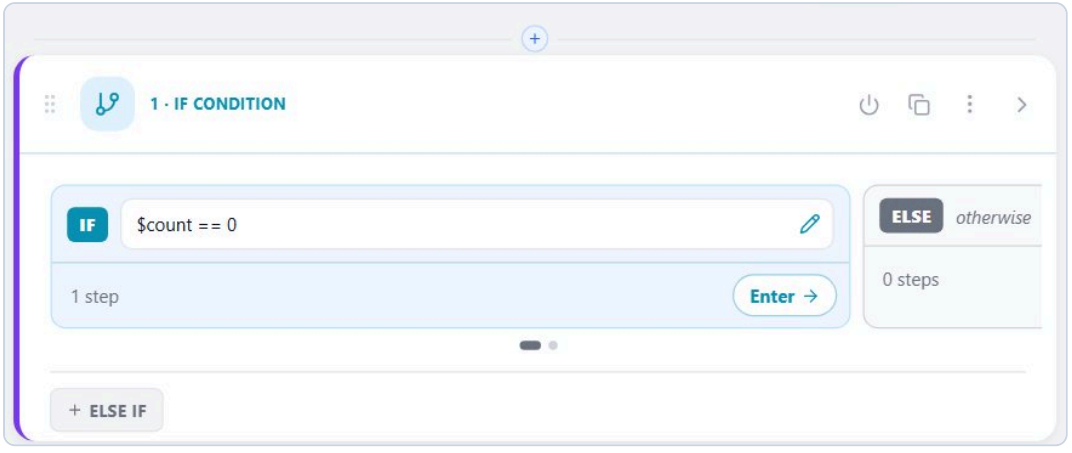
- Fails with your message and code (default `userError`). An enclosing Try that catches the code takes over; otherwise the program ends with Error.
- A code of your own is always catchable.

ERRORS

Code	Kind	When
`userError`	catchable	The default code — or any identifier you give with `code=`.

EXAMPLE

BLOCKS



TEXT

```
var count = 0
if $count == 0
  raise "No parts found", code=noParts
end
```

See also: Try (try, p. 83) · End Program (exit, p. 87)

Call Routine

`call`

Logic & Flow Background programs: Yes

Run a routine, then continue.

SYNTAX

```
call <routine>[(param=…, …)][, out param=$var]
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Routine *	`position 1`	routine name		
Arguments One expression per in / in-out parameter (a template for a text parameter), evaluated before the call.	`rows`	exprMap		
Outputs Out / in-out parameters copied into your variables when the routine returns.	`out=`	mappingOut		

DESCRIPTION AND NOTES

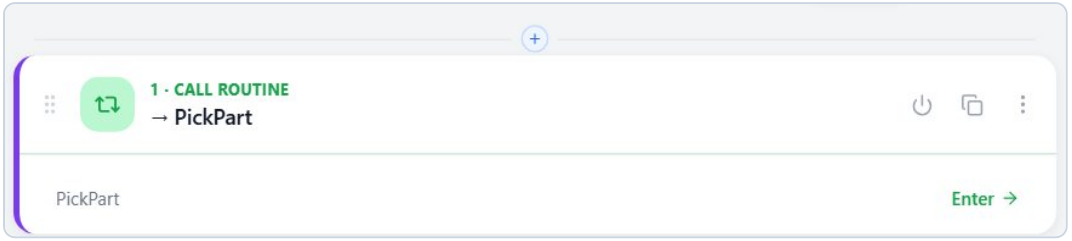
- Runs a routine program, then continues with the next step.
- `in` / `inout` arguments are evaluated in the caller's scope before the routine starts; when it ends (last step or Return), mapped `out` / `inout` parameters are copied into the caller's variables.
- A routine is a boundary: Go To Label and Break / Next inside it never reach the caller.

ERRORS

Code	Kind	When
`unknownVariable`	fatal	An expression names a variable that does not exist (fatal).
`expressionSyntax`	fatal	An expression does not parse (fatal).

EXAMPLE

BLOCKS



TEXT

```
var ok: bool = off
call PickPart(slot=3), out picked=$ok
```

See also: Return (return, p. 87)

Return

`return`

Logic & Flow Background programs: Yes

Leave the routine (or end the program).

SYNTAX

```
return
```

PARAMETERS

No parameters.

DESCRIPTION AND NOTES

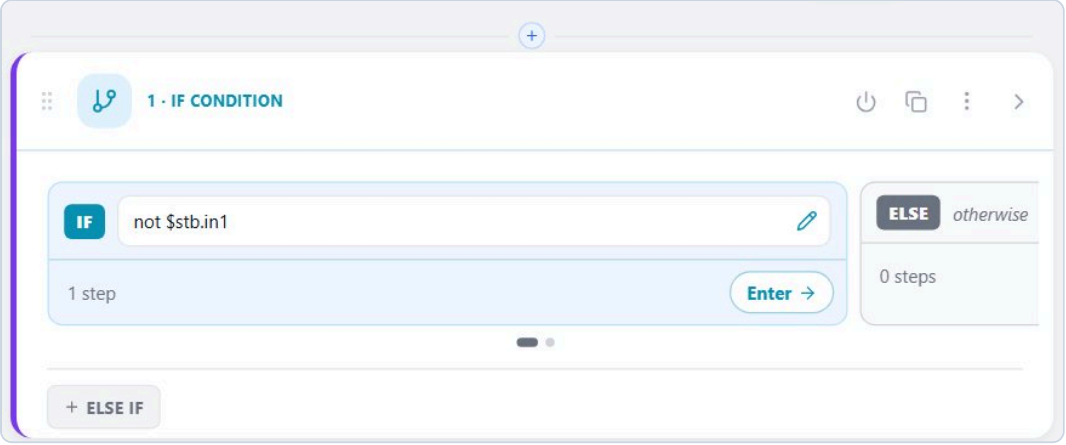
- In a routine: leave it now (out parameters are copied back). In a main or background program: finish with Complete.

ERRORS

This step raises no runtime error codes of its own.

EXAMPLE

BLOCKS



TEXT

```
if not $stb.in1
  return
end
```

See also: Call Routine (call, p. 86) · End Program (exit, p. 87)

End Program

`exit`

Logic & Flow Background programs: Yes

Finish the program now, from anywhere.

SYNTAX

```
exit ["<message>"]
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Message	`position 1`	"text {$var}"		

DESCRIPTION AND NOTES

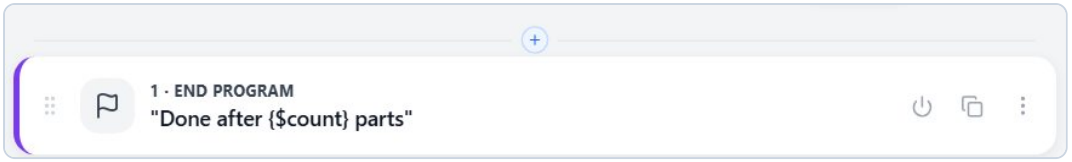
- Finishes the program with Complete from any depth (loops, routines, Try), with the message as the final status text. A background program ends only itself.

ERRORS

This step raises no runtime error codes of its own.

EXAMPLE

BLOCKS



TEXT

```
var count = 0
exit "Done after {$count} parts"
```

See also: Return (return, p. 87) · Raise Error (raise, p. 85)

Label

`label`

Logic & Flow Background programs: Yes

Mark a place Go To Label can jump to.

SYNTAX

```
label <name>
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Name*	`position 1`	text		

DESCRIPTION AND NOTES

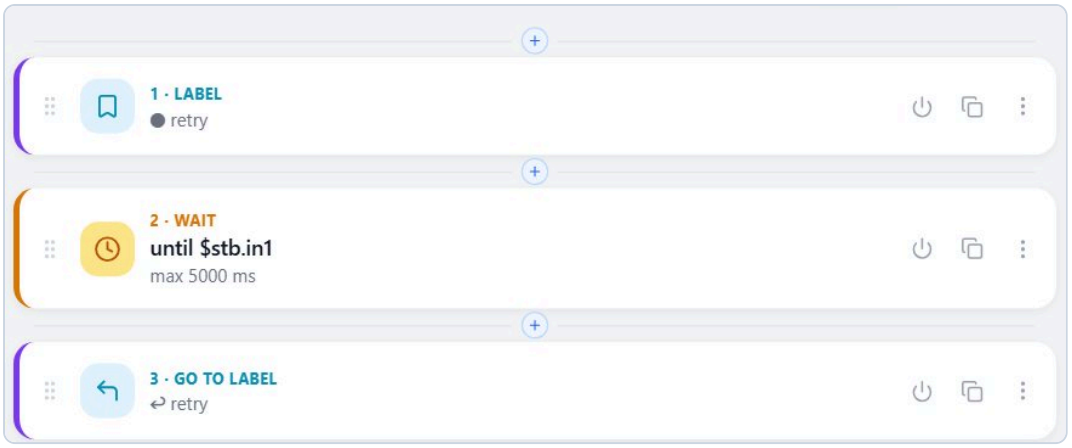
- Marks a place Go To Label can jump to. The name must be unique in the program.

ERRORS

This step raises no runtime error codes of its own.

EXAMPLE

BLOCKS



TEXT

```
label retry
Wait until $stb.in1, timeout=5s
goto retry
```

See also: Go To Label (goto, p. 89)

Go To Label

goto

Logic & Flow Background programs: Yes

Jump to a Label.

SYNTAX

```
goto <label>
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Label *	`position 1`	label name		

DESCRIPTION AND NOTES

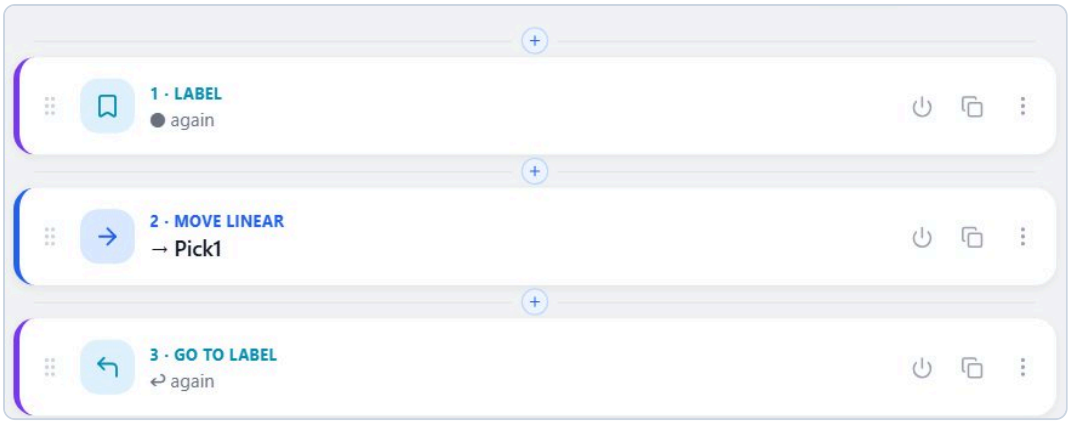
- Jumps to a label in the same program (or the same routine). Prefer loops where they fit — they read better and keep the structure visible.

ERRORS

This step raises no runtime error codes of its own.

EXAMPLE

BLOCKS



TEXT

```
label again
MoveL Pick1
goto again
```

See also: Label (label, p. 88) · Loop (repeat, p. 81)

Pause Program

pause

Logic & Flow Background programs: Yes

Pause — the operator continues or stops from the monitor.

SYNTAX

```
pause
```

PARAMETERS

No parameters.

DESCRIPTION AND NOTES

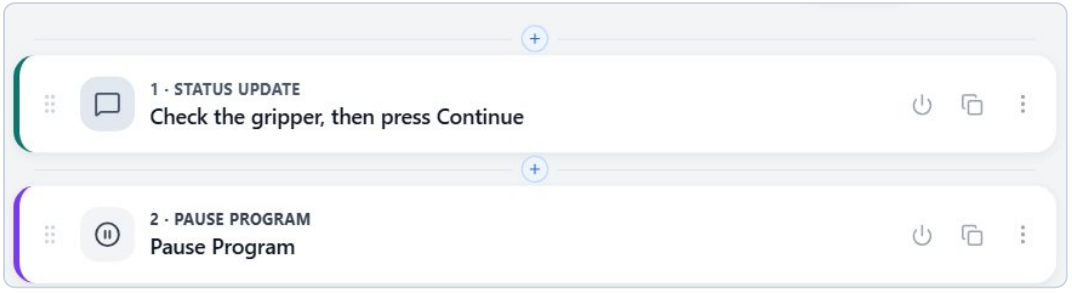
- Pauses the program (status Stopped) with its place kept; the operator continues or stops it from the monitor.

ERRORS

This step raises no runtime error codes of its own.

EXAMPLE

BLOCKS



TEXT

```
status "Check the gripper, then press Continue"
pause
```

See also: Prompt (prompt, p. 120)

Note

/

Logic & Flow Background programs: Yes

A note or section header — not executed.

SYNTAX

```
# text | ## Section
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Style	`rows`	note | section `note` Note · `section` Section header		`note`
Text	`position 1`	text		

DESCRIPTION AND NOTES

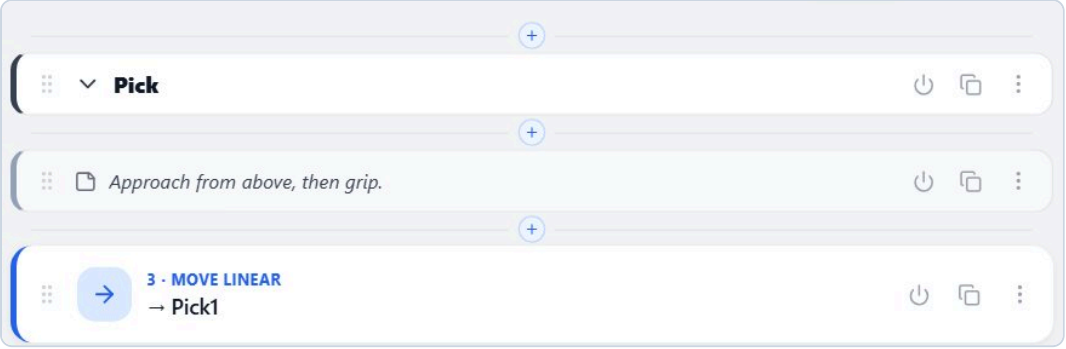
- Not executed and not counted as a step. `## text` is a section header the editor can collapse; a comment paragraph followed by a blank line is a note.

ERRORS

This step raises no runtime error codes of its own.

EXAMPLE

BLOCKS



TEXT

```
## Pick
# Approach from above, then grip.

MoveL Pick1
```

Variables & Timing

Store values, edit lists, count time, and wait before continuing.

Step	Keyword	Description	Page
Set Variable	`$`	Assign a value or expression to a variable.	92
List	`$`	Append, insert, remove, set, sort or copy list elements.	93
Stopwatch	`stopwatch`	Start, stop or reset a stopwatch variable.	95
Wait	`Wait`	Wait for a time, or until a condition holds.	96

Set Variable

`$var = …`

Variables & Timing Background programs: **Yes**

Assign a value or expression to a variable.

SYNTAX

```
$<name> = <expression | "template">
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Variable *	`position 1`	$variable		
Value * fx An expression; a template for a text variable.	`position 2`	expression		

DESCRIPTION AND NOTES

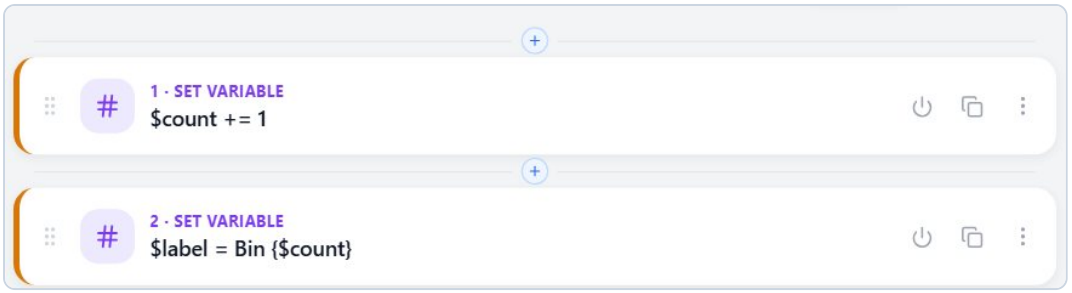
- Evaluates the expression and stores it. A text variable takes a template in quotes: `$label = "Bin {$i}"`.
- Comparisons yield 1 / 0, so they can be stored in a boolean directly.
- Properties (`$robot.x` …), IO names and computed variables cannot be assigned.

ERRORS

Code	Kind	When
`computedVariable`	fatal	The target is a computed variable (fatal).
`unknownVariable`	fatal	An expression names a variable that does not exist (fatal).
`expressionSyntax`	fatal	An expression does not parse (fatal).

EXAMPLE

BLOCKS



TEXT

```
var count = 0
var label: string
$count = $count + 1
$label = "Bin {$count}"
```

See also: List ($, p. 93)

List

`$list.…(…)`

Variables & Timing Background programs: Yes

Append, insert, remove, set, sort or copy list elements.

SYNTAX

```
$<list>.append(…) | .insert(i, …) | .removeAt(i) | .clear() | [i] = … | [i].field = … | .sort(by=…, desc) | .reverse() | .copyFrom($other)
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
List *	`position 1`	$list		
Action *	`action=`	append | insert | removeAt | clear | set | setField | sort | reverse | copyFrom `append` Append · `insert` Insert · `removeAt` Remove at · `clear` Clear · `set` Set element · `setField` Set field · `sort` Sort · `reverse` Reverse · `copyFrom` Copy from		`append`
Index fx when listAction = insert / removeAt / set / setField	`index=`	number or (expr)		
Value fx For Number and Boolean lists. when listAction = append / insert / set	`value=`	expression		
Fields For Point and Record lists. Append with no fields on a Points list appends the current position. when listAction = append / insert / set / setField	`fields=`	exprMap		
Copy from when listAction = copyFrom	`from=`	$list		

Parameter	In text	Type / values	Unit	Default
Sort by field Empty = the element value. when listAction = sort	`by=`	text		
Descending when listAction = sort	`desc=`	on / off		`off`

DESCRIPTION AND NOTES

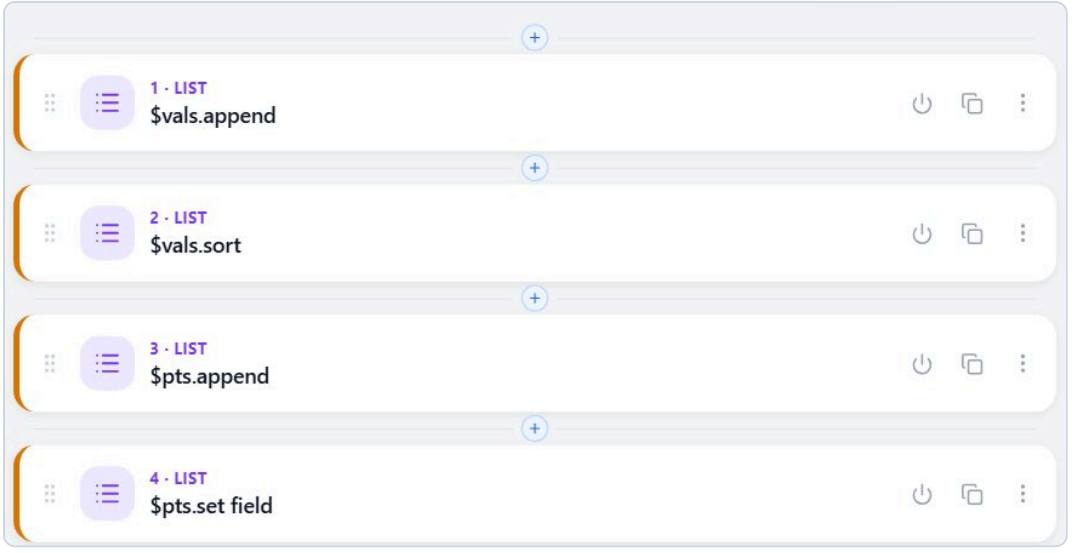
- Number / Boolean lists take a value; Point / Record lists take fields (`x=…, y=…`). `$pts.append()` with no fields appends the current position.
- Everything is evaluated before the list changes, so a failing step leaves the list as it was. The declared list is never changed — a re-run starts from it.

ERRORS

Code	Kind	When
`indexOutOfRange`	catchable	The index is outside the list (catchable).
`badListOperation`	fatal	The step does not fit the list: fields on a Number list, a value for a Points list, an incompatible copy (fatal).
`unknownVariable`	fatal	An expression names a variable that does not exist (fatal).
`expressionSyntax`	fatal	An expression does not parse (fatal).

EXAMPLE

BLOCKS



TEXT

```
var vals: list number
var pts: list point
$vals.append(5)
$vals.sort(by=value, desc)
$pts.append(x=$robot.x, y=$robot.y, z=50)
$pts[0].z = 30
```

See also: Set Variable ($, p. 92) · Save Position (SavePosition, p. 71) · Loop (repeat, p. 81)

Stopwatch

stopwatch

Variables & Timing Background programs: Yes

Start, stop or reset a stopwatch variable.

SYNTAX

```
stopwatch start|stop|reset $<name>
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Action*	`position 1`	start | stop | reset `Start` Start · `Stop` Stop · `Reset` Reset		`Start`
Stopwatch*	`position 2`	$variable		

DESCRIPTION AND NOTES

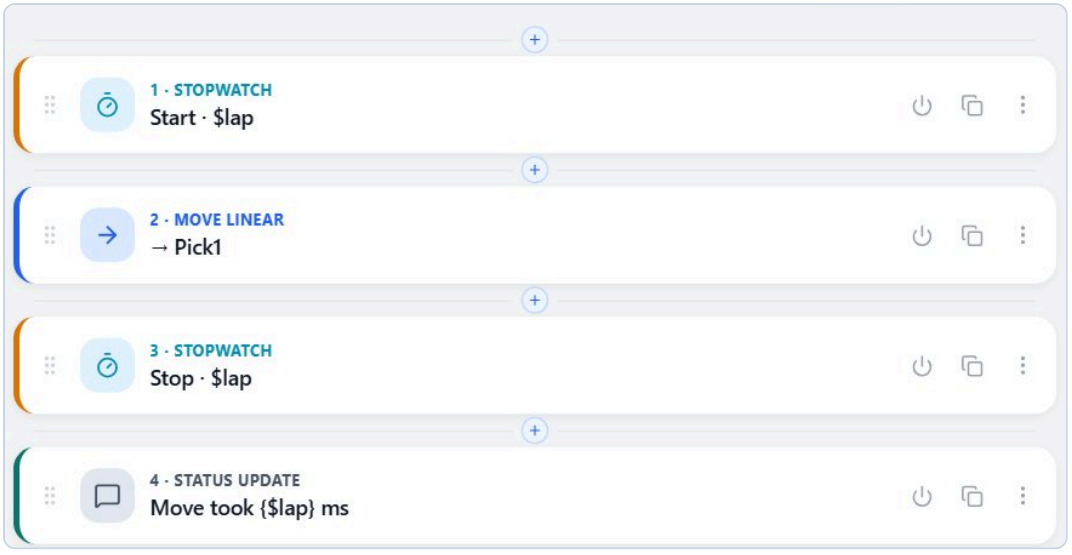
- A stopwatch variable holds elapsed milliseconds while it runs; `stop` freezes it, `reset` sets it to 0.

ERRORS

This step raises no runtime error codes of its own.

EXAMPLE

BLOCKS



TEXT

```
var lap: stopwatch
stopwatch start $lap
MoveL Pick1
stopwatch stop $lap
status "Move took {$lap} ms"
```

See also: Wait (Wait, p. 96)

Wait

Wait

Variables & Timing Background programs: Yes

Wait for a time, or until a condition holds.

SYNTAX

```
Wait <ms> | Wait until <condition>[, timeout=…][, timedOut=$v][, onTimeout=continue|error]
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Wait for	`mode=`	duration | condition `duration` A duration · `condition` A condition		`duration`
Duration fx when waitMode = duration	`position 1`	duration (500ms, 2s)	ms	
Until fx when waitMode = condition	`until=`	condition		
Timeout Empty or 0 = wait forever. when waitMode = condition	`timeout=`	duration (500ms, 2s)	ms	
Timed-out flag when waitMode = condition	`timedOut=`	$variable		
On timeout when waitMode = condition	`onTimeout=`	continue | error `continue` Continue · `error` Fail (catchable)		`continue`

DESCRIPTION AND NOTES

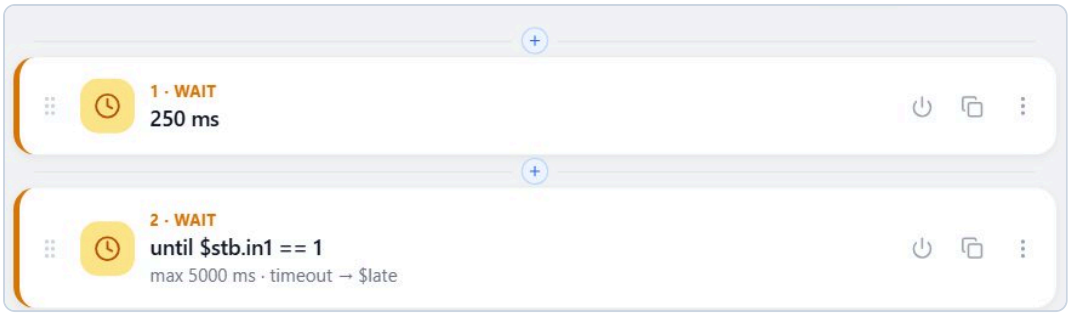
- `Wait 500ms` pauses for a time. `Wait until <condition>` holds until the condition is true, checked every tick.
- With `timeout=`, `timedOut=$v` is set to 1 when time ran out. `onTimeout=continue` (default) carries on; `onTimeout=error` fails with `waitTimeout`, which a Try can catch.
- A condition that holds on the tick the timeout falls due counts as met.

ERRORS

Code	Kind	When
`waitTimeout`	catchable	Condition wait timed out with `onTimeout=error` (catchable).
`unknownVariable`	fatal	An expression names a variable that does not exist (fatal).
`expressionSyntax`	fatal	An expression does not parse (fatal).

EXAMPLE

BLOCKS



TEXT

```
var late: bool = off
Wait 250ms
Wait until $stb.in1 == 1, timeout=5s, timedOut=$late
```

See also: Try (try, p. 83) · Set Output (SetOutput, p. 98)

I/O & Aux Axes

Drive digital outputs, load cells and auxiliary stepper axes.

Step	Keyword	Description	Page
Set Output	`SetOutput`	Turn a digital output on or off, toggle it or pulse it.	98
Tare Load Cell	`Tare`	Zero a load cell.	99
Read Load Cell	`ReadLoadCell`	Read a load cell into a variable.	100
Start Recording	`StartRecording`	Record a load cell into a list variable.	101
Stop Recording	`StopRecording`	Stop a recording and store its samples.	102
Export Data	`ExportData`	Write variables to a CSV or JSON file.	102
Aux Move	`AuxMove`	Move an aux axis by a distance or a number of steps.	103
Aux Continuous Run	`AuxContinuous`	Run an aux axis continuously until an Aux Stop.	104
Aux Stop	`AuxStop`	Stop aux axis motion — ramped or immediate.	105
Aux Motor Enable	`AuxEnable`	Enable or disable the motor drivers of an aux card.	106
Gripper	`GripperAction`	Open, close or grip to a jaw gap.	107
Conveyor	`ConveyorAction`	Run a conveyor forward or reverse, or stop it.	108
Rotary Index	`RotaryIndex`	Index a rotary axis a number of positions.	109

Set Output

`SetOutput`

I/O & Aux Axes Background programs: Yes

Turn a digital output on or off, toggle it or pulse it.

SYNTAX

```
SetOutput <stb|relay|io> [<device>] <number> <on|off|toggle|(expr)>[, pulse=…][, blocking=on]
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Card *	`position 1`	stb | relay | io `stb` Driver board · `relay` USB relay · `io` IO device		`stb`
Device when outputCard = io	`position 2`	device name		
Output *	`position 3`	number		
Action	`action=`	set | toggle `set` Set · `toggle` Toggle		`set`
On fx An expression: non-zero = on. when outputAction = set	`position 4`	on / off or (expr)		`on`
Pulse Set for this long, then flip back. Empty = hold.	`pulse=`	duration (500ms, 2s)	ms	
Wait for pulse	`blocking=`	on / off		`off`

DESCRIPTION AND NOTES

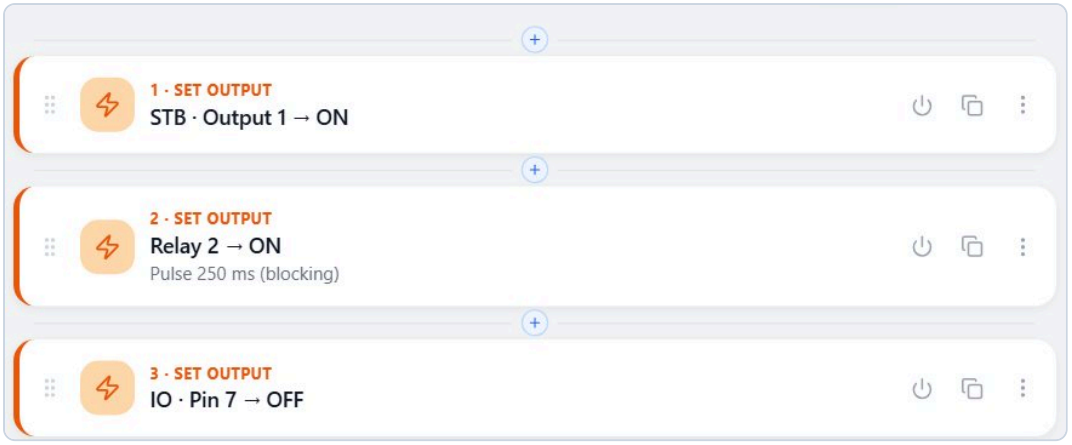
- Cards: `stb` (driver board outputs), `relay` (USB relay channels), `io` (an IO device such as an Arduino Nano — name the device).
- The value may be `on`, `off`, `toggle` or an expression in parentheses (non-zero = on).
- `pulse=` sets the output for that long, then flips it back. `blocking=on` waits for the pulse to finish; otherwise the program moves on and the flip-back happens in the background.

ERRORS

Code	Kind	When
`unknownVariable`	fatal	An expression names a variable that does not exist (fatal).
`expressionSyntax`	fatal	An expression does not parse (fatal).

EXAMPLE

BLOCKS



TEXT

```
SetOutput stb 1 on
SetOutput relay 2 on, pulse=250ms, blocking=on
SetOutput io Arduino_Nano 7 toggle
```

See also: Wait (Wait, p. 96)

Tare Load Cell

Tare

I/O & Aux Axes Background programs: Yes

Zero a load cell.

SYNTAX

```
Tare device=…, pin=…
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Device	`device=`	device name		
Pin	`pin=`	number		

DESCRIPTION AND NOTES

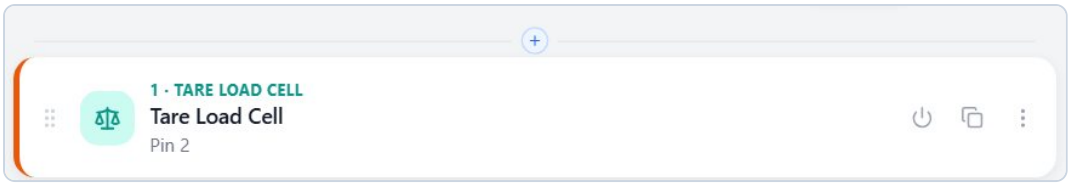
- Zeroes a load cell: its current raw reading becomes the tare offset.

ERRORS

This step raises no runtime error codes of its own.

EXAMPLE

BLOCKS



TEXT

```
Tare device=Arduino_Nano, pin=2
```

See also: Read Load Cell (ReadLoadCell, p. 100)

Read Load Cell

`ReadLoadCell`

I/O & Aux Axes Background programs: **Yes**

Read a load cell into a variable.

SYNTAX

```
ReadLoadCell $<var>, device=…, pin=…[, unit=g]
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Variable *	`position 1`	$variable		
Device	`device=`	device name		
Pin	`pin=`	number		
Unit	`unit=`	g | kg | N | lb | oz `g` g · `kg` kg · `N` N · `lb` lb · `oz` oz		`g`

DESCRIPTION AND NOTES

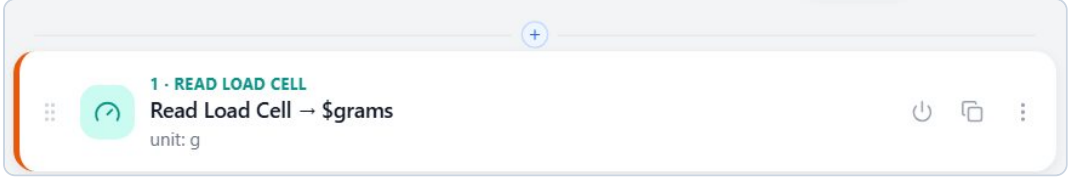
- Reads the calibrated weight into a variable in the chosen unit (grams by default). In expressions a load cell can also be read live as `$io.<device>.<pin name>` (grams).

ERRORS

This step raises no runtime error codes of its own.

EXAMPLE

BLOCKS



TEXT

```
var grams = 0
ReadLoadCell $grams, device=Arduino_Nano, pin=2, unit=g
```

See also: Tare Load Cell (Tare, p. 99) · Start Recording (StartRecording, p. 101)

Start Recording

StartRecording

I/O & Aux Axes Background programs: Yes

Record a load cell into a list variable.

SYNTAX

```
StartRecording $<list>, device=…, pin=…
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Into list*	`position 1`	$list		
Device	`device=`	device name		
Pin	`pin=`	number		

DESCRIPTION AND NOTES

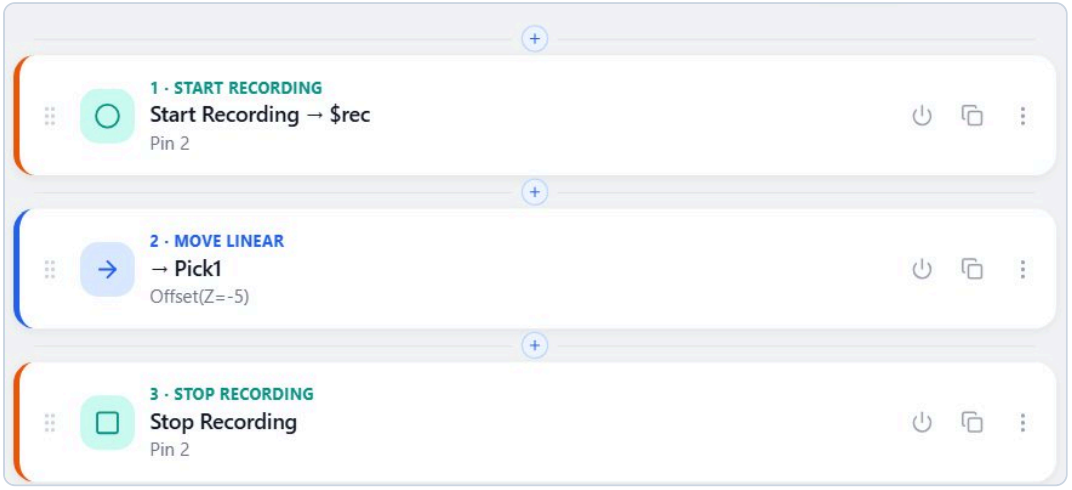
- Starts capturing a load cell. The samples arrive in the list when Stop Recording runs, as records `{t, v, r}`. The recording also shows in the app's Recordings browser.

ERRORS

This step raises no runtime error codes of its own.

EXAMPLE

BLOCKS



TEXT

```
var rec: list record
StartRecording $rec, device=Arduino_Nano, pin=2
MoveL Pick1, offset=(z=-5)
StopRecording device=Arduino_Nano, pin=2
```

See also: Stop Recording (StopRecording, p. 102) · Export Data (ExportData, p. 102)

Stop Recording

StopRecording

I/O & Aux Axes Background programs: Yes

Stop a recording and store its samples.

SYNTAX

```
StopRecording device=…, pin=…
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Device	`device=`	device name		
Pin	`pin=`	number		

DESCRIPTION AND NOTES

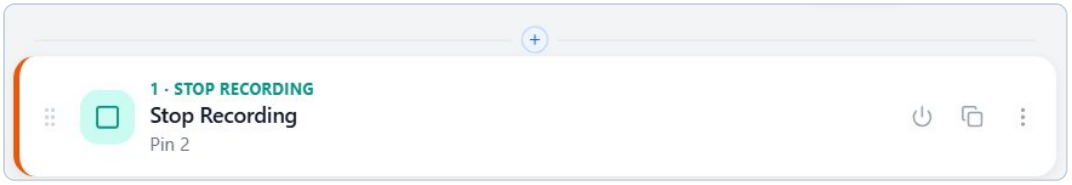
- Stops the recording this program started on that load cell and fills its list variable.

ERRORS

This step raises no runtime error codes of its own.

EXAMPLE

BLOCKS



TEXT

```
StopRecording device=Arduino_Nano, pin=2
```

See also: Start Recording (StartRecording, p. 101)

Export Data

ExportData

I/O & Aux Axes Background programs: Yes

Write variables to a CSV or JSON file.

SYNTAX

```
ExportData csv|json "<path>" … end
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Format	`position 1`	csv | json `csv` CSV · `json` JSON		`csv`
Path *	`position 2`	"text {$var}"		
Columns	`rows`	rows		

DESCRIPTION AND NOTES

- Writes one row per column (`"name" = value`) to a CSV or JSON file on the controller. A value can be an expression, `list $var` or `image $var`.

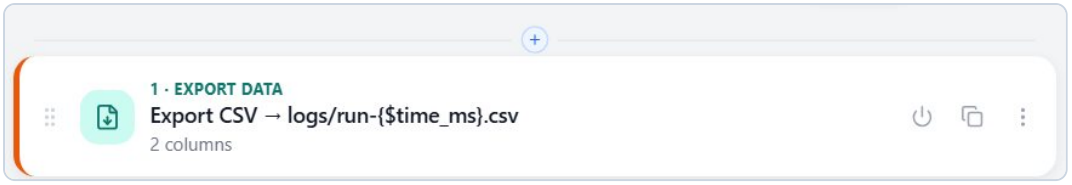
- The path is a template (`{$time_ms}` etc.); a relative path is relative to the controller's folder. Missing folders are created.

ERRORS

This step raises no runtime error codes of its own.

EXAMPLE

BLOCKS



TEXT

```
var grams = 0
var rec: list record
ExportData csv "logs/run-{$time_ms}.csv"
  "weight" = $grams
  "series" = list $rec
end
```

See also: Start Recording (StartRecording, p. 101) · HTTP Request (HttpRequest, p. 117)

Aux Move

AuxMove

I/O & Aux Axes Background programs: Yes

Move an aux axis by a distance or a number of steps.

SYNTAX

```
AuxMove device=…, axis=…, distance=…|steps=…[, unit=mm|deg][, velocity=…][, absolute=on][, wait=off][, until=…]
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Device The aux IO card.	`device=`	device name		
Axis	`axis=`	number		`0`
Unit Empty = raw steps.	`unit=`	mm | deg `mm` mm · `deg` deg		
Distance fx In the unit above.	`distance=`	number or (expr)		
Steps fx Used when no unit is set; the sign is the direction.	`steps=`	number or (expr)		
Velocity fx	`velocity=`	number or (expr)		
Acceleration fx	`accel=`	number or (expr)		
Deceleration fx	`decel=`	number or (expr)		
Absolute	`absolute=`	on / off		`off`
Wait until done	`wait=`	on / off		`on`

Parameter	In text	Type / values	Unit	Default
Stop condition fx Watched every tick while the move runs, e.g. $io.Nano.Load >= 2000.	`until=`	condition		
When true	`untilMode=`	stop | record `stop` Stop the move · `record` Record the position, keep moving		`stop`
Position variable Receives the axis position (mm / deg, or steps).	`untilPos=`	$variable		
Fired flag Set to 1 if the condition fired during the move, else 0.	`untilFlag=`	$variable		

DESCRIPTION AND NOTES

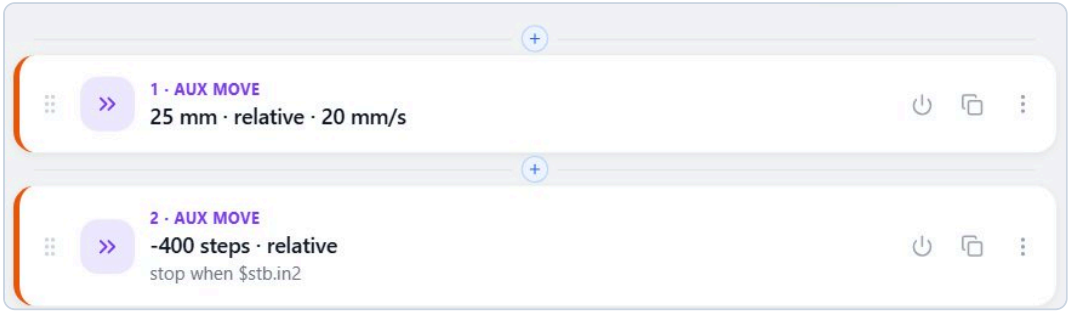
- Moves an auxiliary stepper axis by `distance=` in `unit=` (mm or deg) or by raw `steps=`; the sign is the direction. `absolute=on` moves to a position instead.
- `wait=on` (default) holds the program until the motor finishes; `wait=off` fires and forgets.
- Takes a stop condition only when it waits.

ERRORS

Code	Kind	When
`unknownVariable`	fatal	An expression names a variable that does not exist (fatal).
`expressionSyntax`	fatal	An expression does not parse (fatal).

EXAMPLE

BLOCKS



TEXT

```
AuxMove device=Aux_Stepper, axis=0, unit=mm, distance=25, velocity=20
AuxMove device=Aux_Stepper, axis=0, steps=-400, until=$stb.in2
```

See also: Aux Stop (AuxStop, p. 105) · Aux Continuous Run (AuxContinuous, p. 104) · Gripper (GripperAction, p. 107)

Aux Continuous Run

`AuxContinuous`

I/O & Aux Axes Background programs: Yes

Run an aux axis continuously until an Aux Stop.

SYNTAX

```
AuxContinuous device=…, axis=…, velocity=…[, accel=…]
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Device The aux IO card.	`device=`	device name		
Axis	`axis=`	number		`0`
Unit	`unit=`	mm | deg `mm` mm · `deg` deg		
Velocity fx The sign is the direction.	`velocity=`	number or (expr)		
Acceleration fx	`accel=`	number or (expr)		

DESCRIPTION AND NOTES

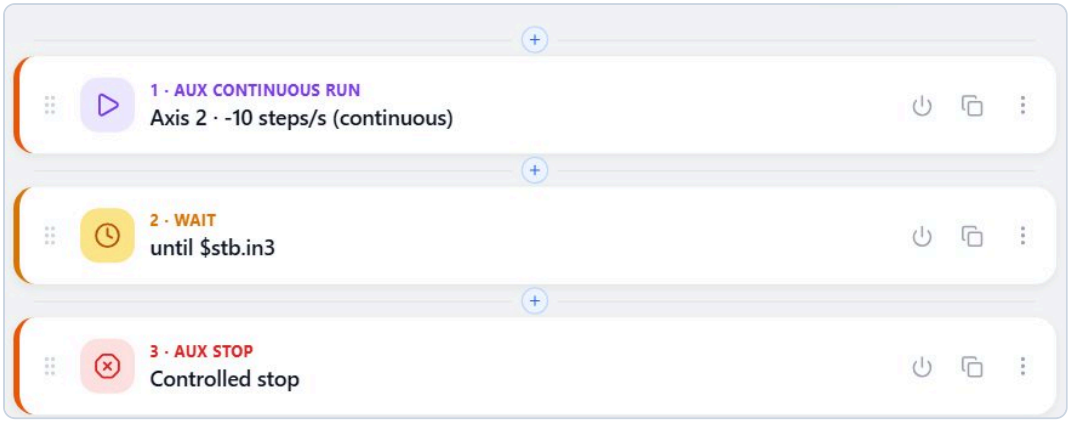
- Runs an aux axis at a constant velocity (sign = direction) until an Aux Stop.

ERRORS

This step raises no runtime error codes of its own.

EXAMPLE

BLOCKS



TEXT

```
AuxContinuous device=Aux_Stepper, axis=2, velocity=-10
Wait until $stb.in3
AuxStop device=Aux_Stepper, axis=2
```

See also: Aux Stop (AuxStop, p. 105)

Aux Stop

AuxStop

I/O & Aux Axes Background programs: Yes

Stop aux axis motion — ramped or immediate.

SYNTAX

```
AuxStop device=…, axis=…[, decel=…][, immediate=on]
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Device The aux IO card.	`device=`	device name		

Parameter	In text	Type / values	Unit	Default
Axis	`axis=`	number		`0`
Deceleration fx	`decel=`	number or (expr)		
Immediate	`immediate=`	on / off		`off`

DESCRIPTION AND NOTES

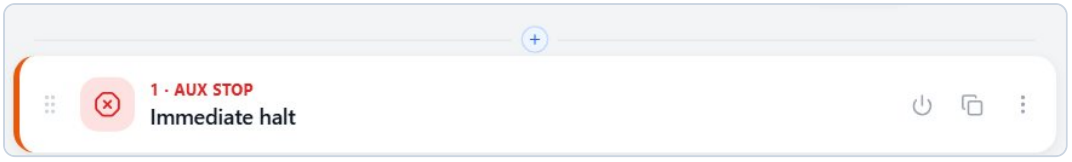
- Stops an aux axis with a ramp (`decel=`), or at once with `immediate=on` .

ERRORS

This step raises no runtime error codes of its own.

EXAMPLE

BLOCKS



TEXT

```
AuxStop device=Aux_Stepper, axis=2, immediate=on
```

See also: Aux Continuous Run (AuxContinuous, p. 104) · Aux Move (AuxMove, p. 103)

Aux Motor Enable

`AuxEnable`

I/O & Aux Axes Background programs: Yes

Enable or disable the motor drivers of an aux card.

SYNTAX

```
AuxEnable device=…, axis=…, on|off
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Device The aux IO card.	`device=`	device name		
Axis	`axis=`	number		`0`
Enabled	`position 1`	on / off		`on`

DESCRIPTION AND NOTES

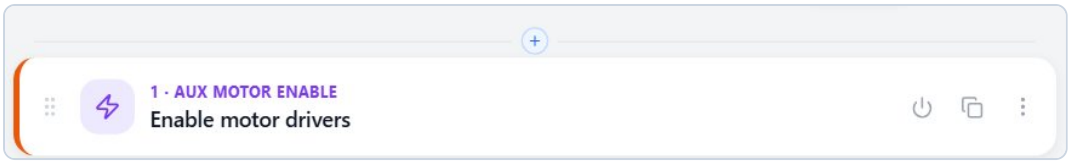
- Enables or disables the motor drivers of an aux card. A disabled motor can be turned by hand.

ERRORS

This step raises no runtime error codes of its own.

EXAMPLE

BLOCKS



TEXT

```
AuxEnable on, device=Aux_Stepper, axis=0
```

See also: Aux Move (AuxMove, p. 103)

Gripper

GripperAction

I/O & Aux Axes Background programs: Yes

Open, close or grip to a jaw gap.

SYNTAX

```
GripperAction open|close|to <mm>[, device=…, axis=…][, velocity=…][, until=…]
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Action *	`position 1`	open | close | to `Open` Open · `Close` Close · `GripTo` Grip to		`Open`
Jaw gap fx when gripperAction = GripTo	`to=`	number or (expr)	mm	
Device The aux IO card.	`device=`	device name		
Axis	`axis=`	number		`0`
Speed fx	`velocity=`	number or (expr)	mm/s	
Acceleration fx	`accel=`	number or (expr)	mm/s²	
Stop condition fx Watched every tick while the move runs, e.g. $io.Nano.Load >= 2000.	`until=`	condition		
When true	`untilMode=`	stop | record `stop` Stop the move · `record` Record the position, keep moving		`stop`
Position variable Receives the axis position (mm / deg, or steps).	`untilPos=`	$variable		
Fired flag Set to 1 if the condition fired during the move, else 0.	`untilFlag=`	$variable		

DESCRIPTION AND NOTES

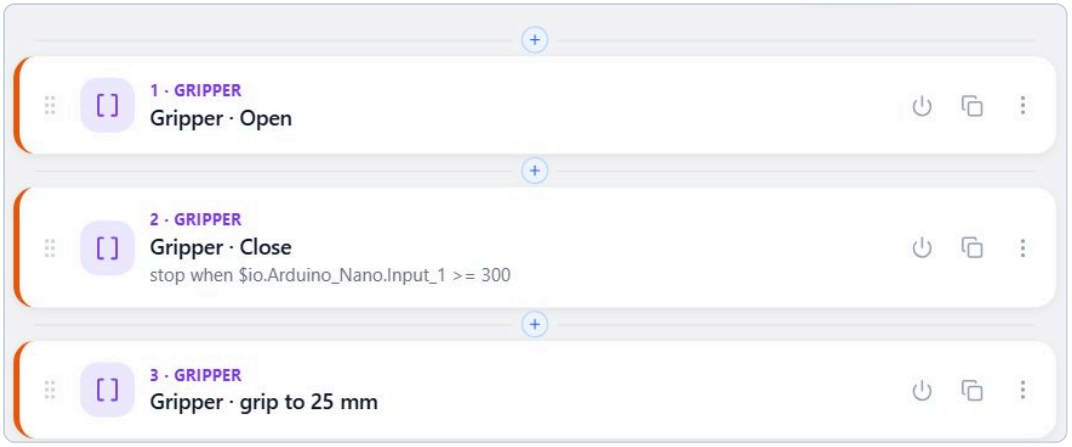
- Drives a gripper-role aux axis to its configured open or closed gap, or to a jaw gap in mm (`to 25`).
- With a stop condition the jaws stop when it holds — for example close until a load cell reads a grip force.

ERRORS

Code	Kind	When
`unknownVariable`	fatal	An expression names a variable that does not exist (fatal).
`expressionSyntax`	fatal	An expression does not parse (fatal).

EXAMPLE

BLOCKS



TEXT

```
GripperAction open
GripperAction close, until=$io.Arduino_Nano.Input_1 >= 300
GripperAction to 25
```

See also: Aux Move (AuxMove, p. 103) · Jump Linear (JumpL, p. 63)

Conveyor

ConveyorAction

I/O & Aux Axes Background programs: **Yes**

Run a conveyor forward or reverse, or stop it.

SYNTAX

```
ConveyorAction forward|reverse|stop[, device=…, axis=…]
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Action *	`position 1`	forward | reverse | stop `RunForward` Forward · `RunReverse` Reverse · `Stop` Stop		`RunForward`
Device The aux IO card.	`device=`	device name		
Axis	`axis=`	number		`0`

DESCRIPTION AND NOTES

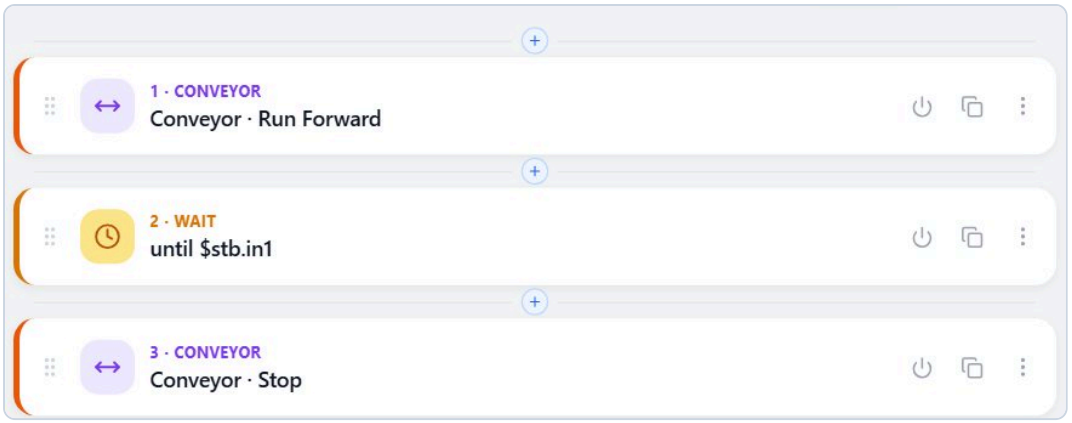
- Runs a conveyor-role aux axis forward or in reverse, or stops it.

ERRORS

This step raises no runtime error codes of its own.

EXAMPLE

BLOCKS



TEXT

```
ConveyorAction forward, device=Aux_Stepper, axis=2
Wait until $stb.in1
ConveyorAction stop, device=Aux_Stepper, axis=2
```

See also: Rotary Index (RotaryIndex, p. 109) · Aux Continuous Run (AuxContinuous, p. 104)

Rotary Index

`RotaryIndex`

I/O & Aux Axes Background programs: **Yes**

Index a rotary axis a number of positions.

SYNTAX

```
RotaryIndex +1|-1[, count=…][, device=…, axis=…]
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Direction	`position 1`	number `1` +1 · `-1` -1		`1`
Count	`count=`	number at least 1		`1`
Device The aux IO card.	`device=`	device name		
Axis	`axis=`	number		`0`

DESCRIPTION AND NOTES

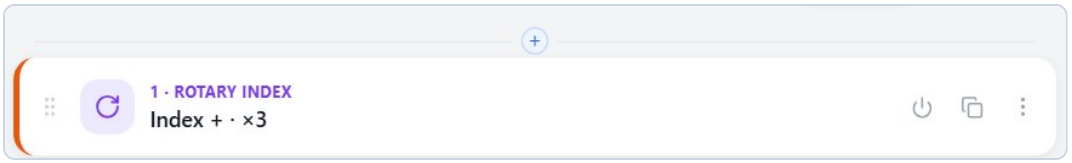
- Turns a rotary-role aux axis by `count` positions in the given direction (+1 or −1).

ERRORS

This step raises no runtime error codes of its own.

EXAMPLE

BLOCKS



TEXT

```
RotaryIndex +1, count=3, device=Aux_Stepper, axis=3
```

See also: Conveyor (ConveyorAction, p. 108)

Vision

Run camera inspections and capture or save images.

Step	Keyword	Description	Page
Run Vision	`RunVision`	Run a vision program and write its results into variables.	111
Capture Image	`CaptureImage`	Store the current camera frame in an image variable.	112
Save Image	`SaveImage`	Save a camera snapshot to a file.	113

Run Vision

`RunVision`

Vision Background programs: Yes

Run a vision program and write its results into variables.

SYNTAX

```
RunVision <program>[, frame=pixel|normalized|robot][, pass=$v] … end
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Vision program *	`position 1`	vision program name		
Coordinates	`frame=`	pixel | normalized | robot `pixel` Pixels · `normalized` Normalized 0–1 · `robot` Robot (mm)		`pixel`
Pass variable	`pass=`	$variable		
Tool outputs Rows of tool · value → variable.	`rows`	mappingOut		

DESCRIPTION AND NOTES

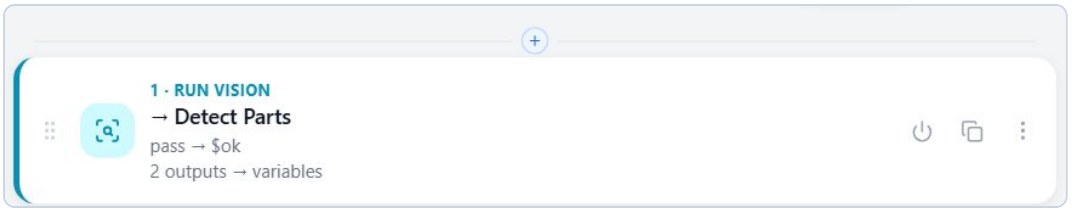
- Runs a vision program on a fresh camera frame and writes tool results into variables, one row per `tool.value -> $var`. Values include any key of the tool's values (`angle`, `count`, `x` …), `passed`, `found`, `status`, and the lists `points`, `items`, `cells`, `lines`, `pose`.
- `frame=` sets the coordinates of positions written: `pixel` (default), `normalized` (0–1) or `robot` (mm on the camera's calibration plane — needs a calibrated camera).
- `pass=$v` gets 1 when every tool that counts toward the pass passed. Variables named in rows are created if not declared.

ERRORS

Code	Kind	When
`visionTimeout`	catchable	No fresh result in time (default 30 s) (catchable).
`visionFailed`	catchable	The vision processor is lost, or the robot frame is not available (catchable).

EXAMPLE

BLOCKS



TEXT

```
var ok: bool = off
var edgeAngle = 0
var cells: list record
RunVision Detect_Parts, frame=normalized, pass=$ok
  Top_edge.angle -> $edgeAngle
  Slots.cells -> $cells
end
```

See also: Capture Image (CaptureImage, p. 112) · Save Image (SaveImage, p. 113) · Try (try, p. 83)

Capture Image

`CaptureImage`

Vision Background programs: Yes

Store the current camera frame in an image variable.

SYNTAX

```
CaptureImage $<image>[, camera=…]
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Image variable*	`position 1`	$variable		
Camera	`camera=`	camera name		

DESCRIPTION AND NOTES

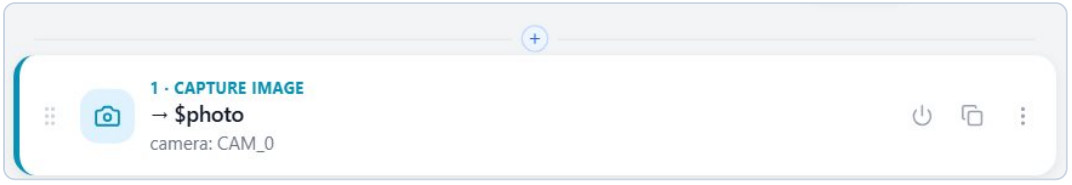
- Stores the current camera frame (JPEG) in an image variable — to send over HTTP, hand to a plugin, export or show on a display page.

ERRORS

Code	Kind	When
`visionFailed`	catchable	The camera has no frame (catchable).

EXAMPLE

BLOCKS



TEXT

```
var photo: image
CaptureImage $photo, camera=Top_Camera
```

See also: Save Image (SaveImage, p. 113) · HTTP Request (HttpRequest, p. 117)

Save Image

SaveImage

Vision Background programs: Yes

Save a camera snapshot to a file.

SYNTAX

```
SaveImage "<path>"[, camera=…]
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Path*	`position 1`	"text {$var}"		
Camera	`camera=`	camera name		

DESCRIPTION AND NOTES

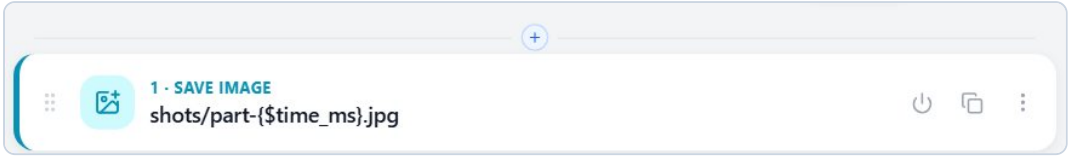
- Saves a camera snapshot to a file on the controller. The path is a template: `"shots/{$time_ms}.jpg"`.

ERRORS

Code	Kind	When
`visionFailed`	catchable	The camera has no frame (catchable).

EXAMPLE

BLOCKS



TEXT

```
SaveImage "shots/part-{$time_ms}.jpg", camera=Top_Camera
```

See also: Capture Image (CaptureImage, p. 112)

Background

Start, stop and wait on programs running in parallel.

Step	Keyword	Description	Page
Start Background	`StartBackground`	Start a background program in parallel.	114
Stop Background	`StopBackground`	Stop a background program.	114
Wait for Background	`WaitForBackground`	Wait until a background program finishes.	115

Start Background

`StartBackground`

Background Background programs: Yes

Start a background program in parallel.

SYNTAX

```
StartBackground <program>
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Background program *	`position 1`	program name		

DESCRIPTION AND NOTES

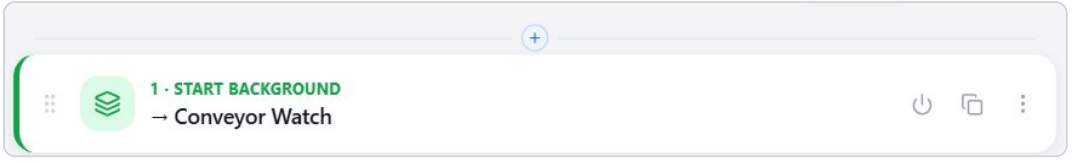
- Starts a background program running in parallel with this one. Background programs skip motion, tool and homing steps.

ERRORS

This step raises no runtime error codes of its own.

EXAMPLE

BLOCKS



TEXT

```
StartBackground Conveyor_Watch
```

See also: Stop Background (StopBackground, p. 114) · Wait for Background (WaitForBackground, p. 115)

Stop Background

`StopBackground`

Background Background programs: Yes

Stop a background program.

SYNTAX

```
StopBackground <program>
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Background program *	`position 1`	program name		

DESCRIPTION AND NOTES

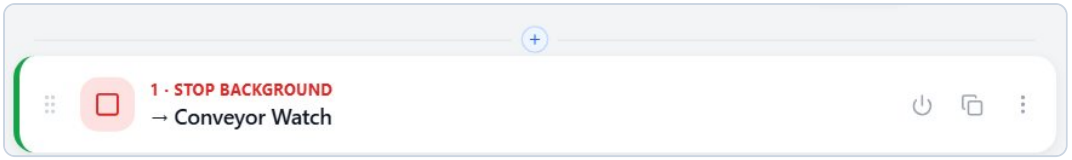
- Stops a running background program.

ERRORS

This step raises no runtime error codes of its own.

EXAMPLE

BLOCKS



TEXT

```
StopBackground Conveyor_Watch
```

See also: Start Background (StartBackground, p. 114)

Wait for Background

`WaitForBackground`

Background Background programs: Yes

Wait until a background program finishes.

SYNTAX

```
WaitForBackground <program>
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Background program *	`position 1`	program name		

DESCRIPTION AND NOTES

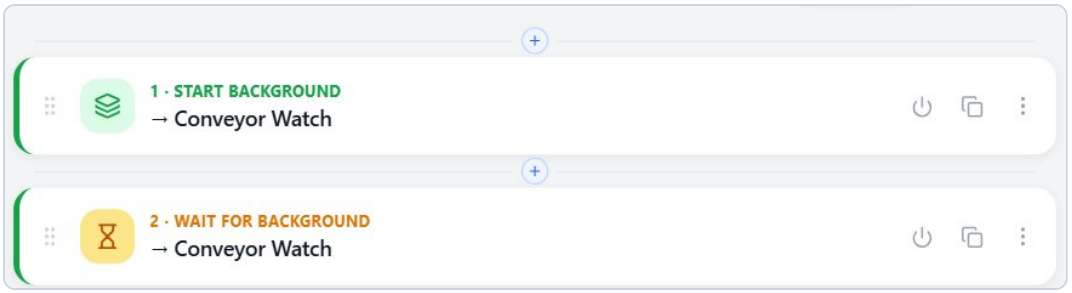
- Holds this program until the background program finishes.

ERRORS

This step raises no runtime error codes of its own.

EXAMPLE

BLOCKS



TEXT

```
StartBackground Conveyor_Watch
WaitForBackground Conveyor_Watch
```

See also: Start Background (StartBackground, p. 114)

Communication

Talk to the operator and to other machines over HTTP.

Step	Keyword	Description	Page
HTTP Request	`HttpRequest`	POST JSON to a server and read values from the reply.	117
HTTP Receive	`HttpReceive`	Wait for an HTTP POST to a named webhook.	118
Status Update	`status`	Publish a message, warning or error to the monitor.	119
Prompt	`prompt`	Ask the operator: show a message with buttons and an optional input.	120

HTTP Request

`HttpRequest`

Communication Background programs: **Yes**

POST JSON to a server and read values from the reply.

SYNTAX

```
HttpRequest "<url>"[, wait=on][, timeout=…] send … receive … end
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
URL *	`position 1`	text		
Wait for the reply	`wait=`	on / off		`on`
Timeout	`timeout=`	duration (500ms, 2s)	ms	
Send	`send=`	rows		
Receive	`receive=`	mappingOut		

DESCRIPTION AND NOTES

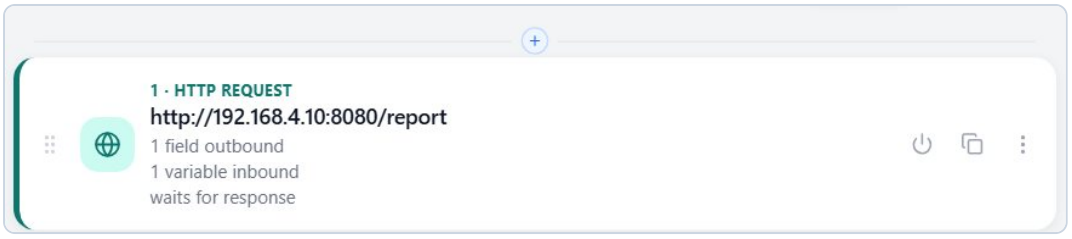
- POSTs a JSON object built from the `send` rows (`"key" = expression` , `list $var` , `image $var`) and, with `wait=on` , reads `receive "key" -> $var` values from the JSON reply.
- An empty or non-JSON reply is not a failure. With `wait=off` the request never fails.

ERRORS

Code	Kind	When
`httpFailed`	catchable	Network error or a non-2xx status (catchable).
`httpTimeout`	catchable	No reply in time (catchable).

EXAMPLE

BLOCKS



TEXT

```
var count = 0
var reply = 0
HttpRequest "http://192.168.4.10:8080/report", wait=on, timeout=5s
  send "count" = $count
  receive "next" -> $reply
end
```

See also: HTTP Receive (HttpReceive, p. 118) · Export Data (ExportData, p. 102)

HTTP Receive

`HttpReceive`

Communication Background programs: Yes

Wait for an HTTP POST to a named webhook.

SYNTAX

```
HttpReceive "<name>"[, timeout=…] receive … end
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Name * Other machines POST to /webhook/<name>.	`position 1`	text		
Timeout	`timeout=`	duration (500ms, 2s)	ms	
Receive	`receive=`	mappingOut		

DESCRIPTION AND NOTES

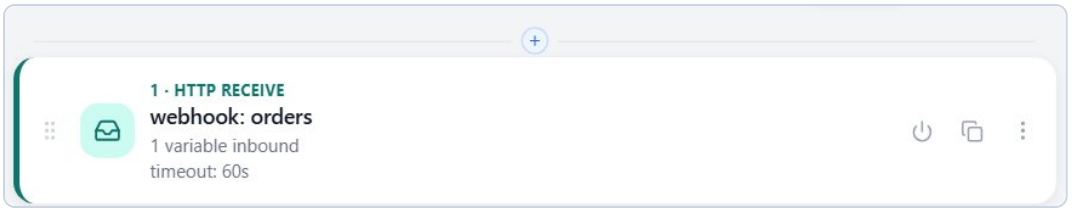
- Waits for another machine to POST JSON to `/webhook/<name>` on the controller, then copies `receive` keys into variables.

ERRORS

Code	Kind	When
`receiveTimeout`	catchable	Nothing arrived in time (catchable).

EXAMPLE

BLOCKS



TEXT

```
var x = 0
HttpReceive "orders", timeout=60s
  receive "x" -> $x
end
```

See also: HTTP Request (HttpRequest, p. 117)

Status Update

`status`

Communication Background programs: **Yes**

Publish a message, warning or error to the monitor.

SYNTAX

```
status [info|warning|error] "<message>"
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Severity	`rows`	Info | Warning | Error `Info` Info · `Warning` Warning · `Error` Error		`Info`
Message	`position 1`	"text {$var}"		
Warning	`warning=`	"text {$var}"		
Error	`error=`	"text {$var}"		

DESCRIPTION AND NOTES

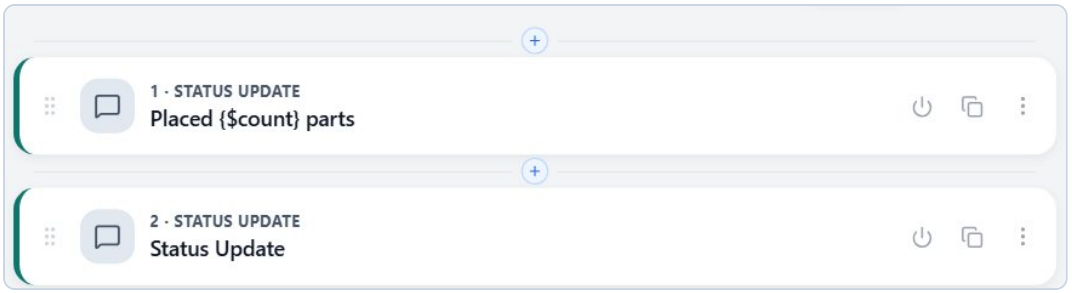
- Publishes a message to the monitor and the run log: `status "…"`, `status warning "…"`, `status error "…"`. The text is a template. An error status does not stop the program.

ERRORS

This step raises no runtime error codes of its own.

EXAMPLE

BLOCKS



TEXT

```
var count = 0
status "Placed {$count} parts"
status warning "Tray almost full"
```

See also: Prompt (prompt, p. 120) · Raise Error (raise, p. 85)

Prompt

`prompt`

Communication Background programs: Yes

Ask the operator: show a message with buttons and an optional input.

SYNTAX

```
prompt "<message>"[, title=…][, buttons=(…)][, input=number|text][, result=$v][, value=$v][, timeout=…]
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Message*	`position 1`	"text {$var}"		
Title	`title=`	"text {$var}"		
Buttons	`buttons=`	(a, b, ...) range 1 – 4		`OK`
Button variable A number variable gets the button index (from 0); a text variable gets its label.	`result=`	$variable		
Input	`input=`	none | number | text `none` None · `number` Number · `text` Text		`none`
Default fx when promptInput = number / text	`default=`	expression		
Minimum when promptInput = number	`min=`	number		
Maximum when promptInput = number	`max=`	number		
Input variable when promptInput = number / text	`value=`	$variable		
Timeout 0 = wait forever.	`timeout=`	duration (500ms, 2s)	ms	`0`

Parameter	In text	Type / values	Unit	Default
On timeout press Button index chosen on timeout. Empty = fail (catchable).	`onTimeout=`	number range 0 – 3		

DESCRIPTION AND NOTES

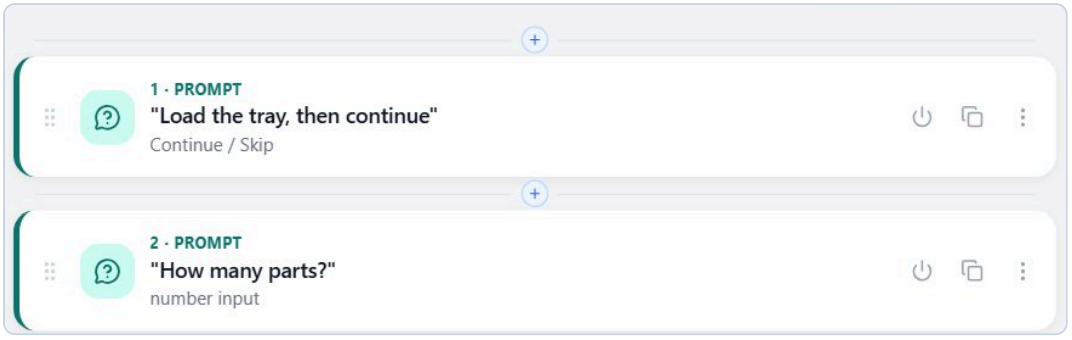
- Shows the operator a dialog with 1–4 buttons and optionally a number or text input, and waits. A number result variable gets the button index (from 0), a text one its label.
- With `timeout=` and `onTimeout=<button>` the button is pressed on timeout; without a button the prompt fails `promptTimeout`.
- The dialog appears in the app on any screen. A Stop keeps it on screen but refuses answers; Continue shows it again.

ERRORS

Code	Kind	When
`promptTimeout`	catchable	Timed out with no timeout button (catchable).

EXAMPLE

BLOCKS



TEXT

```
var choice = 0
var qty = 0
prompt "Load the tray, then continue", title="Setup", buttons=("Continue", "Skip"), result=$choice
prompt "How many parts?", input=number, default=10, min=1, max=50, value=$qty
```

See also: Status Update (status, p. 119) · Pause Program (pause, p. 90)

Plugins

Steps provided by installed plugins.

Step	Keyword	Description	Page
Plugin	`plugin`	A step provided by an installed plugin.	122

Plugin

`plugin`

Plugins Background programs: Yes

A step provided by an installed plugin.

SYNTAX

```
plugin <id>.<step>[, timeout=…] <param> = … <output> -> $v end
```

PARAMETERS

Parameter	In text	Type / values	Unit	Default
Plugin step *	`position 1`	pluginRef		
Parameters Typed by the plugin manifest.	`rows`	exprMap		
Outputs	`rows`	mappingOut		
Timeout Empty = the manifest default; 0 = none.	`timeout=`	duration (500ms, 2s)	ms	

DESCRIPTION AND NOTES

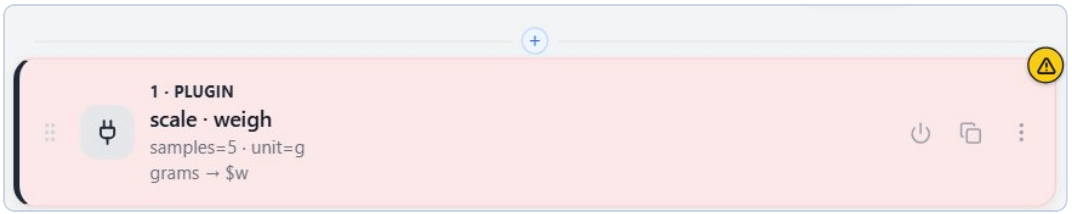
- Runs a step an installed plugin declares in its manifest. Parameter rows: unquoted = an expression, quoted = exact text. Output rows `key -> $var` write the plugin's results.
- The program waits (without blocking anything else) until the plugin answers. Background programs can run plugin steps.

ERRORS

Code	Kind	When
`pluginNotRunning`	catchable	The plugin is not running (catchable).
`pluginFailed`	catchable	The plugin answered with an error (catchable).
`pluginTimeout`	catchable	No answer in time (catchable).

EXAMPLE

BLOCKS



TEXT

```
var w = 0
plugin scale.weigh, timeout=5s
  samples = 5
  unit = "g"
  grams -> $w
end
```

See also: HTTP Request (HttpRequest, p. 117)

CHAPTER 17

Functions and Properties Reference

Everything an expression can read. Operators and precedence are described in Expressions and Conditions; this chapter lists the functions, the read-only properties and the IO names.

Operators

Operator	Meaning	Example
`+ - * /`	arithmetic (`/` by zero gives 0)	`$w * $h`
`%`	remainder, with the sign of the left side (by zero gives 0)	`$i % 4`
`^`	power, right-associative	`2 ^ 10`
`== != < <= > >=`	comparison → 1 / 0 (`==` within 1e-9)	`$grams >= 100`
`and or not` (`&& || !`)	logic → 1 / 0	`$a and not $b`
`c ? a : b`	conditional	`$n > 0 ? $sum / $n : 0`
`( )`	grouping	`($a + $b) / 2`

Notes on functions

- Function names are case-insensitive.
- Angles are in **degrees**, in and out.
- `round` rounds half away from zero: `round(2.5)` = 3, `round(-2.5)` = -3; negative digits round to tens, hundreds …
- The list functions take a bare list reference: `sum($vals)`, `avg($samples.v)` (a field of each element of a Point or Record list). An empty list gives 0.
- An unknown function is `unknownFunction`; a wrong number of arguments is `badArity`.
- A running plugin's functions are called as `<pluginId>.<name>(…)`, its properties read as `$<pluginId>.<name>` — see Plugins.

Function list

Function	Result
`abs(x)`	Absolute value
`sign(x)`	-1, 0 or 1
`sqrt(x)`	Square root
`pow(x, y)`	x raised to y (same as x ^ y)
`min(a, b, …) / min($list) / min($list.field)`	Smallest argument, or smallest element of a list (0 when empty)
`max(a, b, …) / max($list) / max($list.field)`	Largest argument, or largest element of a list (0 when empty)
`clamp(x, lo, hi)`	x limited to lo…hi
`round(x) / round(x, digits)`	Round half away from zero, optionally to digits decimals
`floor(x)`	Round down

Function	Result
`ceil(x)`	Round up
`trunc(x)`	Drop the fraction (toward zero)
`mod(a, b)`	Remainder, same as a % b (0 when b is 0)
`sin(deg)`	Sine of an angle in degrees
`cos(deg)`	Cosine of an angle in degrees
`tan(deg)`	Tangent of an angle in degrees
`asin(x)`	Arcsine, in degrees
`acos(x)`	Arccosine, in degrees
`atan(x)`	Arctangent, in degrees
`atan2(y, x)`	Angle of (x, y) in degrees
`deg(rad)`	Radians to degrees
`rad(deg)`	Degrees to radians
`hypot(x, y)`	sqrt(x^2 + y^2)
`dist(x1, y1, x2, y2)`	Distance between two XY points
`dist3(x1, y1, z1, x2, y2, z2)`	Distance between two XYZ points
`if(cond, a, b)`	a when cond is non-zero, else b (same as cond ? a : b)
`len($list)`	Number of elements in a list
`sum($list) / sum($list.field)`	Sum of a number list, or of one field of each element
`avg($list) / avg($list.field)`	Average of a number list, or of one field of each element (0 when empty)
`minOf($list) / minOf($list.field)`	Smallest element of a number list (0 when empty) — same as min($list)
`maxOf($list) / maxOf($list.field)`	Largest element of a number list (0 when empty) — same as max($list)
`rand() / rand(lo, hi)`	Uniform random number in [0, 1) or [lo, hi)
`map(x, inLo, inHi, outLo, outHi)`	Re-map x from one range to another
`lerp(a, b, t)`	a + (b - a) × t
`inBox(x, y, z, x1, y1, z1, x2, y2, z2)`	1 when (x, y, z) is inside the box with opposite corners (x1, y1, z1) and (x2, y2, z2), edges included; else 0
`inCircle(x, y, cx, cy, r)`	1 when (x, y) is within r of (cx, cy), edge included; else 0
`$text.contains("x")`	1 when the text contains x, else 0
`$text.startsWith("x")`	1 when the text starts with x, else 0
`$text.endsWith("x")`	1 when the text ends with x, else 0
`$text.indexOf("x")`	Position of the first x in the text (from 0), or -1 when absent
`$text.toNumber()`	The text as a number; 0 when it is not one
`$text.length`	Number of characters in the text (on a list: its element count)

Properties

Read-only; resolved live whenever an expression reads them. Never assignable.

Robot

Property	Type	Value
`$robot.x`	number	Current TCP X (mm)
`$robot.y`	number	Current TCP Y (mm)
`$robot.z`	number	Current TCP Z (mm)
`$robot.rx`	number	Current TCP RX (deg)
`$robot.ry`	number	Current TCP RY (deg)
`$robot.rz`	number	Current TCP RZ (deg)
`$robot.targetX`	number	Commanded target X (mm)
`$robot.targetY`	number	Commanded target Y (mm)
`$robot.targetZ`	number	Commanded target Z (mm)
`$robot.targetRx`	number	Commanded target RX (deg)
`$robot.targetRy`	number	Commanded target RY (deg)
`$robot.targetRz`	number	Commanded target RZ (deg)
`$robot.moving`	boolean	1 while the robot is moving
`$robot.homed`	boolean	1 once the robot has been homed
`$robot.faulted`	boolean	1 while a joint-limit fault is active
`$robot.driverConnected`	boolean	1 while the motor driver is connected
`$robot.speedS`	number	Current linear speed default (mm/s)
`$robot.accelS`	number	Current linear accel default (mm/s^2)
`$robot.decelS`	number	Current linear decel default (mm/s^2)
`$robot.speedJ`	number	Current joint speed default
`$robot.accelJ`	number	Current joint accel default
`$robot.decelJ`	number	Current joint decel default
`$robot.speedOverride`	number	Speed override (percent)
`$robot.joint1`	number	Joint 1 angle (deg)
`$robot.joint2x`	number	Joint 2 X readout
`$robot.joint2z`	number	Joint 2 Z readout
`$robot.joint4`	number	Joint 4 angle (deg)
`$robot.joint1Referenced`	boolean	1 once J1/X is homed or set to a known value
`$robot.joint2Referenced`	boolean	1 once Horizontal/Y is homed or set to a known value
`$robot.joint3Referenced`	boolean	1 once Vertical/Z is homed or set to a known value
`$robot.joint4Referenced`	boolean	1 once J4/RZ is homed or set to a known value

Program

Property	Type	Value
`$program.runCount`	number	Times this program has been started (this run included)
`$program.stepIndex`	number	Completed top-level steps in this run
`$program.stepCount`	number	Total steps in this run
`$program.elapsedMs`	number	Milliseconds since this run started
`$program.loopDepth`	number	Number of loops currently executing
`$program.speedFactor`	number	This program's Set Speed Factor (percent, 100 = full speed)

Time

Property	Type	Value
`$time.now`	number	Unix time in milliseconds
`$time.hour`	number	Local hour (0–23)
`$time.minute`	number	Local minute (0–59)
`$time.second`	number	Local second (0–59)
`$time.dayOfWeek`	number	Day of the week, 0 = Sunday
`$time.dayOfYear`	number	Day of the year (1–366)

Aux axes

Property	Type	Value
`$aux.<deviceId>.moving`	boolean	1 while the aux device is moving
`$aux.<deviceId>.<axis>.position`	number	Aux axis position (steps)

Built-in variable

Property	Type	Value
`$time_ms`	number	Unix time in milliseconds (built-in variable; same as $time.now)

Cameras

Property	Type	Value
`$camera.<cameraId>.calibrated`	boolean	1 when the camera has a camera-to-robot calibration

Plugins

Property	Type	Value
`$<pluginId>.<name>`	number	A running plugin's live property, e.g. $scale.weight

IO names

Name	Value
`$stb.in1 … $stb.in4`	Driver board inputs (1/0)
`$stb.out1 … $stb.out4`	Driver board outputs (1/0)
`$relay.1 … $relay.4`	USB relay channels (1/0)
`$io.<device>.<pin>`	An IO device pin: digital 1/0, a load cell in grams. Names with spaces use their underscore form: $io.Arduino_Nano.Load_Cell

CHAPTER 18

Error and Problem Codes

Three kinds of codes:

Kind	When	Effect
Runtime error codes	a step fails while the program runs	Catchable codes can be handled with a Try; fatal codes always end the program with Error. The code is in the program status (`errorCode`) and the Try's code variable.
Validation problems	the controller checks the program (every edit, and before a run)	Errors block running; warnings are hints.
Parse problems	the text view is read back into a program	Errors block switching to Blocks and saving; warnings (unknown names) do not. Each has a line and column.

A Raise Error step may use any identifier as its code (default `userError`); such codes are always catchable.

Runtime error codes

Code	Label	Kind	When
`waitTimeout`	Wait timed out	catchable	A condition Wait with `onTimeout=error` reached its timeout. Raised by: Wait
`visionTimeout`	Vision timed out	catchable	Run Vision got no fresh result in time. Raised by: Run Vision
`visionFailed`	Vision failed	catchable	The vision processor was lost, the robot output frame is not available, or a camera had no frame (Capture / Save Image). Raised by: Run Vision, Capture Image, Save Image
`httpFailed`	HTTP request failed	catchable	HTTP Request: network error or a non-2xx status. Raised by: HTTP Request
`httpTimeout`	HTTP request timed out	catchable	HTTP Request: no reply in time. Raised by: HTTP Request
`receiveTimeout`	HTTP receive timed out	catchable	HTTP Receive: nothing was posted in time. Raised by: HTTP Receive
`pluginFailed`	Plugin step failed	catchable	A plugin step or function answered with an error. Raised by: Plugin
`pluginTimeout`	Plugin step timed out	catchable	A plugin step did not answer in time. Raised by: Plugin
`pluginNotRunning`	Plugin not running	catchable	The plugin is installed but not running. Raised by: Plugin
`promptTimeout`	Prompt timed out	catchable	A Prompt timed out and has no timeout button. Raised by: Prompt
`pointNotFound`	Point not found	catchable	A template or list target names no saved point, a Points list is empty, or a via point is unknown. Raised by: Move Linear, Move Joint, Jump Linear, Jump Joint, Move Arc, Save Position

Code	Label	Kind	When
`arcDegenerate`	Arc points are in a line	catchable	Move Arc: start, via and end are on one line, or two are the same. Raised by: Move Arc
`indexOutOfRange`	List index out of range	catchable	List / Save Position: the index is outside the list. Raised by: Save Position, List
`userError`	Raised error	catchable	The default code of Raise Error. Any identifier you raise yourself is also catchable. Raised by: Raise Error
`motionFault`	Motion fault	fatal	The motion system reported a fault. Raised by: Move Linear, Move Joint, Jump Linear, Jump Joint, Move Arc, Thread Move, Run Homing, CNC Program
`jointLimit`	Joint limit	fatal	A target is outside a joint's soft limits. Raised by: Move Linear, Move Joint, Jump Linear, Jump Joint, Move Arc, Thread Move, Run Homing, CNC Program
`driverDisconnected`	Driver disconnected	fatal	The motor driver disconnected. Raised by: Move Linear, Move Joint, Jump Linear, Jump Joint, Move Arc, Thread Move, Run Homing, CNC Program
`emergencyStop`	Emergency stop	fatal	The emergency stop was pressed.
`userStop`	Stopped by user	fatal	The operator stopped the program.
`unknownVariable`	Unknown variable	fatal	An expression names a variable, property or IO point that does not exist. Raised by: Move Linear, Move Joint, Jump Linear, Jump Joint, Set Speed (Linear), Set Speed (Joint), Set Blend Radius, Set Speed Factor, If Condition, Switch, Loop, Call Routine, Set Variable, List, Wait, Set Output, Aux Move, Gripper
`expressionSyntax`	Expression error	fatal	An expression does not parse (also `unknownFunction` / `badArity`). Raised by: Move Linear, Move Joint, Jump Linear, Jump Joint, Set Speed (Linear), Set Speed (Joint), Set Blend Radius, Set Speed Factor, If Condition, Switch, Loop, Call Routine, Set Variable, List, Wait, Set Output, Aux Move, Gripper
`computedVariable`	Write to a computed variable	fatal	A step tried to write a computed variable. Raised by: Set Variable
`badListOperation`	List step does not fit the list	fatal	A List or Save Position step does not fit its list (an authoring error). Raised by: Save Position, List

Validation problems

Reported by the validator (`ValidateBuiltProgram`) for both views, on the step and field concerned.

Code	Severity	Meaning
`unknownPoint`, `unknownTool`, `unknownLocal`, `unknownRoutine`, `unknownVisionProgram`, `unknownGrid`, `unknownStack`, `unknownProgram`	error	A referenced item does not exist.

Code	Severity	Meaning
`unknownLabel` / `duplicateLabel`	error	Go To Label names no label / two labels share a name.
`unknownVariable` / `unknownProperty`	error	An expression names something that does not exist.
`expressionSyntax` / `unknownFunction` / `badArity`	error	An expression does not parse.
`readOnlyProperty`	error	A property or IO name used as a write target or declared as a variable.
`missingField`	error	A required field is empty.
`emptyLoop` / `emptyBranch`	error	A loop or branch with no steps.
`routineRecursion`	error	A routine calls itself (directly or indirectly).
`loopControlOutsideLoop`	error	Break / Next outside a loop (or crossing a routine boundary).
`computedCycle` / `computedVariable` / `computedGlobalScope` / `computedKindConflict`	error	Computed-variable rules (see Data and variables).
`unknownParam` / `paramOutputType`	error	A routine argument or output that does not fit the routine's parameters.
`unsupportedOnStep`	error	A field the step does not support (e.g. a stop condition on Move Arc).
`cameraNotCalibrated` / `badOutputFrame`	error	Run Vision robot frame without a calibrated camera / an unknown frame.
`unknownPlugin` / `unknownPluginStep` / `pluginParamMissing` / `pluginParamEnum` / `pluginOutputType` / `unknownPluginFunction`	error	Plugin step and function checks.
`paramMissing`	warning	A routine parameter with no argument keeps its declared value.
`duplicateCaseValue`	warning	Two Switch cases share a value.
`pluginNotRunning`	warning	The plugin is installed but not running.
`untaughtGrid`	warning	A grid or stack whose steps are all zero.
`pointWriteInBackground`	warning	A background program overwrites a saved point.
`disabledStep` / `unreachableStep` / `unusedVariable` / `unknownStepType`	warning	Housekeeping hints.

Parse problems

Reported when text is read back (`ParseProgramText`), with a line and column. A line with a problem is skipped and parsing carries on, so every problem is reported at once.

Code	Meaning
`syntax`	Not valid here (with what was expected).
`unknownKeyword`	A statement word that does not exist (with "did you mean …").
`unknownArgument`	A named value the step does not have.
`duplicateArgument`	A named value given twice.
`missingArgument`	A required value is missing.
`badValue`	The wrong kind of value: text where a number belongs, an option that does not exist, an expression without parentheses in a positional slot.
`expressionSyntax`	From the expression checker, pointing inside the expression (also `unknownFunction`, `badArity`).
`missingEnd`	A block without `end`.
`unexpectedEnd`	An `end` with no block to close.
`misplacedClause`	`else` / `case` / `catch` in the wrong place.
`declarationKind`	An impossible declaration, e.g. `persistent computed`.
`stringRequired`	A text variable assigned an expression, or a number variable assigned text.
`duplicateId`	Two steps with the same `@id`.
`ambiguousName`	An underscore-form name fits more than one item — write the quoted name.
`unknownPoint` / `unknownTool` / `unknownRoutine` / `unknownGrid` / `unknownDevice` / `unknownLabel` …	Warning: the name does not resolve (yet); it is kept.
`unknownField`	Warning: an `@with` key that is not a field (dropped).

Appendices

A Simple Robot Script Grammar
B Keyword List
C Quick Reference Cards

APPENDIX A

Simple Robot Script Grammar

The grammar of Simple Robot Script, language version 1, in EBNF. `COND` and `EXPR` are the expression language of Expressions and Conditions.

Lexical rules

Element	Rule
Line	One statement per line. A line continues while a `(`, `[` or `{` is open, or after a trailing `,`. A blank line ends a continuation.
Comment	`#` to the end of the line, outside strings. `##` starts a section header.
Version	The first line `#!srs 1` (optional when parsing).
Strings	`"…"` with the escapes `\"` `\\` `\n` `\t` `\r`; any other backslash is kept (`"C:\data"` works). Template braces `{$x}` are kept and filled in at run time.
Numbers	`3`, `-2.5`, `1e3`. Fields in milliseconds also take `500ms` and `1.5s`.
Booleans	`on` `off` `true` `false` (as values of step arguments and declarations).
Names	Bare when an identifier (`Pick1`), else quoted (`"Pick 1"`) or in underscore form (`Pick_1`).
Variables	`$name`, with `[…]` indexes and `.field` accessors.
Positional values	Right after the keyword, separated by spaces: a name, string, number, `on` / `off`, a `$var`, or an expression **in parentheses**.
Named values	`name=value`, comma-separated; a named value runs to the next top-level comma, so it can be any expression.
Literal or expression	A plain number is stored as the number; anything else as an expression. Parentheses always mean an expression: `speed=(200)`.

Grammar

```
file        = [ "#!srs" INT NL ] { line } ;
line        = ( decl | attr | stmt | comment | blank ) [ "#" text ] NL ;
attr        = "@" ( "name" STRING | "off" | "id" STRING | "with" JSON | "program" JSON ) ;
comment     = "#" text | "##" text ;                (* paragraph → Note; "##" → section *)

decl        = { "global" | "persistent" | "input" | "monitor" }
              ( "var" | "computed" | "param" ( "in" | "out" | "inout" ) ) NAME [ ":" type ] [ "=" init ] ;
type        = "number" | "bool" | "string" | "image" | "stopwatch"
            | "list" ( "number" | "bool" | "point" | "record" ) ;
init        = EXPR | STRING | "(" [ value { "," value } ] ")" ;

stmt        = if | loop | switch | try | assign | listop | flow | keyword-stmt ;
if          = "if" COND NL body { ( "elif" | "else if" ) COND NL body } [ "else" NL body ] "end" ;
loop        = ( "repeat" [ ( "forever" | POS ) ] [ "as" VAR ]
              | "while" COND
              | "for" ( VAR | "_" ) [ "," VAR ] "in" VAR ) [ "," args ] NL body "end" ;
switch      = "switch" EXPR NL { "case" EXPR { "," EXPR } NL body } [ "default" NL body ] "end" ;
try         = "try" [ args ] NL body [ "catch" [ VAR ] [ "," args ] NL body ] "end" ;
assign      = VAR "=" ( EXPR | STRING )
            | VAR "[" EXPR "]" [ "." IDENT ] "=" ( EXPR | "(" IDENT "=" EXPR { "," IDENT "=" EXPR } ")" ) ;
listop      = VAR "." ( "append" | "insert" | "removeAt" | "clear" | "sort" | "reverse" | "copyFrom" )
```

```
              "(" [ arglist ] ")" ;
flow        = "break" | "continue" | "return" | "pause"
            | "exit" [ STRING ] | "raise" STRING [ "," args ] | "label" NAME | "goto" NAME
            | "call" NAME [ "(" IDENT "=" value { "," IDENT "=" value } ")" ] { "," "out" IDENT "=" VAR }
            | "status" [ "info" | "warning" | "error" ] [ STRING ] [ "," args ]
            | "stopwatch" ( "start" | "stop" | "reset" ) VAR
            | "prompt" STRING [ "," args ]
            | "plugin" NAME "." NAME [ "," args ] [ rows ]
            | "unknown" STRING ;
keyword-stmt= KEYWORD { POS } [ [ "," ] args ] [ rows ] ;    (* KEYWORD: a step type name *)
rows        = NL { row NL } "end" ;     (* RunVision, HttpRequest, HttpReceive, ExportData, plugin *)
row         = [ "send" ] KEY "=" ( EXPR | "list" VAR | "image" VAR )
            | [ "receive" ] KEY "->" VAR
            | KEY "." KEY "->" VAR ;                          (* RunVision: tool.value -> $var *)
args        = named { "," named } ;
named       = IDENT "=" value ;
value       = EXPR | STRING | BOOL | NUMBER [ "ms" | "s" ] | tuple | list | target ;
tuple       = "(" IDENT "=" EXPR { "," IDENT "=" EXPR } ")" ;
list        = "(" NAME { "," NAME } ")" ;
target      = "here" | NAME | STRING | VARREF | ( "grid" | "stack" ) NAME "[" EXPR [ "," EXPR ] "]" ;
POS         = NAME | STRING | NUMBER [ unit ] | BOOL | VARREF | "(" EXPR ")" | target | "J" INT | "toggle"
;
NAME        = IDENT | STRING ;
VAR         = "$" IDENT ;
VARREF      = VAR { "[" EXPR "]" | "." IDENT } ;
COND, EXPR  = the expression language, up to the next top-level "," or the end of the line ;
```

Every step type name is also a keyword (`IfCondition`, `Loop`, `SetVariable`, `Note` …): the formatter uses the bare type name plus `@with` for a step it cannot otherwise write.

Expression grammar

```
expr        = cond ;
cond        = or [ "?" cond ":" cond ] ;
or          = and { ( "or" | "||" ) and } ;
and         = not { ( "and" | "&&" ) not } ;
not         = ( "not" | "!" ) not | cmp ;
cmp         = add { ( "==" | "!=" | "<" | "<=" | ">" | ">=" ) add } ;
add         = mul { ( "+" | "-" ) mul } ;
mul         = unary { ( "*" | "/" | "%" ) unary } ;
unary       = ( "-" | "not" | "!" ) unary | pow ;      (* "not" as an operand binds like "-" *)
pow         = atom [ "^" unary ] ;
atom        = NUMBER | "true" | "false" | VARREF | call | "(" expr ")" ;
call        = IDENT { "." IDENT } "(" [ expr { "," expr } ] ")" ;
```

This expression grammar is informative — a summary of the precedence table in Expressions and Conditions (`^` binds tightest and is right-associative; `? :` binds loosest).

APPENDIX B

Keyword List

The words Simple Robot Script reserves. Keywords are case-insensitive; the formatter writes the spelling shown. A name that collides with a keyword (a point called `end`, `here` or `MoveL`) is always written in quotes.

Statement keywords

Every step, by its text keyword, with the page of its reference entry.

Keyword	Step	Category	Page
`# / ##`	Note	Logic & Flow	91
`$var = … / $list.op(…)`	Set Variable	Variables & Timing	92
`$var = … / $list.op(…)`	List	Variables & Timing	93
`AuxContinuous`	Aux Continuous Run	I/O & Aux Axes	104
`AuxEnable`	Aux Motor Enable	I/O & Aux Axes	106
`AuxMove`	Aux Move	I/O & Aux Axes	103
`AuxStop`	Aux Stop	I/O & Aux Axes	105
`break`	Break / Next	Logic & Flow	83
`call`	Call Routine	Logic & Flow	86
`CaptureImage`	Capture Image	Vision	112
`ClearLocal`	Clear Local	Motion Settings	78
`CncProgram`	CNC Program	Motion	71
`ConveyorAction`	Conveyor	I/O & Aux Axes	108
`exit`	End Program	Logic & Flow	87
`ExportData`	Export Data	I/O & Aux Axes	102
`goto`	Go To Label	Logic & Flow	89
`GripperAction`	Gripper	I/O & Aux Axes	107
`HttpReceive`	HTTP Receive	Communication	118
`HttpRequest`	HTTP Request	Communication	117
`if`	If Condition	Logic & Flow	79
`JumpJ`	Jump Joint	Motion	65
`JumpL`	Jump Linear	Motion	63
`label`	Label	Logic & Flow	88
`MoveC`	Move Arc	Motion	67
`MoveJ`	Move Joint	Motion	61
`MoveL`	Move Linear	Motion	59
`pause`	Pause Program	Logic & Flow	90

Keyword	Step	Category	Page
`plugin`	Plugin	Plugins	122
`prompt`	Prompt	Communication	120
`raise`	Raise Error	Logic & Flow	85
`ReadLoadCell`	Read Load Cell	I/O & Aux Axes	100
`repeat`	Loop	Logic & Flow	81
`return`	Return	Logic & Flow	87
`RotaryIndex`	Rotary Index	I/O & Aux Axes	109
`RunHoming`	Run Homing	Motion	69
`RunVision`	Run Vision	Vision	111
`SaveImage`	Save Image	Vision	113
`SavePosition`	Save Position	Motion	71
`SetBlendRadius`	Set Blend Radius	Motion Settings	75
`SetJointPosition`	Set Joint Position	Motion	70
`SetLocal`	Set Local	Motion Settings	77
`SetOutput`	Set Output	I/O & Aux Axes	98
`SetSpeedFactor`	Set Speed Factor	Motion Settings	75
`SetSpeedJ`	Set Speed (Joint)	Motion Settings	74
`SetSpeedL`	Set Speed (Linear)	Motion Settings	73
`SetTool`	Set Tool	Motion Settings	76
`StartBackground`	Start Background	Background	114
`StartRecording`	Start Recording	I/O & Aux Axes	101
`status`	Status Update	Communication	119
`StopBackground`	Stop Background	Background	114
`StopRecording`	Stop Recording	I/O & Aux Axes	102
`stopwatch`	Stopwatch	Variables & Timing	95
`switch`	Switch	Logic & Flow	80
`Tare`	Tare Load Cell	I/O & Aux Axes	99
`ThreadMove`	Thread Move	Motion	68
`try`	Try	Logic & Flow	83
`Wait`	Wait	Variables & Timing	96
`WaitForBackground`	Wait for Background	Background	115

Other reserved words

Flow	`if` `elif` `else` `end` `repeat` `forever` `as` `while` `for` `in` `switch` `case` `default` `try` `catch` `break` `continue` `return` `exit` `raise` `label` `goto` `call` `out` `pause` `status` `stopwatch` `prompt` `plugin` `unknown`
Declarations	`var` `computed` `param` `global` `persistent` `input` `monitor` `number` `bool` `string` `image` `stopwatch` `list` `point` `record`
Values and operators	`on` `off` `true` `false` `here` `grid` `stack` `toggle` `send` `receive` `and` `or` `not` `_`
Attributes	`@name` `@off` `@id` `@with` `@program`
Step type names	`MoveL` `MoveJ` `JumpL` `JumpJ` `MoveC` `ThreadMove` `RunHoming` `SetJointPosition` `CncProgram` `SavePosition` `SetSpeedL` `SetSpeedJ` `SetBlendRadius` `SetSpeedFactor` `SetTool` `SetLocal` `ClearLocal` `IfCondition` `Switch` `Loop` `LoopControl` `Try` `RaiseError` `CallRoutine` `Return` `EndProgram` `Label` `GoToLabel` `PauseProgram` `Note` `SetVariable` `ListOp` `StopwatchControl` `Wait` `SetOutput` `Tare` `ReadLoadCell` `StartRecording` `StopRecording` `ExportData` `AuxMove` `AuxContinuous` `AuxStop` `AuxEnable` `GripperAction` `ConveyorAction` `RotaryIndex` `RunVision` `CaptureImage` `SaveImage` `StartBackground` `StopBackground` `WaitForBackground` `HttpRequest` `HttpReceive` `StatusUpdate` `Prompt` `Plugin`

APPENDIX C

Quick Reference Cards

Card 1 — Program skeleton

```
#!srs 1
var count = 0                    # number (default type)
var ok: bool = off
var msg: string
var pts: list point
computed done: bool = $count >= 12

## Setup
SetTool Gripper
SetSpeedL 200, accel=800, decel=800

## Main
repeat 12 as $i
  …
end
status "Done: {$count}"
```

Card 2 — Motion

Text	Does
`MoveJ Home`	joint move to a saved point
`MoveL Pick1, speed=50`	straight line, own speed
`MoveL here, offset=(z=20)`	relative move
`MoveL Pick1, offset=(z=20), blend=5`	20 mm above, rounded corner
`MoveL $pts[$i]`	to a Points-list element
`MoveL "bin{$i}"`	to the point named by a template
`JumpL Place1, z=60`	lift to Z 60, travel, lower
`JumpL grid Tray[$r, $c], z=40`	to a grid cell (`Tray[$n]` by index)
`MoveL stack Bins[$i]`	to a stack slot
`MoveC Place1, via=Via1`	arc through a via point
`MoveL Pick1, until=$stb.in2, untilFlag=$hit`	stop when an input comes on
`SetSpeedFactor 25`	this program at 25 %
`SetLocal Fixture1` … `ClearLocal`	work in a local frame
`SavePosition $pts, mode=append`	remember where the robot is

Card 3 — Flow

```
if $a > 5 and $ok          switch $grade              try retry=2, delay=1s
  …                        case 1                       …
elif $a == 2                 …                        catch $msg, code=$code
  …                        case 2, 3                    …
else                         …                        end
  …                        default
end                          …                        raise "Empty", code=empty
                           end
repeat 3 as $i             while $stb.in1 == 0        for $v, $i in $vals
  …                          Wait 10ms                  …
end                        end                        end

break · continue · return · exit "msg" · pause · label again · goto again
call PickPart(slot=$i), out picked=$ok
```

Card 4 — Data and I/O

Text	Does
`$count = $count + 1`	assign
`$label = "Bin {$i}"`	assign text (template)
`$vals.append(5)` · `$vals.insert(0, 5)` · `$vals.removeAt($i)` · `$vals.clear()`	list edits
`$pts.append(x=1, y=2, z=3)` · `$pts.append()`	add a point / the current position
`$pts[$i].z = 10` · `$vals.sort(by=value, desc)` · `$copy.copyFrom($vals)`	more list edits
`stopwatch start $lap`	start / stop / reset a stopwatch
`SetOutput stb 1 on` · `SetOutput relay 2 on, pulse=250ms` · `SetOutput stb 2 toggle`	outputs
`Wait 500ms` · `Wait until $stb.in1, timeout=5s, onTimeout=error`	waits
`ReadLoadCell $g, device=Arduino_Nano, pin=2`	read a load cell
`GripperAction open` · `GripperAction close` · `GripperAction to 25`	gripper
`status "…"` · `status warning "…"` · `prompt "…", buttons=("Yes", "No"), result=$c`	operator

Card 5 — Expressions

Topic	Names
Arithmetic	`+ - * / % ^`
Compare	`== != < <= > >=` (→ 1 / 0)
Logic	`and or not`, `c ? a : b`
Numbers	`abs` `sign` `sqrt` `pow` `min` `max` `clamp` `round` `floor` `ceil` `trunc` `mod`
Angles (degrees)	`sin` `cos` `tan` `asin` `acos` `atan` `atan2` `deg` `rad`
Geometry	`hypot` `dist` `dist3` `inBox` `inCircle`
Lists	`len` `sum` `avg` `minOf` `maxOf`, `$l.length`, `$l[i]`, `$pts[i].x`
Other	`if(c, a, b)` `rand()` `map(x, a, b, c, d)` `lerp(a, b, t)`

Topic	Names
Text	`$s.contains("x")` `$s.startsWith` `$s.endsWith` `$s.indexOf` `$s.toNumber()` `$s.length`
Robot	`$robot.x` … `$robot.rz`, `$robot.moving`, `$robot.homed`, `$robot.speedOverride`
Program / time	`$program.runCount`, `$program.elapsedMs`, `$time.hour`, `$time_ms`
IO	`$stb.in1`, `$relay.3`, `$io.Arduino_Nano.Load_Cell`

Card 6 — Error codes

Catchable	Fatal
`waitTimeout` · `visionTimeout` · `visionFailed` · `httpFailed` · `httpTimeout` · `receiveTimeout` · `pluginFailed` · `pluginTimeout` · `pluginNotRunning` · `promptTimeout` · `pointNotFound` · `arcDegenerate` · `indexOutOfRange` · `userError` · your own `raise` codes	`motionFault` · `jointLimit` · `driverDisconnected` · `emergencyStop` · `userStop` · `unknownVariable` · `expressionSyntax` · `computedVariable` · `badListOperation`